

Neural Networks for Sequences

Table of Contents

Fuzzy Choquet Integration of Deep Convolutional Neural Networks for Remote Sensing	2
Deep Neural Networks for Structured Data	25
Granular Computing Techniques for Bioinformatics Pattern Recognition Problems in Non-metric Spaces	46
Multi-classifier-Systems: Architectures, Algorithms and Applications	71
Neural net architectures for temporal sequence processing	99
END	116

Fuzzy Choquet Integration of Deep Convolutional Neural Networks for Remote Sensing



Derek T. Anderson, Grant J. Scott, Muhammad Aminul Islam,
Bryce Murray and Richard Marcum

Abstract What deep learning lacks at the moment is the heterogeneous and dynamic capabilities of the human system. In part, this is because a single architecture is not currently capable of the level of modeling and representation of the complex human system. Therefore, a heterogeneous set of pathways from sensory stimulus to cognitive function needs to be developed in a richer computational model. Herein, we explore the learning of multiple pathways—as different deep neural network architectures—coupled with appropriate data/information fusion. Specifically, we explore the advantage of data-driven optimization of fusing different deep nets—GoogleNet, CaffeNet and ResNet—at a per class (neuron) or shared weight (single data fusion across classes) fashion. In addition, we explore indices that tell us the importance of each network, how they interact and what aggregation was learned. Experiments are provided in the context of remote sensing on the UC Merced and WHU-RS19 data sets. In particular, we show that fusion is the top performer, each network is needed across the various target classes, and unique aggregations (i.e., not common operators) are learned.

Keywords Fuzzy integral · Convolutional neural network · Remote sensing

D. T. Anderson (✉) · G. J. Scott · R. Marcum
Electrical Engineering and Computer Science, University of Missouri, Columbia,
MO, USA
e-mail: andersondt@missouri.edu

G. J. Scott
e-mail: grantscott@missouri.edu

R. Marcum
e-mail: ram7cd@missouri.edu

M. A. Islam · B. Murray
Electrical and Computer Engineering, Mississippi State University,
Starkville, MS, USA
e-mail: mi160@msstate.edu

B. Murray
e-mail: bjm260@msstate.edu

1 Introduction

We humans excel at many robust pattern recognition tasks in which computational systems can only perform well when limited in scope and constrained in operating environment. The human visual system is no exception. Humans develop at an early age a comprehensive visual processing and pattern recognition ability. Our vision allows us to process our physical environment (navigation) and facilitates many higher-level cognitive functions such as object classification and entity resolution. We accomplish this via a complex multistage visual system that begins with basic lightness and color receptors, then builds upon the perceived edges to derive shapes, spatial relationships, and eventually to organization of components into objects of interest – and this is before any higher level cognitive processing.

Deep neural network models follow a similar paradigm conceptually, extracting first edges and other simple geometric primitives in the lowest levels, then later mid-level assemblies of these primitives into visual concepts, which are then combined in higher-level layers as object components (blobs), that are eventually agglomerated into objects. These visual objects are agglomerated within fully connected neural layers for eventual classifications, which is an informational (cognitive) output. What deep architectures lack at the moment is the heterogeneous and dynamic capabilities of the human system, which is in part because a single architecture is not capable of the level of modeling and representation of the complex human system. Therefore, a heterogeneous set of pathways from sensory stimulus to cognitive function needs to be developed in a richer computational model. The model proposed in this chapter represents the learning of multiple pathways—as deep neural networks—coupled with appropriate information fusion. We feel fusion of the cognitive outputs (information) from multiple heterogeneous models (pathways) is the next step towards robust computational cognitive processing of visual, and visual-like, sensory data.

In general, *computational intelligence* (CI) is a branch of mathematics inspired by nature. Specifically, CI is associated with *neural networks* (NNs), *evolutionary algorithms* (EA) and *fuzzy set theory* (FST). NNs were established in 1943 by McCulloch and Pitts [1], FST was established in 1965 by Zadeh [2] and EAs were made popular by Holland in the early 1970s [3] (but arguably have roots going back as far as Turing in 1950). The point is, CI has existed in one form or another since the advent of *artificial intelligence* (AI). In this chapter, we focus on the intersection of NNs and FST for pattern recognition. In the last decade, substantial interest and effort has gone into *deep learning* (DL), a re-branding of NNs. This shift has forced us to re-address fundamental questions like; should humans design features (the classical approach to pattern recognition) or is a machine better at this task? Empirically, DL has more-or-less unanimously topped the charts in many domains (e.g., natural language processing [4, 5], vision [6–10], remote sensing [11–14]). However, while DL has generated great excitement, much remains to be explored and explained. In this chapter, we focus on the specific question of how to perform decision-level fusion of DL networks.

DL can be viewed as a generalization of the classical pattern recognition pipeline—e.g., pre-processing, feature extraction (selection and/or reduction), classification and post-processing. In some settings this is now being called *shallow learning* because there are only a few “layers” in the pattern recognition pipeline. In the context of computer vision, DL can also be decomposed into levels; “low” (e.g., signal/image analysis via convolution), “mid” and “high” (more AI than signal processing, e.g., MLP classification). In the extreme, DL is nothing more than a series of operations that transform data to decisions. The point is, fusion can (and often does) take place at different levels in pattern recognition/DL. For example, keeping with the fusion nomenclature of the *Joint Directors of Laboratories* (JDL) [15], some fusion algorithms do *signal-in-signal-out* (SISO), whereas others do *feature-in-feature-out* (FIFO) and *decision-in-decision-out* (DIDO). If we regard DL as a SISO process (e.g., SI = image and DO = class label), then it can be decomposed into its corresponding SISO, SIFO, FIFO, FIDO, DIDO (and combinations therein). In summary, fusion is not as simple as “cram data into a DL and let it do its thing”.

Herein, we restrict our analysis to *deep convolutional neural networks* (DCNNs) [6–10, 16, 17], versus *auto encoders* (AEs) [18–22], *deep belief nets* (DBNs) [23, 24], etc., for sake of discussion tractability. The reality is, we still know little-to-nothing about fundamental DL fusion questions such as; (i) how/where is fusion currently happening, (ii) based on our current set of neurons/transformations, what is mathematically expressible and what is not (but should be), (iii) how should we be performing fusion at different levels, (iv) how do we address heterogeneity with respect to semantics and/or uncertainty across data/information sources, and (v) how do we explain what fusion is doing (aka *explainable AI* (XAI)), to list a few. Independent of DL, fusion is a complicated topic that often means different things to different people in different fields (and even within the same field). Fusion is a wealth of challenges wrapped up into one term. Fusion ranges from data association (e.g., finding a one-to-one mapping between pixels in one sensor to pixels in another) to the mathematics of aggregation (specific functions/operators). In general, the idea of fusion is to obtain a “better” result than if we only used the individual inputs. However, better is not a well defined concept. In some applications, better might mean taking a set of inputs and reducing them into a single result that can be more efficiently or effectively used for visualization. Better could also refer to obtaining more desirable properties such as higher information content or lower conflict. In areas like pattern recognition, better often refers to some desirable property like more robust and generalizable solutions (e.g., classifiers). Regardless of the task at hand or the particular application, fusion is a core tool at the heart of numerous modern scientific thrusts.

In this chapter, we make the following contributions. First, we discuss two approaches for heterogeneous DCNN architecture fusion; density-based imputation and full *Choquet integral* (ChI) learning (per neuron and “shared weight”). Second, we outline indices for introspection and information theoretic indices to understand the capacity and integral (moving us closer to a so-called XAI solution versus black

box solution). Third, we demonstrate and analyze these ideas on remotely sensed data. Last, we provide open source code at www.derektanderson.com/FuzzyLibrary and www.github.com/scottgs/fi_library.

2 Deep Convolutional Neural Networks

To date, the AE [18–20], CNN [6–10, 16, 17], DBN [23, 24] and *recurrent NNs* (RNNs) [25, 26] are the most mainstream DLs. However, other DL approaches exist, e.g., deep inference nets ([27] Verma et al. Takagi-Sugeno-Kang deep net), deconvolution CNNs (specifically transpose matrix convolution) [28–30] and morphological shared weight neural networks [31, 32]. Herein, we focus on the CNN, which is by far the most employed and often the highest performer. With respect to the CNN, a number of architectures have been explored to date, e.g., AlexNet [7], GoogLeNet [17], VGGNet [33] and their derivatives. These architectures can be downloaded and extended (training, evaluation, visualization) via open source libraries like TensorFlow [34], CaffeNet [35], and MatConvNet [36]. The fundamental challenges of which architecture, how deep versus wide, hyperparameter tuning, what neuron types, how to transfer a DL from one domain to another (transfer learning [37]), and other questions are unanswered. Also, numerous challenges exist; e.g., lack of training data volume (and variety), class imbalance, dimensionality (spatial, temporal and spectral), explainable DL (what did the DL learn, versus a black box), to name a few. While DL has sparked a revolution in computer vision, pattern recognition and AI in general, an overwhelming number of theoretical and applied questions remain ripe for exploration.

In general, most CNNs consist of combinations of the following operations (see Fig. 1). First, let the input to the system, O_0 , be a three dimensional data cube of size $N_0 \times M_0 \times D_0$; where N_0 and M_0 are spatial dimensions and D_0 is the temporal or spectral dimensionality (e.g., RGB imagery has $D_0 = 3$). **(Convolution)** The backbone of a CNN is filtering via convolution. Filtering can take a number of meanings, e.g., enhancement, denoising or detection. Convolution specifics include factors like (i) stride (spatial and/or spectral/temporal “jumps”) and (ii) padding (if no padding is used then the spatial dimension shrinks). **(Pooling)** Pooling is often applied to reduce spatial dimensionality—and combat challenges related to affine variation, noise, etc. Most often, average and max pooling are used. **(Activation)** Nonlinearity is also typically applied, in the form of a function like hyperbolic tangent ($\tanh$), sigmoid, or ReLU ($\text{ReLU}(x) = \max(0, x)$). **(Training Techniques)** In order to combat factors like sensitivity to parameter selection and overtraining, methods like dropout [38], regularization [39] and/or batch normalization [40] (addresses internal covariate shift and vanishing gradients) are often used. Beyond architecture, there are factors like GPU acceleration [41], training (e.g., *stochastic gradient descent* (SGD) [42], SGD with momentum [43, 44], AdaGrad [45], RMSProp [46] and ADAM [47]). The reader can refer to [39] for additional mathematical and algorithm

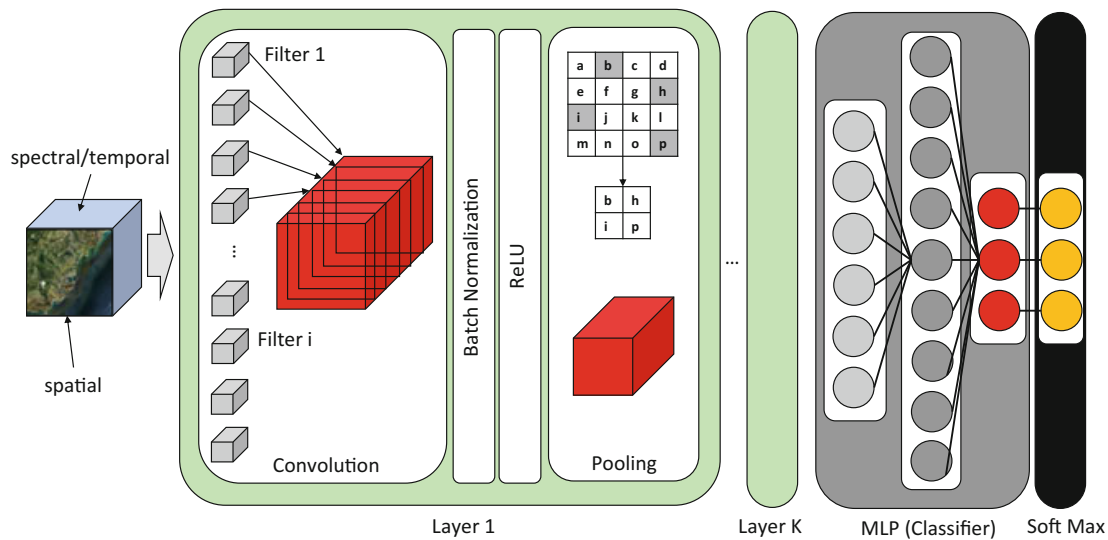


Fig. 1 Example CNN. Input is a 3D cube (x-y spatial, z spectral), green layers consist of subset of convolution (morphology, etc.), pooling (average, max, etc.), batch normalization (or other method to mitigate overfitting, vanishing gradients, internal covariate shift, etc.) and nonlinear function (e.g., ReLU activation). The output of the green layers are typically fed to a MLP and optional post-processing steps (e.g., soft max normalization)

details related to CNNs. The reader can also refer to [48] for a recent survey of DL in remote sensing (theory, applications and open challenges).

The idea of FST in NNs is not new. The reader can refer to the work of Pal and Mitra [49] for neuro-fuzzy pattern recognition. Pal, Mitra, and others (e.g., Keller and the fuzzy perceptron [50]), explored a variety of topics such as fuzzy min-max networks, fuzzy MLPs, and fuzzy Kohonen networks. In terms of aggregation, a few FST works have been explored to date. In 1992 [51], Yager put forth the *ordered weighted average* (OWA) [52]—which technically is a *linear combination of order statistics* (LCOS) since the weights are real-valued numbers (versus sets)—neuron. In 1995, Sung-Bae utilized the OWA for NN aggregation (at the decision/output level) [53]. In 1995, Sung-Bae et al. also explored the fuzzy integral, the Sugeno fuzzy integral not Sugeno's fuzzy ChI, for NN aggregation [54]. Specifically, they used the Sugeno λ -fuzzy measure (FM) and the densities were derived using their respective accuracy rates on training data. In 2017 [55], we (Scott et al.) used the Sugeno and ChIs for DCNN fusion. Specifically, Scott et al. used transfer learning to adapt GoogLeNet, AlexNet and ResNet50 from camera imagery to remote sensing imagery. Scott then applied different aggregations—fuzzy integral, voting, arrogance, and weighted sum—to these DCNNs. Scott's fusion was based on the Sugeno λ FM and the densities were (i) set to the normalized classifier accuracies and (ii) a GA learned the densities (which led to higher performance).

3 Fuzzy Measure and Fuzzy Integrals

The ChI has been successfully demonstrated in numerous applications; e.g., explosive hazard detection [56–58], computer vision [59], pattern recognition [60–64], remote sensing [65], multi-criteria decision making [66, 67], forensic anthropology [68–70], control [71], multiple kernel learning [56, 72–75], multiple instance learning [76], ontologies [77], missing data [78], and most relevant to the current chapter, DL [55]. The ChI is a nonlinear aggregation function that is parameterized by the FM (aka capacity). Countless mathematical variations of the fuzzy integral have been put forward for different reasons; e.g., address different types (i.e., real-valued, interval-valued, set-valued) of uncertainty in the integrand and/or measure, limit the number of input interactions for tractability, etc. Herein, we focus on and succinctly review just the real-valued discrete (finite X) ChI for DCNN fusion.

3.1 Discrete (Finite X) Fuzzy Measure

Let $X = \{x_1, x_2, \dots, x_N\}$ be N sources, e.g., experts, sensors, or in the case of this chapter, DCNNs. The first action we face is how to assign “worth/utility” to different subsets of DCNNs. For example, the well-known backbone of calculus on real-valued domains is the Lebesgue measure; which coincides with length, area and hypervolume. However, when X is a discrete domain, e.g., set of DCNNs, what is the corresponding “measure”? In [59], Keller et al. first investigated the idea of using the fuzzy integral for pattern recognition. A FM is a function, μ , on the power set of X , 2^X , which satisfies (1) (boundary condition) $\mu(\emptyset) = 0$ (often $\mu(X) = 1$ ¹) and (2) (monotonicity) if $A, B \subseteq X$ and $A \subseteq B$, then $\mu(A) \leq \mu(B)$.

3.2 Discrete (Finite X) Fuzzy Integral

The FM models important “interactions” (e.g., subjective worth, statistical correlation, etc.) between different source subsets. The input provided by our sources is $\{h(\{x_1\}), h(\{x_2\}), \dots, h(\{x_N\})\}$. The fuzzy integral is a way to combine the integrand (h) information relative to the FM (μ). Let $h(\{x_i\}) \in \mathfrak{R}^{\geq 0}$ be the data from source i . The discrete (finite X) Sugeno FI is²

¹If $\mu(X) < 1$, properties like idempotency and boundedness are not guaranteed.

²Due to the maximum (t-conorm) and minimum operators (t-norm), the Sugeno FI does not actually generate any possible number between the minimum and maximum of the inputs. Instead, it selects one of the FM or input values, i.e., at most one of $2^N + N$ values.

$$\int_S h \circ \mu = S_\mu(h) = \bigvee_{i=1}^N (h(\{x_{\pi(i)}\}) \wedge \mu(A_i)), \quad (1)$$

where π is the permutation $h(\{x_{\pi(1)}\}) \geq h(\{x_{\pi(2)}\}), \dots, \geq h(\{x_{\pi(N)}\})$, $A_i = \{x_{\pi(1)}, \dots, x_{\pi(i)}\}$ and $\mu(A_0) = 0$. The discrete (finite X) ChI is³

$$\int_C h \circ \mu = C_\mu(h) = \sum_{i=1}^N h(\{x_{\pi(i)}\}) [\mu(A_i) - \mu(A_{i-1})]. \quad (2)$$

Since the ChI is a parametric aggregation function, once the FM is determined the ChI turns into a specific operator. For example: if $\mu(A) = 1, \forall A \in 2^X \setminus \emptyset$, the ChI becomes the maximum operator; if $\mu(A) = 0, \forall A \in 2^X \setminus X$, we recover the minimum; and if $\mu(A) = \frac{|A|}{N}$, we recover the mean. Each of these cases can be viewed as constraints or simplifications on the FM (and therefore the ChI). In general, the discrete ChI has $N!$ unique input sortings and each yields a linear convex sum operator.

3.3 Data-Driven Optimization

The first challenge we must confront is, where do we get the FM (μ) from? One option is to have an expert specify it. However, this is not practical (assuming the expert could even meaningfully assign values to the interactions) as the number of inputs (e.g., DLs) increases. Another option is we can specify or try to learn the worth of just the singletons (the densities). From there, a number of formulas can be used to impute (fill in) the missing variable values. Popular approaches include the Sugeno λ -FM and the S-Decomposable FM [79]. However, while convenient, most often we do not obtain the desired values for variables that we need. With respect to pattern recognition, the focus of this chapter, another route is to learn it from data. Next, we review one way to learn the FM, and therefore the ChI, in the context of DIDO for DL. However, the reader can refer to [80] for an efficient learning method with only data-supported variables and [81] for a review of alternative FM/ChI learning methods.

We quickly summarize one way to learn the full FM/ChI (see [82] for full mathematical explanation). Let $O = \{\mathbf{h}_j, y_j\}$, $j = 1, 2, \dots, M$, be M training examples; where $\mathbf{h}_j$ is the j -th instance with data from N inputs and y_j is the ground-truth for $\mathbf{h}_j$. The sum of squared error for training dataset O is

³The ChI is used frequently for various reasons; e.g., it is differentiable [62], for an additive (probability) measure it recovers the Lebesgue integral, it yields a wider spectrum of values between the minimum and maximum (versus the discrete and relatively small number of values that the Sugeno FI selects from), etc.

$$E(O, \mathbf{u}) = \sum_{j=1}^M e^j = \sum_{j=1}^M (\mathbf{c}_j^T \mathbf{u} - y_j)^2 = \|D\mathbf{u} - \mathbf{y}\|_2^2, \quad (3)$$

where $\mathbf{u} = [\mu(\{x_1\}), \dots, \mu(\{x_N\}), \mu(\{x_1, x_2\}), \mu(\{x_1, x_3\}), \dots, \mu(X)]$ (lexicographic vector of size $2^N - 1$), $\mathbf{c}_j$ holds the coefficients of $\mathbf{u}$ for observation $\mathbf{h}_j$, e.g., for $N = 3$ and $h(\{x_2\}) \geq h(\{x_1\}) \geq h(\{x_3\})$,

$$c = [0, h(\{x_2\}) - h(\{x_1\}), 0, h(\{x_1\}) - h(\{x_3\}), 0, 0, h(\{x_3\})],$$

$D = [\mathbf{c}_1 \ \mathbf{c}_2 \ \dots \ \mathbf{c}_M]^T$ (full dataset), $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_M]^T$, and $\|\cdot\|_2$ is norm-2 operation, $\mathbf{u}$. The regularized SSE optimization problem is

$$\min_{\mathbf{u}} f(\mathbf{u}) = \|D\mathbf{u} - \mathbf{y}\|^2 + \beta v(\mathbf{u}), \quad (4)$$

where $\beta \in \mathbb{R}^{\geq 0}$ (regularization constant, which balances the “cost” (or penalty) of obtaining minimum function error relative to our desire to have minimal model complexity) and $v(\mathbf{u})$ is an index of model complexity (e.g., k-additive and Mobius, Gini-Simpson, ℓ_p -norm, etc. [83]), subject to the FM boundary and monotonicity conditions (see [82] for how to pack the constraints into a linear algebra expression), which can be solved via quadratic programming. Full code (including how to build C) can be found at www.derektanderson.com/FuzzyLibrary and [www.github.com/scottgs/fi_library](https://github.com/scottgs/fi_library).

3.4 Explainable AI (XAI) Fusion

It is one thing to train a network and another to understand it! In this subsection, we highlight FM and ChI indices for the purpose of *explainable AI* (XAI).⁴ XAI is an attempt to explain the inner operations of pattern recognition for purposes like describing it to others for domain knowledge transfer, trust, etc. The Shapley index addresses the importance or worth of each input (aka DL),

$$\Phi_{\mu}(i) = \sum_{K \subseteq X \setminus \{i\}} \zeta_{X,1}(K) (\mu(K \cup \{i\}) - \mu(K)), \quad (5)$$

where $\zeta_{X,1}(K) = \frac{(|X|-|K|-1)!|K|!}{|X|!}$, $K \subseteq X \setminus \{i\}$ denotes all proper subsets from X that do not include source i . The Shapley value of μ is the vector $\Phi_{\mu} = (\Phi_{\mu}(1), \dots, \Phi_{\mu}(N))^t$ and $\sum_{i=1}^N \Phi_{\mu}(i) = 1$. The Shapley index can be interpreted as the average amount of *contribution* of source i across all coalitions. The next index informs us

⁴Go to www.derektanderson.com/FuzzyLibrary and [www.github.com/scottgs/fi_library](https://github.com/scottgs/fi_library).

about how two inputs interact with one another (aka what advantage is there in combining DLs). The interaction index (Murofushi and Soneda [84]) between i and j is

$$\mathcal{I}_\mu(i, j) = \sum_{K \subseteq X \setminus \{i, j\}} \zeta_{X,2}(K) (\mu(K \cup \{i, j\}) - \mu(K \cup \{i\}) - \mu(K \cup \{j\}) + \mu(K)), \quad (6)$$

where $\zeta_{X,2}(K) = \frac{(|X|-|K|-2)!|K|!}{(|X|-1)!}$, $\mathcal{I}_\mu(i, j) \in [-1, 1]$, $\forall i, j \in \{1, 2, \dots, N\}$. A value of 1 (respectively, -1) represents the maximum complementary (respective redundancy) between i and j . Refer to [85] for further details about the interaction index, its connections to game theory and interpretations. Grabisch later extended the interaction index to the general case of any coalition [86],

$$\mathcal{I}_\mu(A) = \sum_{K \subseteq X \setminus A} \zeta_{X,3}(K, A) \sum_{C \subseteq A} (-1)^{|A \setminus C|} \mu(C \cup K), \quad i = 1, \dots, N, \quad (7)$$

where $\zeta_{X,3}(K, A) = \frac{(|X|-|K|-|A|)!|K|!}{(|X|-|A|+1)!}$. Equation (7) is a generalization of both the Shapley index and Murofushi and Soneda's interaction index as $\Phi_\mu(i)$ corresponds with $\mathcal{I}_\mu(\{i\})$ and $\mathcal{I}_\mu(i, j)$ with $\mathcal{I}_\mu(\{i, j\})$.

The above indices are focused strictly on the FM. A different fundamental type of question is what “type” of aggregation is the ChI performing? Answering this question helps us understand how the DL information is being combined (e.g., in an optimistic, pessimistic, expected value like fashion, etc.). In [87], we established an index (D_1) to measure the degree to which a given FM/ChI is an maximum operator. In the following, we discuss the FM in terms of its underlying lattice structure. Let “layer k ” (measure defined on sets of cardinality k) be denoted by $L(k)$, e.g., $L(1) = \{\mu(\{x_1\}), \mu(\{x_2\}), \mu(\{x_3\})\}$ for $N = 3$. Next, let $\mathbf{W} = \frac{[\frac{1}{N}, \dots, 1]}{\sum_{i=1}^N \frac{i}{N}}$ be weights (penalty or costs) for each layer and

$$D_1(\mu) = \sum_{k=1}^1 \frac{\mathbf{W}(k)}{2} (T_1 + T_4) + \left[\sum_{k=2}^N \frac{\mathbf{W}(k)}{3} (T_1 + T_2 + T_4) \right], \quad (8)$$

$T_1 = 1 - \left(\frac{\sum_{I \in L(k)} \mu(I)}{|L(k)|} \right)$, $T_2 = \left(\frac{\sum_{I \in L(i)} \mu(I)}{|L(k)|} - \frac{\sum_{J \in L(k-1)} \mu(J)}{|L(k-1)|} \right)$, $T_3 = \frac{\sum_{I \in L(k)} \mu(I)}{|L(k)|}$ and $T_4 = \frac{\sum_{I \in L(k)} (\mu(I) - T_3)^2}{|L(k)| - 1}$. A value of $D_1 = 0$ means the ChI is the maximum operator. The distance of a learned capacity to a minimum operator (D_2), mean (D_3) and LCOS (D_4), for $\mathbf{W}_2 = \frac{[1, \dots, \frac{1}{N-1}]}{\sum_{i=1}^{N-1} \frac{i}{N-1}}$, is

$$D_2(\mu) = \sum_{k=1}^1 \frac{\mathbf{W}_2(k)}{2} (T_3 + T_4) + \left[\sum_{k=2}^{N-1} \frac{\mathbf{W}_2(i)}{3} (T_3 + T_2 + T_4) \right], \quad (9)$$

$$D_3(\mu) = \frac{1}{2^N - 2} \sum_{k=1}^{N-1} \sum_{I \in L(k)} \left| \mu(I) - \frac{k}{N} \right|, \quad (10)$$

$$D_4(\mu) = \frac{1}{N-1} \sum_{k=1}^{N-1} \sqrt{T_4}. \quad (11)$$

4 DCNN Fusion Based on Fuzzy Integration

The focus of this chapter is fusing different state-of-the-art DCNN architectures. However, the procedures outlined are applicable to other neural inputs (see Fig. 2).

4.1 DCNN Architectures Used for Fusion

The first NN used herein for fusion is CaffeNet [35], which is a derivative of AlexNet with similar structure, except that the order of pooling and normalization is reversed to

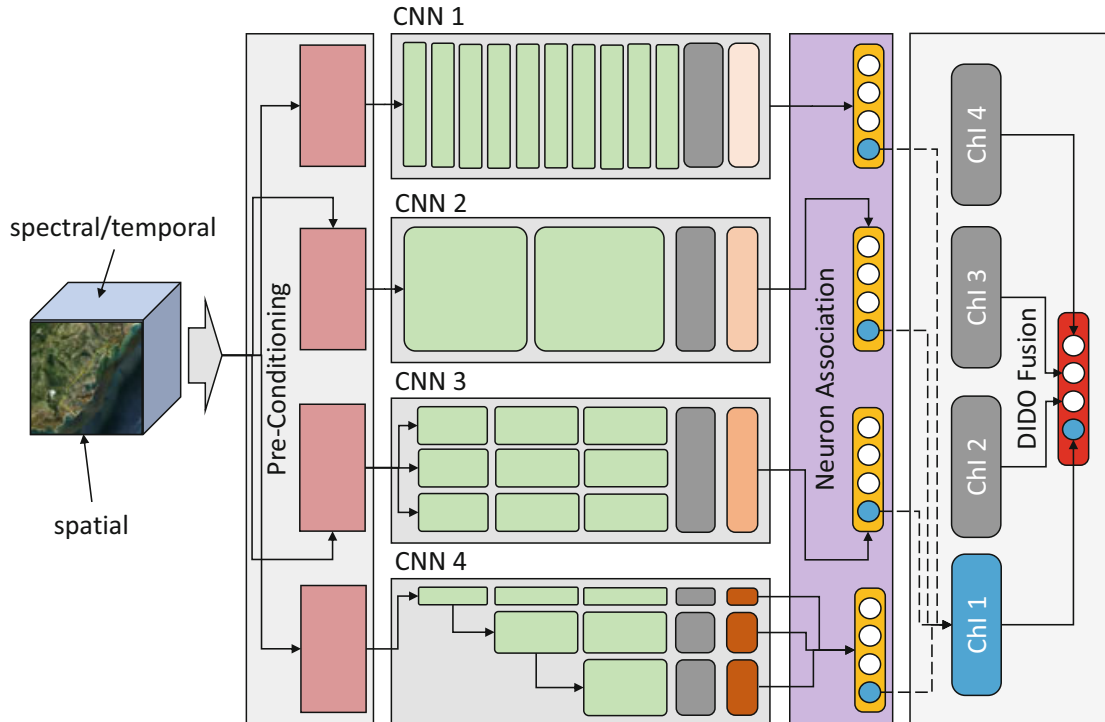


Fig. 2 Illustration of DIDO DCNN fusion. Note, many possibilities exist; e.g., variations in architecture, pre-conditioning/transforms (e.g., conversion to frequency analysis versus spatial domain, band selection or grouping, etc.), training data, etc. Next, neuron mapping/association is required followed by aggregation. Herein, a different fusion operator is learned per output neuron (versus shared fusions/weights)

reduce learnable parameters. CaffeNet contains five convolutional feature extraction steps and three fully connected layers for classification. Classification is performed with two fully connected inner product layers and a final soft-max layer for the network output. The output of soft-max layer is effectively a classification vector. CaffeNet represents the most simple and shallow of our DL investigated herein.

GoogLeNet [17] is a much deeper NN than CaffeNet—it has 27 parameterized layers. Because of this network depth, GoogLeNet has three classification outputs at various stages of the network to facilitate error back propagation. GoogLeNet’s novel *inception layer* processes the input with max-pooling, 1×1 , 3×3 , and 5×5 convolutions simultaneously in a feature extraction step, and the outputs are concatenated as the layer output to achieve a multi-scale feature extraction. Using multiple convolutions at each stage follows the intuition that features from different kernel scales can be extracted and processed at the same time, thereby extracting multi-scale visual features. GoogLeNet is from a family of networks commonly referred to as *inception networks*.

ResNet [88] is a collection of DCNN architectures inspired by VGGNet [33]. In both ResNet and VGGNet, the primary kernels used to construct the convolution layers are 3×3 . The architecture design incorporates the following rules to govern their structure. First, if the output of the feature map is the same, then the same number of 3×3 convolutional layers will be used. Second, if the output of the feature map is halved, then it will use twice as many 3×3 convolutional kernels. The ResNet architectures employ residual connections that bypass two or more convolution layers at a time, allowing error to better propagate backward through the network. These are commonly referred to as *residual networks*, and here the ResNet50 and ResNet101 architectures are used within our experimental design. These networks have 50 and 101 feature extraction steps, respectively.

4.2 Transfer Learning, Neuron Association and Conditioning

If we design a set of custom DCNNs then it is trivial to ensure a bijection (one-to-one and onto) output neuron mapping. However, if existing community pretrained DCNNs (GoogLeNet, AlexNet, etc.) are leveraged—a task encountered frequently in practice—then this is not guaranteed. One way to resolve the one-to-one mapping task is to replace and retrain the DCNN classification layers per the labels for the task at hand. This is a type of transfer learning that keeps the feature layers intact. In [89], we (i) replaced and retrained the classification layers and we also (ii) updated the feature weights (e.g., convolution layers). Thus, we built custom classifiers for remote sensing of aerial imagery based on a network initialized by ground-perspective RGB imagery. In addition, data augmentation via rotation and image flipping was applied as well. However, we remark that other avenues exist; e.g., one could manually resolve the mapping or use an automated method based on an ontology. Regardless, using multiple custom or pretrained networks of different architectures raises another

question; are the outputs numerically (e.g., all in $[0, 1]$) and semantically “to scale” (e.g., does a (e.g., $a = 0.5$) in domain i map to a in the other domains). One way to mitigate this issue in practice is to add a soft max normalization (aka normalized exponent function) layer after the raw neuronal output layer. For example, if η is the soft max output for neuron o_j then the soft max function is $\eta(o_j) = \frac{e^{o_j}}{\sum_{n=1}^N e^{o_n}}$. Thereby, we bound the domain of input for the subsequent fusion layer of our pattern recognition system, ensure the data across networks and neurons is well conditioned.

4.3 Imputation: λ -FM ChI

The first fusion approach explored here is to exploit our knowledge about the performance of the individual DCNNs on training data [54, 55]. A classical approach to obtaining the remaining $2^N - 2 - N$ FM values (beyond the densities) is the Sugeno λ -measure. For sets $A, B \subseteq X$, such that $A \cap B = \phi$,

$$\mu_\lambda(A \cup B) = \mu_\lambda(A) + \mu_\lambda(B) + \lambda\mu_\lambda(A)\mu_\lambda(B), \quad (12)$$

for some $\lambda > -1$. In particular, Sugeno showed that λ can be found by solving

$$\lambda + 1 = \prod_{i=1}^N (1 + \lambda\mu(x_i)), \lambda > -1, \quad (13)$$

where there exists exactly one real solution such that $\lambda > -1$. Some advantages of the Sugeno λ -measure include its simplicity, the N densities can be more tractable to acquire, fewer number of parameters can help address overfitting (versus using the full 2^N variables), and it is a probability measure when $\lambda = 0$. However, there is no guarantee in practice that the values that it imputes are what we actually need. Simply speaking, more information or a different imputation formula may be required; e.g., the S-Decomposable imputation formula, $\mu(A) = \bigvee_{i \in A} (\mu(x_i))$ (where $\bigvee$ is a t-conorm). Algorithm (1) describes how to use the Sugeno λ -measure to fuse a set of pretrained DLs based on individual performance for density.

Algorithm 1 λ -FM Based Imputation of ChI from Pre-Trained DCNNs

INPUT: DL_i - N DCNNs (B neurons each); $\bar{O}$ - labeled training data

1. Run each DCNN on $\bar{O}$, get *overall accuries* (OA); $a_{b,i} \in [0, 1]$ (i.e., performance of DL i on class b)
2. Assign the i th density its corresponding OA; i.e., $\mu_{\lambda_b}(x_i) = a_{b,i}$
3. Find λ_b (using $\{\mu_{\lambda_b}(\{x_1\}), \dots, \mu_{\lambda_b}(\{x_N\})\}$) for Sugeno λ -FM (solve Eq. (13))
4. Recursively calculate $\mu_{\lambda_b}(A)$, $\forall A \in 2^X \setminus \{\{x_1\}, \dots, \{x_N\}\}$, using the densities and λ_b (Eq. (12))

OUTPUT: B full fuzzy measures - $\{\mu_{\lambda_1}, \dots, \mu_{\lambda_B}\}$

4.4 Optimization Approach: Learning the Full ChI

As stated in Sect. 4.3, there is no guarantee that imputation from densities results in the input interactions that we desire (and thus results in an appropriate aggregation operator). Algorithm (2) shows how to use quadratic programming for acquisition of the full FM for DIDO fusion of DCNNs (Algorithm (3) is how to learn a single “shared” FM to be applied to all neurons). Thus, training data is directly used to learn these crucial interactions—which means better selection of appropriate aggregation operator. However, as we discuss in [80], this process can lead to a big boost in performance but it is not without flaw. Specifically, in [80] we show that training data only typically supports a subset of FM variables. In return, we put forth an extended optimization of the ChI by (1) identifying which variables are supported by data, (2) optimizing just those variables and then (3) looking at imputation methods to infer the value of data unsupported variables based on application specific criteria. We do not have space to go into depth about the extension here, the reader can refer to [80] for full details.

Algorithm 2 Learn a Full FM/ChI Per Class for a Set of Pre-Trained DCNNs

INPUT: DL_i - N DCNNs (B neurons each); $\bar{O}$ - training data; β - regularization

1. Per class/output neuron (b), run each instance ($1 \leq j \leq |\bar{O}|$) through each DCNN (i); get $h_j^b(x_i)$ terms
2. Per neuron (b), construct the individual D_b from the $h_j^b(x_i)$ terms
3. Run B independent QPs (on the D_b respectively); yielding $\{\mu_1, \dots, \mu_B\}$

OUTPUT: B full fuzzy measures - $\{\mu_1, \dots, \mu_B\}$

Algorithm 3 Learn a Single “Shared Weight” Full FM/ChI for Pre-Trained DCNNs

INPUT: DL_i - N DCNNs (B neurons each); $\bar{O}$ - training data; β - regularization

1. Per class/output neuron (b), run each instance ($1 \leq j \leq |\bar{O}|$) through each DCNN (i); get $h_j^b(x_i)$ terms
2. Per neuron (b), construct the individual D_b from the $h_j^b(x_i)$ terms
3. Use QP to solve $(\|D_1 \mathbf{u} - \mathbf{y}_1\|_2^2 + \dots + \|D_B \mathbf{u} - \mathbf{y}_B\|_2^2 + \beta v(\mathbf{u}))$; yields μ

OUTPUT: Full fuzzy measure - μ

5 Experiments

In this chapter, two benchmark remote sensing datasets suitable for classification tasks of objects or land-cover/land-use are used. Remote sensing data represents a significant pattern recognition challenge. As can be seen in Figs. 3 and 9 below, the variability and complexity of overhead imagery is immense. The visual cues exist at multiple levels: fine-scale (e.g. airplane shapes, vehicle presence, etc.) to



Fig. 3 Sample image chips from the 21 class UCM benchmark dataset, each 256×256 pixels approximately 0.3m *ground sampling distance* (GSD) spatial resolution. Classes in left-to-right, top-down order: 1 *agricultural*, 2 *airplane*, 3 *baseball diamond*, 4 *beach*, 5 *buildings*, 6 *chaparral*, 7 *dense residential*, 8 *forest*, 9 *freeway*, 10 *golf course*, 11 *harbor*, 12 *intersection*, 13 *medium residential*, 14 *mobile home park*, 15 *overpass*, 16 *parking lot*, 17 *river*, 18 *runway*, 19 *sparse residential*, 20 *storage tanks*, and 21 *tennis court*. In Sect. 5.1, neuron indices are used instead of text descriptions for sake of compactness

large-scale (e.g., road way configurations in overpasses versus intersections versus freeway). In fact the entire field of photo-interpretation revolves around developing human expertise in this pattern recognition task. For each of the datasets herein DCNNs were trained using the techniques in [89], including transfer learning and data augmentation via rotation and flipping. The trained DCNNs are then used in a locked state, i.e., no further learning happens in DL during the fusion stage. The training of the DCNNs are done in five-fold, cross validation manner; such that we have 5 sets of 80% training and 20% testing for both datasets. Per DCNN fold, three-fold CV fusion is used (due to limited data).

5.1 UC Merced (UCM) Dataset

The UC Merced (UCM) benchmark dataset [90, 91] has been used in a wide range of remote sensing research, including prior work in classification of objects and land-cover such as [55, 89, 92]. Figure 3 shows exemplar image chips from each class of the UCM dataset. The dataset includes 21 classes that are a mix of objects (airplane, baseball diamond, etc.) and landcover (beach, chaparral, etc.). We see that some classes, e.g., harbor and parking lot, are complex compositions of sub-entities (boats and vehicles); while others are general structural patterns of shapes (e.g., intersection and baseball diamonds).

Table 1 is the result of fusion on the UCM dataset. First, we see that aggregation outperforms no aggregation (i.e., the individual DCNNs) in four out of five folds.

Table 1 Fusion results for the UCM dataset

	Method									
	ChI Per Neuron	ChI Shared	SLFM Shared	CNet	GNet	RNet 50	RNet 101	Max	Avg	Min
Fold 1	0.979	0.977	0.984	0.957	0.957	0.985	0.973	0.978	0.981	0.976
Fold 2	0.991	0.994	0.993	0.964	0.983	0.978	0.988	0.993	0.994	0.993
Fold 3	0.994	0.990	0.996	0.971	0.985	0.992	0.988	0.996	0.996	0.998
Fold 4	0.992	0.996	0.996	0.988	0.980	0.983	0.988	0.996	0.992	0.998
Fold 5	0.989	0.985	0.989	0.976	0.973	0.983	0.980	0.989	0.989	0.986

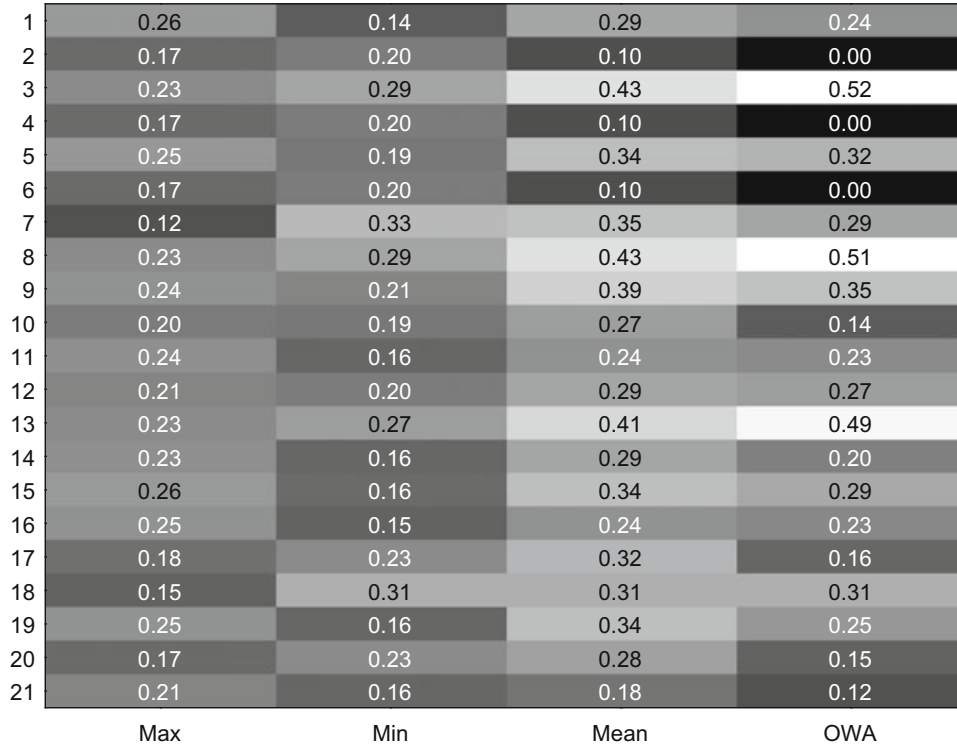


Fig. 4 Color coded matrix showing the distances obtained using the four reported indices of introspection ($D_1(\mu)$ to $D_4(\mu)$) relative to the learned full ChI per neuron on fold 1 of the UCM dataset. y-axis is the neuron index (see Fig. 3) and x-axis is the distance measure. Neurons two, four and six are OWA operators (but not min, max or mean like)

Second, we see that min, max and average (basic aggregation operators) do well in comparison to the ChI. However, these three operators are specific instances of the ChI, which informs us that there are challenges with variety and thus generalizability of this particular data set (otherwise they should have been selected). Next, it is interesting to see that the shared weight fusion solutions do as well as they do. It is our suspicion—something to be explored in future work—that a shared FM for the ChI helps combat overfitting. It is also our suspicion—again, subject of future work—that while the Sugeno λ -FM would not be our first choice, it might also help combat overfitting as it has just N parameters versus the otherwise $2^N - 1$. However, the performance of the individual DCNNs (which were used as the densities) are so high that ultimately this forces the Sugeno λ -FM to more-or-less be the maximum operator.

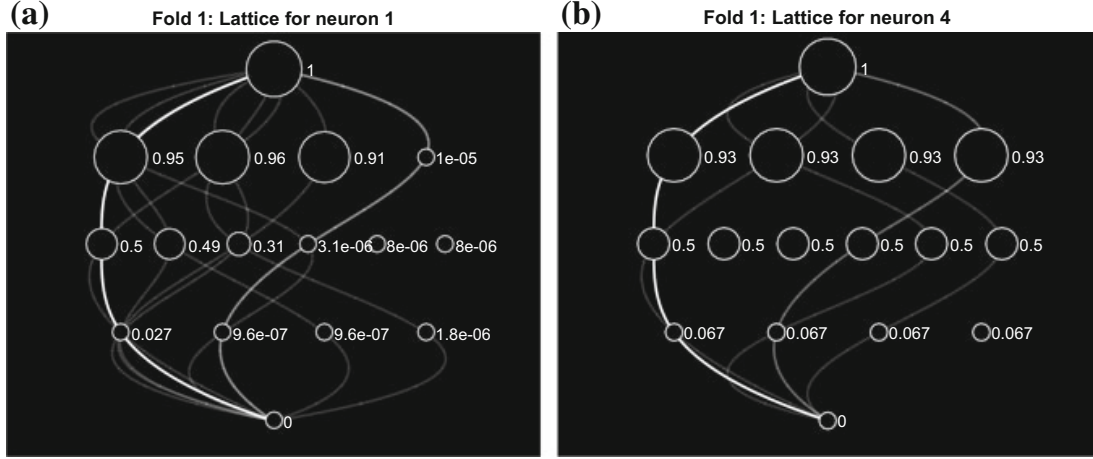


Fig. 5 Example of two full FMs for the **a** first and **b** fourth neuron in fold 1 of the UCM dataset. “Layer” l (from bottom to top) in the image denotes FM variables with cardinality l . Thus, layer 0 (bottom node) is the empty set, the next layer is the singletons, top is $\mu(X)$, etc. Each variable is presented in lexicographic order, i.e., layer 2 is $\{x_1, x_2\}$, $\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_2, x_3\}$, $\{x_2, x_4\}$ and $\{x_3, x_4\}$. The nodes are also drawn size-wise proportional to their value (a minimum size and maximum was specified to make them still show up for 0 valued variables). In addition, the “paths” drawn indicate the visitation frequency (the brighter the line, the higher the visitation) for the test data in fold 1. Furthermore, the fourth neuron learned an OWA with weights $(0.067, 0.433, 0.43, 0.07)^t$ – a trimmed mean operator. Conversely, neuron one is more complex to decode. It does not reduce into a single compact description like an OWA. However, we can view it in terms of the $N!$ walks (possible sorts). Since the $h(\{x_1\}) \geq h(\{x_2\}) \geq h(\{x_3\}) \geq h(\{x_4\})$ is encountered frequently, we decode and analyze its weights. The linear convex sum weights for the ChI of this walk (sorting) are $(0.027, 0.473, 0.45, 0.05)$ respectively. Thus, it is a weighted average of GoogLeNet and ResNet50. This analytic process can be repeated for the other $N! - 1$ walks if desired

Next, Fig. 4 gives us a feel for what type of aggregation strategy is being used for the 21 classes. Again, the max, min and mean are all OWAs, so we can start first with analyzing column four. There are three neurons (2, 4 and 6–i.e., airplane, beach and chaparral) that learned an OWA. The other neurons have learned something more unique, which helps justify the inclusion of the ChI versus say a simpler operator (see Fig. 5(a)). At that, none of the learned OWAs are that similar to our extreme markers of max (a t-conorm or union like operator), min (a t-conorm or intersection like operator) or average (an expected value like operator). For example, Fig. 5(b) shows one of these OWA operators, which breaks down into a trimmed mean operator.

Last, Fig. 6 shows the FM and Shapley values. While it is more-or-less impossible to read individual values in these plots, they show that there is no consensus in values nor importance of DCNNs. Meaning, different output neurons (classes) appears to use these different DCNNs in different ways. Furthermore, Fig. 7 shows the corresponding interaction index values. These values also reinforce the complex interplay and back-and-forth exchange of complementary, independent and redundant information between DCNNs across output neurons (classes). In total, the combination of

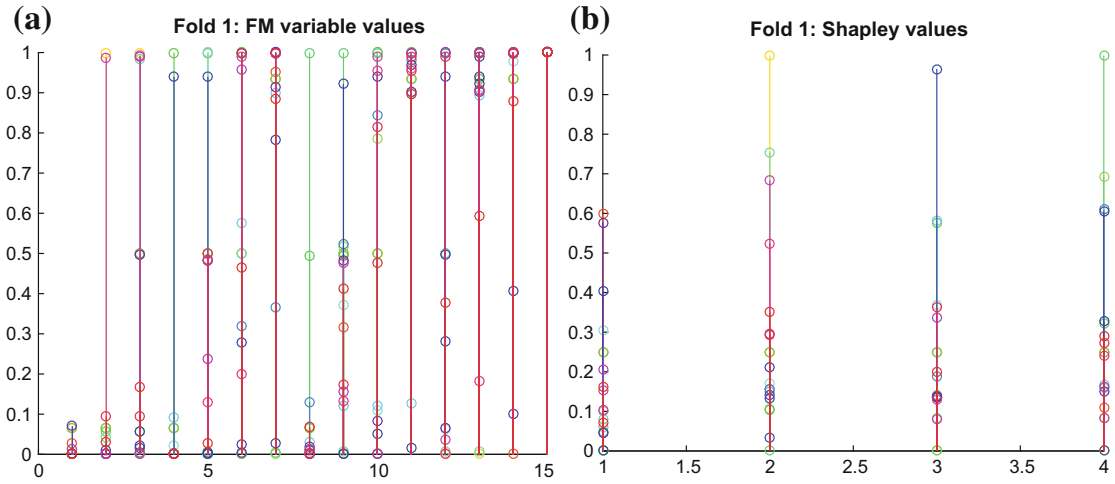


Fig. 6 Learned full ChI per neuron on fold 1 of the UCM dataset. **a** Plot of the $2^4 - 1$ binary encoded FM variables, i.e., for $N = 3$ the order is $(x_1, x_2, x_{12}, x_3, x_{13}, x_{23}, x_{123}, x_4, x_{14}, x_{24}, x_{124}, x_{34}, x_{134}, x_{234}, x_{1234})$. This plot shows the agreement/disagreement of variable values across the 21 neurons. If all neurons required the same fusion then each x-axis location would have a single convergent set of circles (FM variable values). However, we can see that each x-axis location (FM variable) has for the most part significant variation (outside the CaffeNet density). **b** Plot of the 4 neuron Shapley index values across the 21 neurons. Again, this plot shows the variety of values learned. With respect to individual output neurons, some NNs could be eliminated. However, across the 21 neurons, no NN can be eliminated (we would expect to see approximately all zero values for that Shapley if so)

analysis of underlying aggregation function, importance of individual DCNNs and their pair-wise interaction behavior help the claim that performance appears to be improving due to diversity in the way these DCNNs operate. This is in line with our intuition about these DCNNs based on the ways their architectures were created.

Last, Fig. 8 shows example images *missed* by our fusion approach. As the reader can visually verify, these examples are extreme and represent incorrectly labeled or fundamentally ambiguous labels. We would not expect fusion to be able to fix this type of problem. At that, it is hard to say that the DCNNs should have got these, as a human might just as well mistaken them.

5.2 WHU-RS19 (RSD) Dataset

The WHU-RS19 (RSD) dataset is composed of 600×600 pixel, JPEG compressed images [93]. This class includes 19 classes, and approximately 50 chips per class. This imagery was screen scrapped from Google Earth, and therefore they are of variable spatial resolutions. Figure 9 shows exemplar image chips from each class of the RSD benchmark dataset. Similar to the UCM dataset, this dataset is a mixture of landcover and objects within the image chips. Table 2 shows the result of fusion, Fig. 10 are the indices of introspection, Fig. 11 are example lattices, Fig. 12 are the FM and the Shapley indices and Fig. 13 are example interaction indices. Overall, we see

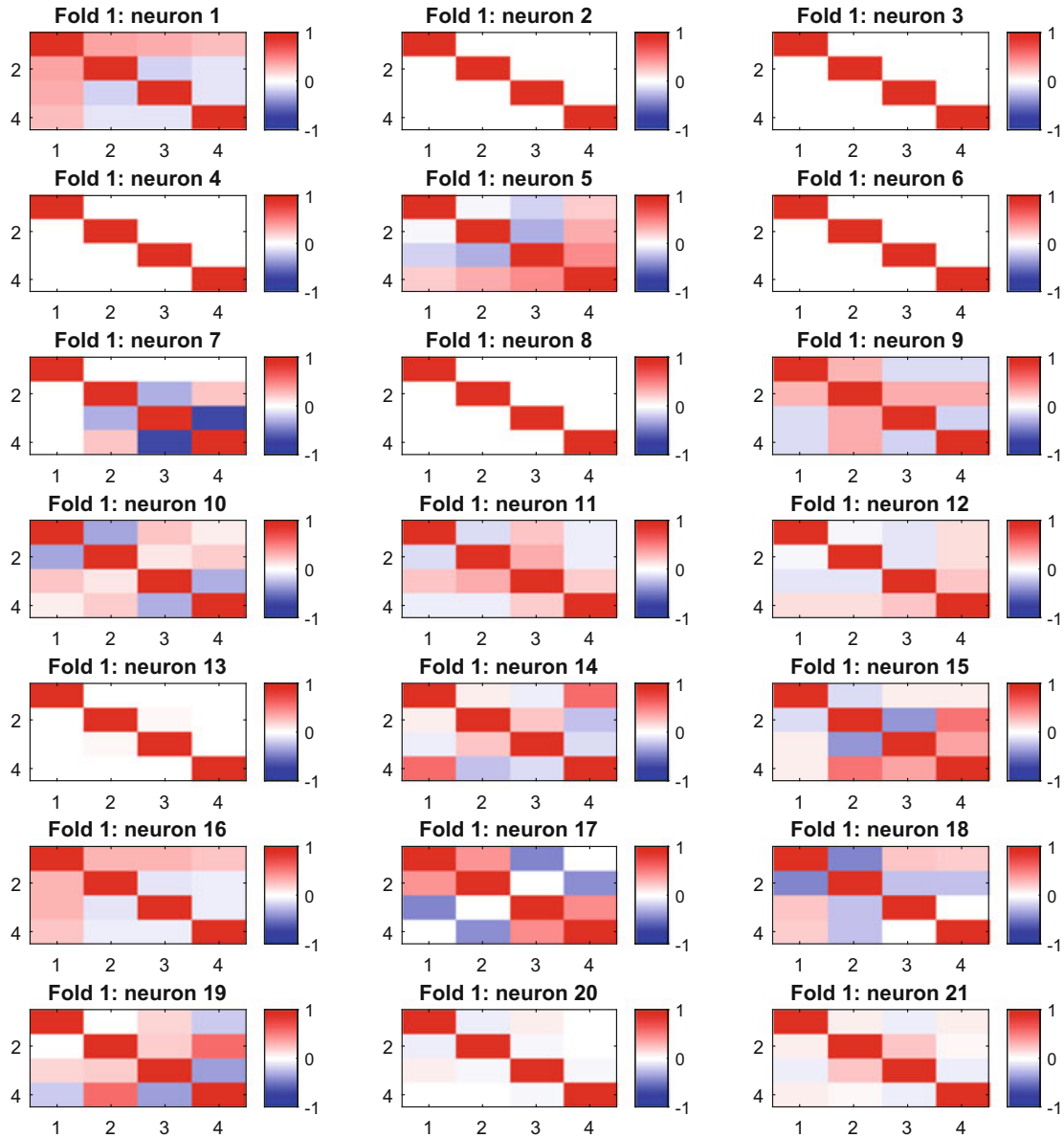


Fig. 7 Interaction index values for the learned full ChI per neuron on fold 1 of the UCM dataset. Index 1 is CaffeNet, 2 is GoogLeNet, 3 is ResNet50 and 4 is ResNet101. Consider neuron 1. CaffeNet has positive interactions (complementary information) with the other three NNs (0.37, 0.34 and 0.3 respectively). On the other hand, GoogLeNet has negative interaction values (redundancy) with the ResNet NNs (-0.19 and -0.1 respectively). The two ResNet NNs have a negative interaction index of -0.12 . Also, in neuron 7, CaffeNet has approximately a zero interaction index with the other NNs (independence), whereas GoogLeNet has a value of -0.29 with ResNet50 and a positive interaction value of 0.22 with ResNet101. Last, ResNet50 and ResNet101 have a large negative interaction index of -0.72 with each other

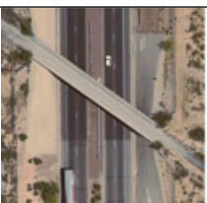
Misclassified Image	Other images from correct class	Predicted Exemplar
 (a)	 Dense Residential	 Mobile Home Park
 (b)	 Intersection	 Overpass
 (c)	 Medium Residential	 Dense Residential
 (d)	 Golf Course	 Forest
 (e)	 Dense Residential	 Medium Residential

Fig. 8 Five images *missed* by the fusion framework; **a** dense residential misclassified as mobile home park, **b** (incorrectly labeled) intersection misclassified as overpass (correct label), **c** medium residential misclassified as dense residential, **d** (incorrectly labeled) golf course misclassified as forest, and **e** dense residential misclassified as medium residential



Fig. 9 Sample image chips from the 19 class RSD benchmark dataset, each 600×600 pixels of various spatial resolution. Classes in left-to-right, top-down order: 1 *airport*, 2 *beach*, 3 *bridge*, 4 *commercial area*, 5 *desert*, 6 *farmland*, 7 *football field*, 8 *forest*, 9 *industrial area*, 10 *meadow*, 11 *mountain*, 12 *park*, 13 *parking lot*, 14 *pond*, 15 *port*, 16 *railway station*, 17 *residential area*, 18 *river*, and 19 *viaduct*. In Sect. 5.2, neuron indices are used instead of text descriptions for sake of compactness

Table 2 Fusion results for the RSD dataset

	Method								
	ChI Per Neuron	ChI Shared	SLFM Shared	CNet	GNet	RNet50	Max	Avg	Min
Fold 1	0.989	0.991	0.991	0.982	0.977	0.988	0.991	0.991	0.991
Fold 2	0.992	0.984	0.992	0.978	0.994	0.989	0.987	0.992	0.987
Fold 3	0.984	0.992	0.979	0.955	0.988	0.966	0.979	0.979	0.979
Fold 4	0.983	0.983	0.983	0.983	0.960	0.971	0.983	0.988	0.987
Fold 5	0.998	1.00	1.00	0.977	0.994	0.994	1.00	1.00	1.00

the same general trend (as the UCM dataset). Namely, (i) aggregation outperforms no aggregation in general and (ii) there are challenges with variety (and therefore generalizability) in the RSD data set as well.

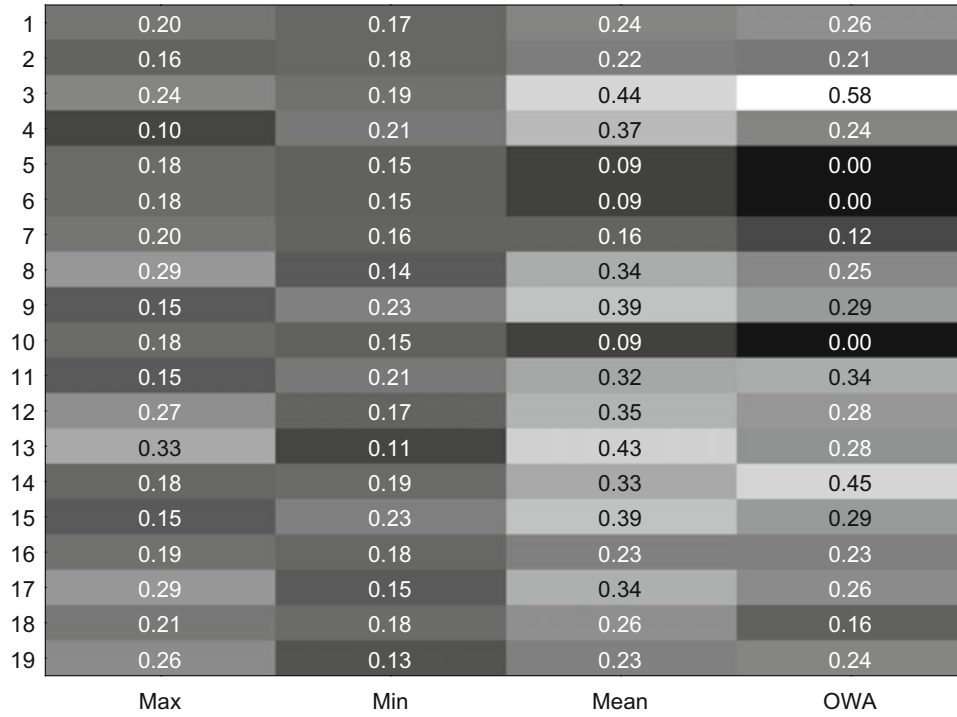


Fig. 10 Color coded matrix showing the distances obtained using the four reported indices of introspection ($D_1(\mu)$ to $D_4(\mu)$) relative to the learned full ChI per neuron on fold 1 of the RSD dataset. y-axis is the neuron index (see Fig. 9) and x-axis is the distance measure

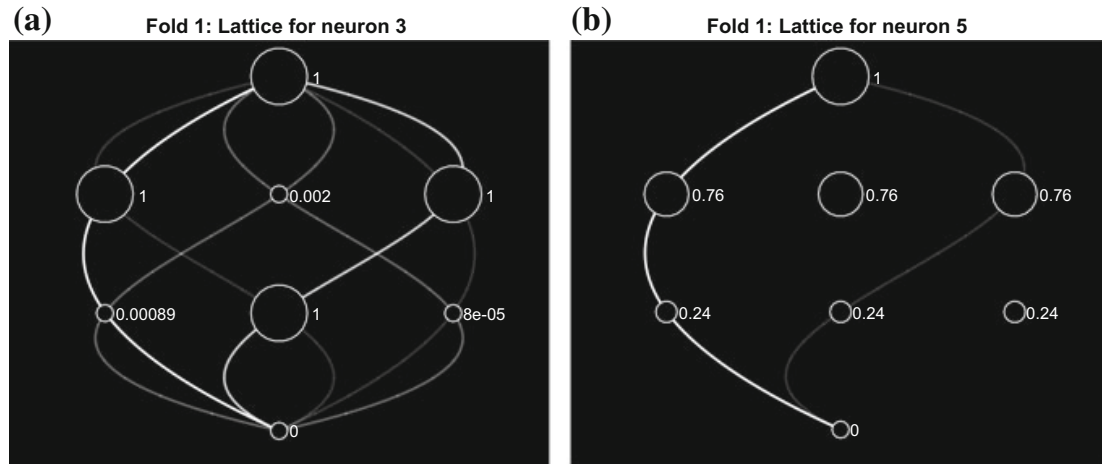


Fig. 11 Example FMs for fold 1 of the RSD dataset. Neuron three is for all intents and purposes a binary FM (see [94] for a formal characterization of binary FMs, the resultant FI and efficient ways of representing and learning such a function). For binary FMs, the Sugeno FI and the ChI are mathematically equivalent [94]. The FI is acting like a “dynamic maximum operator” with respect to FM variables that have a value one—or conversely a “dynamic minimum” with respect to zero valued FM variables. For example, if $h(\{x_1\}) \geq h(\{x_2\}) \geq h(\{x_3\})$ (aka CaffeNet is more confident than GoogLeNet followed by ResNet) then we take the output of GoogLeNet. However, if $h(\{x_2\})$ (GoogLeNet) is the most confident then we take its input. This reasoning can be followed to get similar stories for the other $N! - 2$ walks. Another interesting observation of neuron 3, versus neuron 5, is a slightly more diverse visitation (walk) pattern

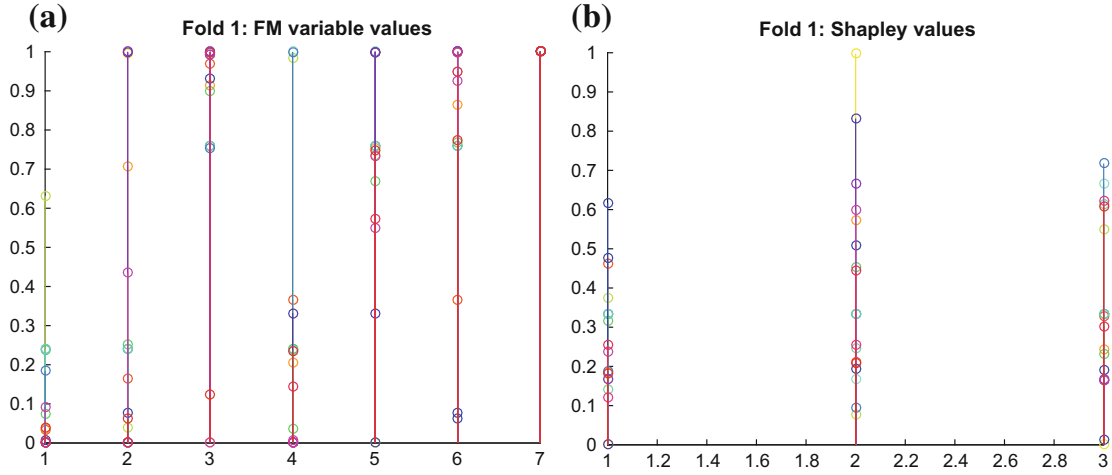


Fig. 12 Binary encoded $2^3 - 1$ FM variables and the 3 Shapley index values for the nineteen output neurons in the RSD dataset. As demonstrated in the UCM dataset, great variability exists in FM variable and Shapley values for these nineteen output neurons

6 Conclusion and Future Work

In summary, this chapter outlined a data-driven method for optimizing Choquet integral-based fusion of heterogeneous deep convolutional neural networks for pattern recognition in remotely sensed data. To the best of our knowledge, no one has previously learned the full fuzzy Choquet integral for fusing neural networks, just density-based fuzzy measures. This chapter brought together state-of-the-art advancements in two important parts of computational intelligence; fuzzy set theory and neural networks. Specifically, CaffeNet, GoogLeNet, ResNet50 and ResNet101 were fused at the per-output-neuron and with respect to a single “shared weight” solution. In a strive for explainable AI, versus black box solutions, different indices of introspection of the Choquet integral and information theoretic indices of the fuzzy measure were highlighted for analysis of the final deep learning solution. These indices showed us that there does appear to be diversity in these different heterogeneous DCNNs. Two benchmark remote sensing datasets were used, UCM and RSD, and our fused results showed improvement over the individual deep learners. However, this data set and DCNNs were saturated and therefore limited data (both volume and variety) existed for training fusion. Last, analysis of mislabeled imagery from fusion revealed incorrectly labeled data and ambiguous image chips that would lead to a human mislabeling imagery.

While encouraging, more research (theory and application) is needed. In future work, we will migrate our Choquet integral solution into a strictly neural representation for optimization and speed. Furthermore, we will move away from DIDO and explore fusion neurons at various layers in the network. We will also investigate what types of neural inputs should be fed to DIDO fusion; e.g., combinations of deep and shallow, different convolution map scales, etc. Future work will also include simultaneously learning the DCNNs and our fusion operators (they are learned inde-

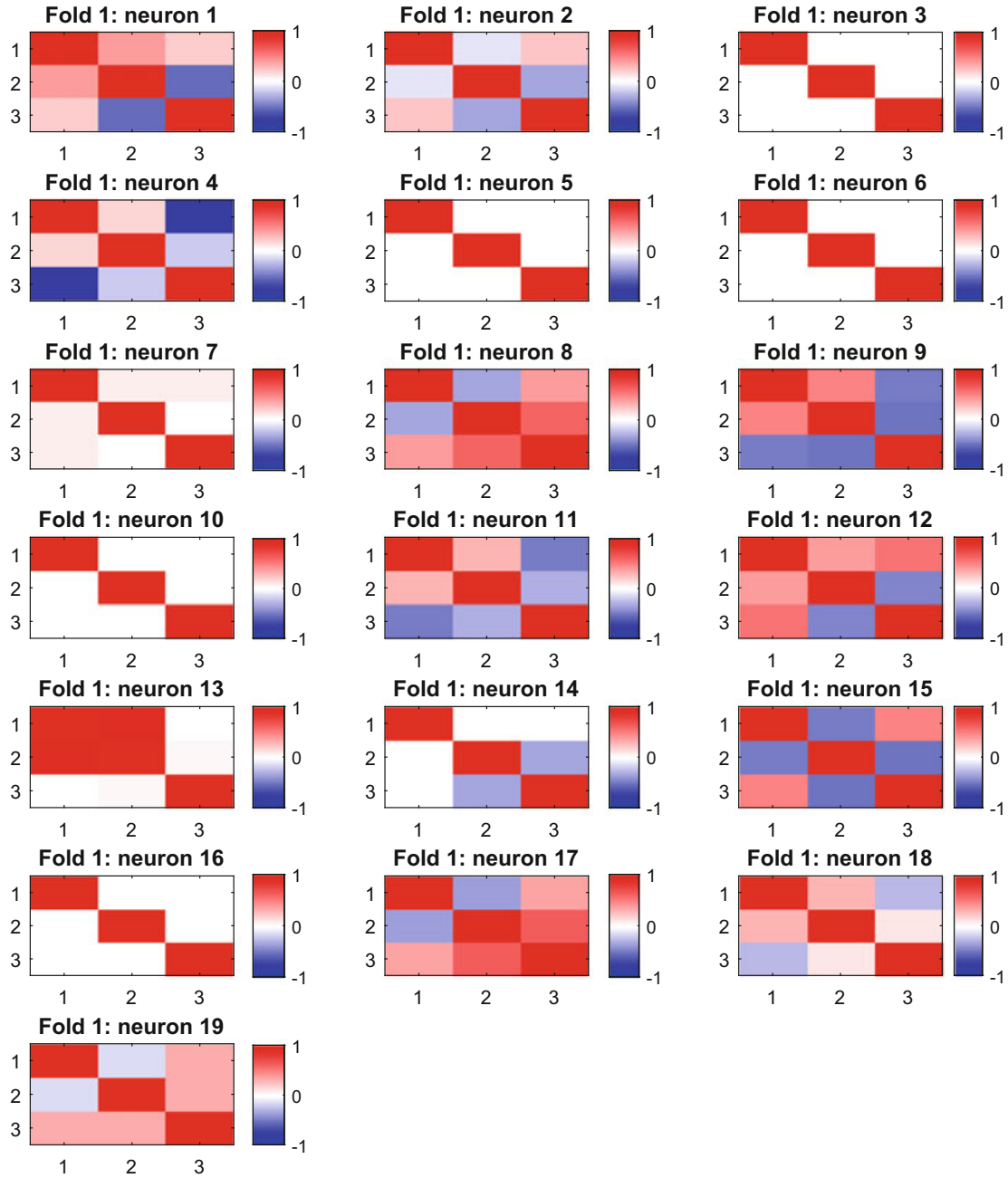


Fig. 13 Interaction index values for nineteen outputs in the RSD dataset. Index 1 is CaffeNet, 2 is GoogLeNet and 3 is ResNet50

pendently herein). Last, we will look to use our explainable AI methods to make improvements to the fusion and DCNNs, manually as well as possibly using them directly computationally to promote diversity and/or aid in the design of our networks.

Deep Neural Networks for Structured Data



Monica Bianchini, Giovanna Maria Dimitri, Marco Maggini
and Franco Scarselli

Abstract Learning machines for pattern recognition, such as neural networks or support vector machines, are usually conceived to process real-valued vectors with predefined dimensionality even if, in many real-world applications, relevant information is inherently organized into entities and relationships between them. Instead, Graph Neural Networks (GNNs) can directly process structured data, guaranteeing universal approximation of many practically useful functions on graphs. GNNs, that do not strictly meet the definition of deep architectures, are based on the unfolding mechanism during learning, that, in practice, yields networks that have the same depth of the data structures they process. However, GNNs may be hindered by the long-term dependency problem, i.e. the difficulty in taking into account information coming from peripheral nodes within graphs — due to the local nature of the procedures for updating the state and the weights. To overcome this limitation, GNNs may be cascaded to form layered architectures, called Layered GNNs (LGNNs). Each GNN in the cascade is trained based on the original graph “enriched” with the information computed by the previous layer, to implement a sort of incremental learning framework, able to take into account progressively further information. The applicability of LGNNs will be illustrated both with respect to a classical problem in graph-theory and to pattern recognition problems in bioinformatics.

Keywords Graph neural networks · Deep neural networks · Protein structure prediction

M. Bianchini (✉) · M. Maggini · F. Scarselli
Department of Information Engineering and Mathematics,
University of Siena, Siena, Italy
e-mail: monica@diism.unisi.it

M. Maggini
e-mail: maggini@diism.unisi.it

F. Scarselli
e-mail: franco@diism.unisi.it

G. M. Dimitri
Department of Computer Science, University of Cambridge, Cambridge, UK
e-mail: gmd43@cam.ac.uk

1 Introduction

The formal representation of objects is a key issue in pattern recognition problems. Actually there are two fundamental ways for implementing a pattern recognition system, namely using statistical or structural approaches. In the statistical framework, objects are described by feature vectors, whereas structural approaches exploit symbolic data structures, such as strings, trees, or graphs. Both approaches show pros and cons. If, in fact, evaluating the similarity between two entities is easily defined in vectorial spaces, and can be efficiently obtained, nevertheless this representation is not suited to explicitly describe relationships between the different subparts of objects. On the other hand, graphs can easily describe relations among subparts of complex data, but with a significant increase in computational complexity. For instance, the problem of evaluating the identity of two feature vectors has a linear complexity (w.r.t. the vector dimension), whereas testing the isomorphism between graphs is just an exponential problem [1].

Anyway, in several pattern recognition applications, data can actually be suitably represented in form of structures, f.i. in image processing [2] and bioinformatics [3]. Indeed, for these applications, information is inherently represented by atomic entities, sharing some common properties and dependent on each other, as described by relationships that encode their mutual influence and interactions. Contextual, hierarchical or causal connections between parts of a given pattern provide crucial information. The intrinsic subsymbolic nature of these tasks prevents a natural representation of data by vectors, since the feature extraction procedure is problem-dependent, heuristic, computationally expensive, and may also produce loss of information. The way in which structured data should be processed can neither be simply related to the “symbolic” information (normally collected in labels, describing the atomic entities) nor to the “subsymbolic” organization of entities. Hence, the processing scheme for these pattern representations should be designed in order to take into account both the entity labels and their relationships. In particular, when dealing with patterns encoded as general graphs, we may consider *graph-focused* and *node-focused* applications. In the graph-focused framework, the decision to be taken is related to the entire structure, while for node-focused problems, an output is expected for each node in the graph. For instance, a chemical compound can be modeled by a graph G , where nodes represent atoms or small molecules, whereas edges describe covalent bonds. The problem of estimating the probability that a molecule is mutagenic, i.e. may induce a particular disease [4], is graph-focused, since it is a property of the compound as a whole, not of a specific atom. Instead, the prediction of secondary structure elements and disordered regions (or loops) in proteins [5] is configured as a node-focused task. In computer vision, images can be modeled by Region Adjacency Graphs (RAGs) [6], with labeled nodes denoting homogeneous regions (from the visual point of view) and arcs defining their adjacency relationship. In this context, the localization and detection of an object is a node-focused task [7], while image classification is graph-focused [8].

Since the late '90s, different neural network models for processing graphs have been proposed, both for supervised and unsupervised tasks [9]. In Recursive Neural Networks (RNNs) [10], subsymbolic and symbolic information collected within graphs, and respectively related to the topological organization of nodes and to their labels, is encoded into a set of state variables, associated with each node. The states are updated dynamically following the graph topology, and used to calculate the outputs. RNN models are only able to process directed positional acyclic graphs (DPAGs) and are normally used to address graph-focused problems. Instead, Graph Neural Networks (GNNs) [11] process input data encoded as general labeled graphs. GNNs are trained based on a supervised learning algorithm that incorporates, into the error function, a criterion aimed at enforcing a contractive dynamics, to stabilize the learning procedure also for cyclic input graphs. Moreover, GNNs are able to naturally model both node-focused and graph-focused functions. Therefore, GNNs have recently been applied to many forefront applications. In particular, in bioinformatics, GNNs were employed for modelling quantitative structure-activity relationship problems, i.e. for the prediction of the mutagenicity and of the biodegradability of molecules [12]. Besides, GNNs were successfully exploited for object localization in images [13] and, with respect to web applications, for page ranking [14], sentence extraction [15], and document clustering and classification [16].

Unfortunately, all recursive models are plagued by the long-term dependency pathology, i.e. they struggle in processing deep structures, because of the local nature of the learning procedure. Actually, the error contribution, which vanishes during backpropagation, prevents a sensible update of the weights and, therefore, of the states, related to far nodes. In fact, practical issues are detected when dynamic neural network models are expected to learn tasks in which the relevant events in the input/output sequence span long intervals or, equivalently in the case of graphs, the dependencies involve nodes connected by a “long” path in the graph structure. This concept has been theoretically formalized in [17] in the case of recurrent networks, proving that the system is unable to latch temporal information robustly, since the gradient contribution due to information t time steps away vanishes as t increases. It is worth noting that the long-term dependency problem conflicts with the idea that deep networks, composed of a large number of layers, are necessary to cope with complex applications, being able to implement, with a reduced number of neurons, functions that cannot be realized with shallow architectures [18]. In fact, controlling the number of resources (in terms of neurons) needed to solve a given problem means also stemming the exponential growth in computational complexity (both in space and in time) of the learning procedure. In this perspective, many recent studies have been focused on how to train networks layer by layer, mixing unsupervised and supervised algorithms [19].

In this chapter, we describe a stacked architecture, referred to as Layered GNN (LGNN), that is composed of a cascade of GNNs, each of which takes in input the original graph and the information computed by the previous GNN in the cascade. LGNNs, that can be properly defined as deep architectures, are trained layer by layer, exploiting the provided targets and training each network in the cascade to correct the solution computed by the previous one. Basically, each GNN is expected to focus

only on those patterns that were misclassified by the previous GNNs in the hierarchy, thus implementing a kind of incremental learning, that is progressively enriched by taking into account information from further nodes.

An experimental evaluation is reported, based on synthetic and real-world datasets. Synthetic data are used as a benchmark for a central task in graph theory, i.e. finding cliques — fully connected subgraphs — inside graphs. Instead, publicly available datasets come from the bioinformatic field and are related to the prediction of the secondary structure of proteins, and to the classification of chemical compounds with respect to mutagenicity. The reported results are promising and guarantee a significantly improved accuracy with respect to standard GNNs. Finally, as a pattern recognition application of the clique search problem, we describe how to model the protein surface in order to identify the onset of transient pockets — typically involved in protein–protein and protein–drug interactions — for drug design. Indeed, transient pockets are cavities that can appear and disappear on a protein surface, often containing active sites where pharmaceutical agents can be anchored. Standard methods for protein 3D structure prediction are unable to identify and localize transient pockets, due to their short persistence and complex pattern, whereas connectionist models seem to capture sufficient information in the vicinity of the putative pocket to predict its appearance (dimension and duration). In particular, in drug discovery, it often happens that several molecules of apparently unrelated structures are active on the same drug target. In fact, if the interatomic distances among a subset of atoms in a drug match the distances among a similar-sized subset in a second drug, a putative shared pharmacophore exists, which guarantees the interaction of both drugs with the target. Constructing a complete edge-weighted graph representing each drug (with atoms as vertices and edge weights that stand for interatomic distances), the problem of interest is reformulated as that of finding the Maximum Common Subgraph (MCS) between them, where the MCS is defined as the largest subset of atom pairs that have matching distances. To solve the MCS problem, couples of protein graphs are first converted to *correspondence* graphs, identifying all graph-to-graph pairs of elements with matching distances, on which the application of a clique-detection algorithm efficiently identifies the MCS. Recently, the described approach has been also extended to the identification of complementary surfaces on proteins, a core task within the problem of computational docking of biomolecules.

The chapter is organized as follows. Section 2 presents the GNN model with a detailed explanation of the related learning procedure, while Sect. 3 describes the Layered GNNs, showing some possible design choices for this architecture. Some applications of LGNNs to pattern recognition problems are illustrated in Sect. 4, together with some experimental results. Finally, conclusions and future works are reported in Sect. 5.

2 Graph Neural Networks

Complex patterns can be described by a structured representation encoded as a graph. In fact, graphs allow us to represent data as a set of atomic parts, the *nodes*, and binary relationships among them, the *arcs* or *edges*. For instance, an image can be represented as a set of regions. Each region corresponds to a node of the graph and the edge connecting two nodes may encode that the two corresponding regions are adjacent. In this case, the relationship between the two linked entities is symmetric and the corresponding link will not be oriented (usually in this case the term *edge* is used). Other type of relationships (for instance inclusion between two regions) may be asymmetrical and the corresponding link will be oriented (the term *arc* is preferable in this case).

Graph Neural Networks (GNNs) provide a computational scheme able to compute a *node-focused* function on a given graph. In particular, the output of the computation is, generally, a set of real valued vectors attached to the nodes of the input graph. For pattern recognition applications, like for instance classification, where the graph represents the object to be processed, a single output can be assigned to the graph by selecting a node, in order to implement a function $\tau(G, v)$ from the graph G to a vector in $\mathbb{R}^r$, given the node v in G . The function implemented by the GNN depends on parameters, the *weights*, that can be adapted by a supervised learning procedure.

2.1 Graphs

A graph is defined as a pair $G = (V, E)$, being V the set of nodes and $E \subseteq V \times V$ the set of edges. An edge is denoted by the pair of connected nodes as (u, v) , where $u, v \in V$. If the graph is *undirected* the order of the two nodes in the pair is not considered, whereas for *directed* graphs the pair defines an arc where the first node is the source and the second node the destination of the link.

The pair (V, E) describes the pattern by representing its parts and its structure through the connection topology. The representation can be completed by adding attributes to each part (node) and, eventually, to the relationships (arcs or edges). For instance, when a node stands for a region in an image, perceptual or geometrical features of the region, such as its average color, extension, perimeter, etc., can be stored as attributes of the node. If an edge stands for the adjacency relationship, its attributes may correspond to the relative distance of the barycenters of the two regions, the relative orientation of the two shapes and so on [7]. The attributes attached to each node or edge/arc are referred to as a *label*. We can assume that the same label space L is used for all the nodes. In the following, we will consider only the case of graphs without labeled edges/arcs, but the definitions and computational models can be easily extended by introducing the edge/arc label space [20]. Since we examine neural network models to process graphs, we assume that the attributes are encoded as real values such that the label is a real vector, i.e. $\mathcal{L}(v) \in L \subset \mathbb{R}^q$. A labeled graph

is defined as a triple $G_L = (V, E, \mathcal{L})$, being $\mathcal{L} : V \rightarrow L$ the node labeling function that allows us to retrieve the label $\mathcal{L}(v) \in L$ of the node $v \in G_L$. To simplify the notation, the label of node v will be directly referred to as $\mathbf{l}_v \in \mathbb{R}^q$.

Given a graph, we can define several properties that depend on the topology of its connections. In particular, in the following, we will consider the *neighbourhood* of a given node v as the set of nodes directly connected to it. Formally it can be defined as $\text{ne}[v] = \{u \in V | (u, v) \in E \vee (v, u) \in E\}$. For undirected graphs there is no need to distinguish between the pairs (u, v) and (v, u) , whereas for directed graphs the concept of neighbourhood considers both incoming and outgoing arcs. In fact, in this latter case, the *parents* of a node v are defined as $\text{pa}[v] = \{u \in V | (u, v) \in E\}$ and the *children* of v as $\text{ch}[v] = \{u \in V | (v, u) \in E\}$. In this case, $\text{ne}[v] = \text{ch}[v] \cup \text{pa}[v]$ holds. In the definition of the GNN processing scheme we will refer to the general concept of neighbours of a node as described before, without making any distinction between directed or undirected graphs, to simplify the presentation. In practice, the GNN architecture can be designed to implement a different processing to incoming arcs with respect to the outgoing ones [20]. Finally, we define the *node degree* as the cardinality of its neighbourhood $|\text{ne}[v]|$.

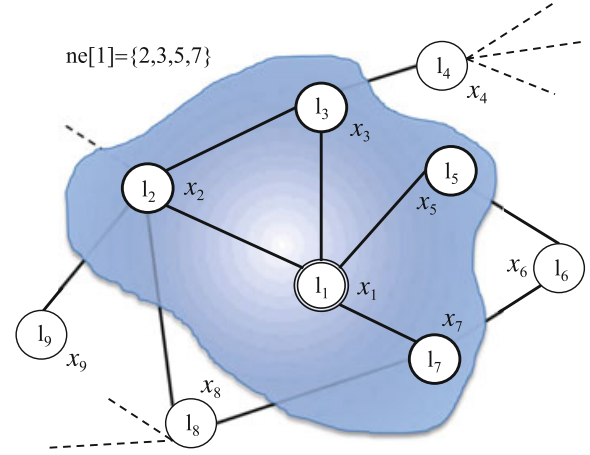
2.2 Graph Neural Networks

The computation of the GNN is performed locally at each node by means of a *state vector* $\mathbf{x}_v \in \mathbb{R}^s$, that stores a hidden representation which depends on the node and its context in the graph. In fact, the state is computed for each node through a diffusion mechanism based on the graph topology, that allows us to locally encode the information concerning both the graph structure and the node labels. The encoding is performed by a learnable function that can be tuned to extract the relevant features to obtain the desired processing. The state variables $\mathbf{x}_v, v \in V$, are additional labels stored into the graph nodes that represent the state of the computation. Thus, the GNN model can be thought as a set of identical computational units that calculate a local state for each node, based on the states of its neighbours and on its label and links (see Fig. 1). The computational scheme is obtained by exploiting a *state transition function* f that models how $\mathbf{x}_v$ is obtained given the context of v . Its arguments are the label $\mathbf{l}_v$ of node v , the states and the labels of the neighbours, $\mathbf{x}_{\text{ne}[v]}$ and $\mathbf{l}_{\text{ne}[v]}$ respectively. The transition function will also depend on a vector of parameters $\theta_f \in \mathbb{R}^p$, that are adapted during the learning process. Hence, the state update equations can be formally expressed as

$$\mathbf{x}_v = f(\mathbf{l}_v, \mathbf{x}_{\text{ne}[v]}, \mathbf{l}_{\text{ne}[v]} | \theta_f) . \quad (1)$$

The state transition function f can be realized by an Artificial Neural Network (ANN). The network will have an appropriate architecture to deal with a variable number of arguments (i.e. inputs), since the degree of each node v can be different. Apart from this requirement, that will be clarified in the following showing a possible

Fig. 1 Local state computation with the GNN state transition function



$$\mathbf{x}_1 = f(\mathbf{l}_1, \underbrace{\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_5, \mathbf{x}_7}_{\mathbf{x}_{\text{ne}[1]}}, \underbrace{\mathbf{l}_2, \mathbf{l}_3, \mathbf{l}_5, \mathbf{l}_7}_{\mathbf{l}_{\text{ne}[1]}} | \theta_f)$$

implementation, the number of outputs will be equal to the state space dimension s and the network architecture will be defined by the choice of the type and number of neurons, the number of layers, etc., as for the classical ANNs. The parameter vector θ_f will collect all the neural network connection weights. In this processing scheme, the same function (i.e. the same weight vector) is exploited for all the nodes of the input graph.

Given the state transition function and an input graph G , the state computation yields a vector assigned to each node v in G . By ordering the nodes, we can stack all the node variables into a single global vector $\mathbf{x} \in \mathbb{R}^{s|V|}$. Similarly, all the node labels define the vector $\mathbf{l} \in \mathbb{R}^{q|V|}$. Given this notation, the global computation carried out on the input graph is described by the equation

$$\mathbf{x} = F(\mathbf{x}, \mathbf{l} | \theta_f), \quad (2)$$

being F the *global transition function*, whose entries result from stacking the transition functions f applied to each node v , with an appropriate projection of the two vectors $\mathbf{x}$ and $\mathbf{l}$ to yield the variables related to the neighbourhood of v . The vector of the learnable parameters is still θ_f , since all the local instances of the transition function share the same weights.

The result of the state computation is the vector $\mathbf{x}$ that satisfies Eq. (2), that is a system of non-linear equations in the variable $\mathbf{x}$. This equation can have multiple solutions, but we are interested only in those cases when the solution is unique. This requirement can be satisfied if the function F is a *contraction map* with respect to the variable $\mathbf{x}$. In this case, Eq. (2) has a unique solution by the Banach Fixed Point Theorem [21]. The condition, that guarantees the function F to be a contraction map, requires the existence of a value μ , $\mu \in [0, 1)$, for which $\|F(\mathbf{x}, \mathbf{l} | \theta_f) - F(\mathbf{y}, \mathbf{l} | \theta_f)\| \leq \mu \|\mathbf{x} - \mathbf{y}\|$, for any $\mathbf{x}$ and $\mathbf{y}$. This condition can eventually be forced in the learning process.

The Banach Theorem defines also the procedure to compute the solution of Eq. (2). In fact, the fixed point can be obtained by an iterative process of *state update* with the equation

$$\mathbf{x}(t+1) = F(\mathbf{x}(t), \mathbf{l}|\theta_f), \quad (3)$$

being $\mathbf{x}(t)$ the state value at iteration t . The sequence defined by the recurrent equation (3) converges to the fixed point when F is a contraction map. The iterative computation is performed locally at each node by applying the state function as

$$\mathbf{x}_v(t+1) = f(\mathbf{l}_v, \mathbf{x}_{\text{ne}[v]}(t), \mathbf{l}_{\text{ne}[v]}|\theta_f), \quad (4)$$

until the states converge to the solution. Notice that the final state is dependent on the input graph. The node labels are explicitly considered as an input of the computation, whereas the graph topology is implicitly exploited by the diffusion process among neighbour nodes in the graph. In fact, the computation is performed by a set of computational units, connected with the same topology of the edges in the input graph. When processing different graphs, the same units are arranged in a different structure, yielding different encodings. This computational structure constitutes the *encoding network* (see Fig. 2). This network is obtained by replicating the computational unit f for each node and by connecting the units using the same topology of the edges. The same transition function yields different encoding networks when applied to different input graphs.

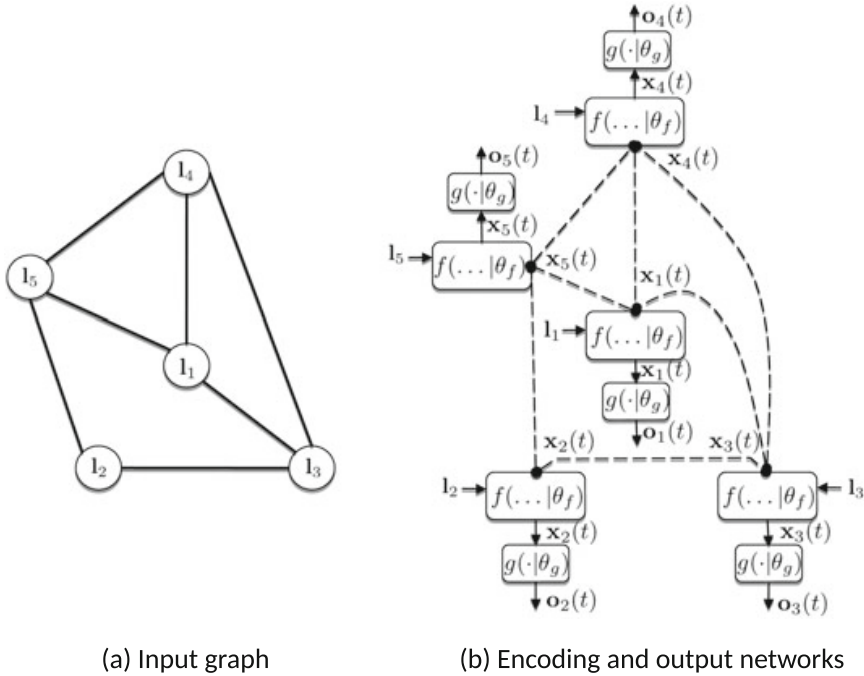


Fig. 2 The *encoding* and the *output* networks (b) when processing an input graph (a). The *graph neural network*, realizing $f(\dots, \theta_f)$ and $g(\cdot, \theta_g)$, is replicated on the input graph nodes and its weights, θ_f and θ_g , are shared among all the replicas

The output vector $\mathbf{o}_v \in \mathbb{R}^m$ is computed at each node v , depending on its label and state. The *local output function* g is exploited at each node after the computation of the fixed point for the state vector. g is parametrized by a vector of adaptable parameters θ_g , that can be adjusted during the learning process. For each node v the output is calculated as

$$\mathbf{o}_v = g(\mathbf{x}_v, \mathbf{l}_v | \theta_g) . \quad (5)$$

Figure 2 depicts the output computation as obtained after the state encoding step. To implement a graph-focused function, $\tau(G)$, the output is considered only for a predefined node having specific properties [20]. As shown in Fig. 2, the *Graph Neural Network* is defined by the functions f and g .

The function g can be realized by a feedforward neural network with $s + q$ inputs and m outputs, without any other restriction on the network architecture. The parameter vector θ_g collects the neural network connection weights.

The model for the state transition function f depends on the properties of the considered graphs. For *positional graphs* with bounded degree, i.e. graphs in which the edges connected to a node have a predefined order, the function f has a predefined maximum number of arguments and there is a given correspondence between each argument and an edge attached to the node. Hence, f can be realized by a feedforward neural network with s outputs and a fixed number of inputs. If d_M is the maximum node degree, the number of inputs will be $q + d_M(q + s)$. For nodes having a degree $d < d_M$ some arguments will not be available and a specific `nil` vector can be exploited to encode the label or the state corresponding to the missing edges (f.i. a vector with all 0s can be used as `nil`). This approach is feasible when d_M is sufficiently small and the variability of the node degree in the input graphs is not too high. If the graphs are *non-positional*, f can be realized with a model that makes it independent of the neighbourhood size and of the edge order. In this case, the state update function can be implemented as

$$\mathbf{x}_v(t + 1) = \sum_{u \in \text{ne}[v]} h(\mathbf{l}_v, \mathbf{x}_u(t), \mathbf{l}_u | \theta_h) . \quad (6)$$

The function h computes the contribution of each node u in the neighbourhood of v and all the contributions are summed up yielding the independence from the edge order and number. Two different implementations (*linear* and *neural*) for the function h in Eq. (6) have been proposed in [11, 20]. For neural GNNs, the function h is implemented by a feedforward neural network with $2q + s$ inputs and s outputs. However, this solution may not produce a contraction map for any value of θ_h . In this case, the learning objective can be defined as to include a cost term that penalizes mappings that are not contractions.

2.3 The GNN Learning Procedure

We consider a supervised learning scheme in which a supervisor provides a set of examples with a target value for the network output. In this case, each example consists of a graph and a target value for one or more of its nodes. Formally, the learning set is defined as

$$\mathcal{L}_e = \{(G_p, v_{pj}, \mathbf{t}_{pj}) \mid G_p \in \mathcal{G}, p = 1, \dots, P, \\ v_{pj} \in V_{G_p}, \mathbf{t}_{pj} \in \mathbb{R}^m, j = 1, \dots, n_p\}, \quad (7)$$

where $\mathcal{G}$ is the set of graphs provided as examples. For the input graph G_p , the supervisor may provide a set of target vectors $\mathbf{t}_{pj}$, $j = 1, \dots, n_p$, for a subset $S_{G_p} = \{v_{pj} \in V_{G_p}, j = 1, \dots, n_p\}$. For graph-focused tasks, the supervision is given only for one node and a single target is provided for each graph in $\mathcal{G}$. The learning process corresponds to the minimization of an objective function that penalizes the error of the GNN response with respect to the given targets. Usually, the quadratic loss is exploited, defined as

$$E(\theta|\mathcal{L}_e) = \frac{1}{P} \sum_{p=1}^P e(\theta|G_p) = \frac{1}{2P} \sum_{p=1}^P \sum_{v_{pj} \in S_{G_p}} \|\mathbf{t}_{pj} - \mathbf{o}_{v_{pj}}(\theta|G_p)\|_2^2, \quad (8)$$

where $\theta = [\theta'_f \theta'_g]'$ collects the parameters of the encoding and the output functions. When the transition and output functions, f and g , are differentiable with respect to θ , the loss $E(\theta|\mathcal{L}_e)$ is a differentiable function and it can be minimized by gradient descent. In fact, it can be proven (see Theorem 1 in [11]) that, when $F(\mathbf{x}, \mathbf{l}|\theta_f)$ is a contraction mapping, the loss $E(\theta|\mathcal{L}_e)$ depends on its unique fixed point that is a continuous and continuously differentiable function of the parameters θ_f .

If the model, selected for the state transition function f , is not guaranteed to implement a contraction for any value of the parameters θ_f , the learning objective must include a term to penalize non-contractive behaviours. The penalty term is obtained from the gradient of the function F as

$$p(\theta_f) = \beta L \left(\left\| \frac{\partial F(\mathbf{x}, \mathbf{l}|\theta_f)}{\partial \mathbf{x}} \right\| \right), \quad (9)$$

where β is the weight of the penalty term, $L(y)$ is usually the hinge $\max(0, y - \mu)$, able to penalize values of y greater than the target contraction constant $\mu \in [0, 1)$ (see [11] for more details). Given the objective function of Eq. (8), with eventually the addition of the penalty in Eq. (9), for each graph $G_p \in \mathcal{G}$, the gradient of the loss with respect to the parameters θ is computed by the following procedure.

1. *State computation.* Starting from the initial state $\mathbf{x}(0|G_p)$, the state transition function

$$\mathbf{x}(t+1|G_p) = F(\mathbf{x}(t|G_p), \mathbf{l}_{G_p}|\theta_f)$$

is iterated until the condition $\|\mathbf{x}(T|G_p) - \mathbf{x}(T-1|G_p)\| < \epsilon$ is verified at the iteration T , where $\epsilon > 0$.

2. *Error evaluation.* The output network $g(\mathbf{x}_{v_{pj}}(T|G_p), \mathbf{l}_{v_{pj}}|\theta_g)$ is used to compute the outputs at the supervised nodes $v_{pj} \in S_{G_p}$ and the partial loss $e(\theta|G_p)$ of Eq. (8) is accumulated.
3. *Error backpropagation and gradient computation.* The value of the partial gradient $\nabla_{\theta} e(\theta|G_p)$ is computed by error backpropagation on the output and encoding networks, and the computed value is accumulated to yield the total gradient, as $\nabla_{\theta} E(\theta|\mathcal{L}_e) = \frac{1}{P} \sum_{p=1}^P \nabla_{\theta} e(\theta|G_p)$.

After processing all the graphs in $\mathcal{L}_e$, the total gradient $\nabla_{\theta} E(\theta|\mathcal{L}_e)$ can be exploited to update the GNN weights using a gradient descent rule as, for instance,

$$\theta_{k+1} = \theta_k - \eta_{k+1} \nabla_{\theta} E(\theta|\mathcal{L}_e)|_{\theta=\theta_k}, \quad (10)$$

where k is the learning epoch, the gradient of $E(\theta|\mathcal{L}_e)$ is computed for $\theta = \theta_k$ and η_{k+1} is the *learning rate*. An adaptive learning rate scheme can be possibly employed in the learning process. The initial weights θ_0 are usually set to small random values (f.i. sampled using a uniform distribution with mean 0). The learning process can be stopped after a given maximum number of epochs, when the loss is below a predefined value, or when the gradient norm is below a threshold.

The error backpropagation on the encoding network follows a scheme that is obtained by combining Backpropagation Through Structure and Backpropagation Through Time [22]. In fact, the relaxation process, applied for T steps to compute the state fixed point, corresponds to a layered unfolding network having T layers, where each layer is a replica of the function $F(\mathbf{x}(t|G_p), \mathbf{l}_{G_p}|\theta_f)$ for a given time step $t = 1, \dots, T$. The output of layer t is the input for layer $t+1$, such that their units are interconnected by the state variables $\mathbf{x}(t)$. The resulting unfolding network of Fig. 3 has a multilayer structure, where the connection topology between adjacent layers is determined by the edges in the input graph. The replicas of the function F in the unfolding network share the same parameters θ_f . Finally, the state units of layer T are connected to the output functions to compute the GNN outputs.

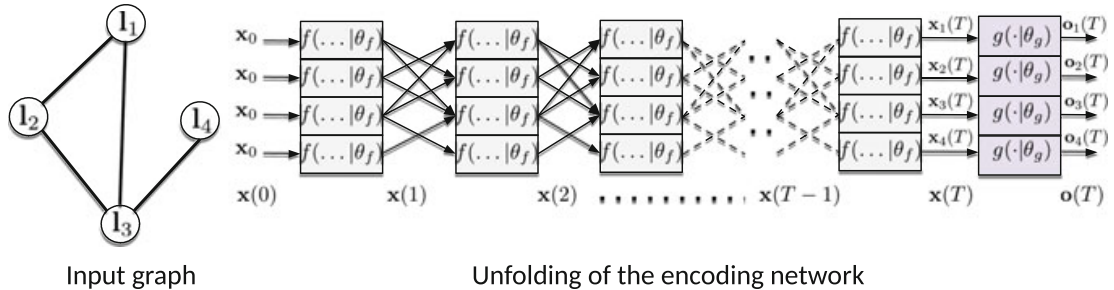


Fig. 3 The unfolding network obtained when computing the state on an input graph G_p

The unfolding network represents an efficient schema for the computation of the gradient of the loss function with respect to the GNN parameters θ . In fact, the original Backpropagation algorithm for feed-forward neural networks can be directly exploited on the unfolding network (see [20] for more details).

3 Layered Graph Neural Networks

The computation performed by GNNs can be layered to yield a deep processing architecture [23]. Given an input graph G in the learning set, we can add GNN layers such that each new layer exploits the original graph and the computation performed by the GNN of the previous layer. Formally, the GNN of layer k , $\mathcal{N}^k$, will process a graph G^{k-1} having the same topology of $G = G^0$, i.e. $V^{k-1} = V$ and $E^{k-1} = E$, but enriched labels assigned to the nodes. In particular, three different cases can be considered for layers $k = 2, \dots, K$, being K the number of layers:

1. the node labels in G^{k-1} are obtained by concatenating the original labels in G with the outputs computed by the previous GNN layer, i.e., $\mathbf{l}_v^{k-1} = [\mathbf{l}_v, \mathbf{o}_v^{k-1}] \in \mathbb{R}^{q+m}$, where $\mathbf{l}_v \in \mathbb{R}^q$ is the label of node v in G and $\mathbf{o}_v^{k-1} \in \mathbb{R}^m$ is the output of the GNN $\mathcal{N}^{k-1}$ on node v ;
2. the node labels in G^{k-1} are the concatenation of the original labels with the node states computed by the GNN in the previous layer, i.e., $\mathbf{l}_v^{k-1} = [\mathbf{l}_v, \mathbf{x}_v^{k-1}] \in \mathbb{R}^{q+s}$, where $\mathbf{x}_v^{k-1} \in \mathbb{R}^s$ is the state for node v computed by $\mathcal{N}^{k-1}$;
3. the node labels in G^{k-1} are obtained by adding both the outputs and the states computed by the GNN in the previous layer to the original labels, i.e., $\mathbf{l}_v^{k-1} = [\mathbf{l}_v, \mathbf{x}_v^{k-1}, \mathbf{o}_v^{k-1}]$.

The first layer $k = 1$ will process the original graph. The resulting computational scheme is depicted in Fig. 4. The outputs of the Layered GNN (LGNN) are the output values computed by the last GNN layer $\mathcal{N}^K$.

For instance, let us assume that the input G represents the region adjacency graph (RAG) of an image, i.e. a graph where nodes stand for homogeneous regions of the image and edges denote their adjacency relationships. Moreover, let us assume that the goal of the application is that of classifying the regions, i.e. the nodes, according to the class of the object they represent. In such a case, the first GNN takes in input a RAG G and produces a preliminary classification of the nodes. The resulting classification (and/or the states of the GNN, as described in points 1–3) is added to the labels of the RAG G in order to construct the graph G^1 . Then, the second GNN is fed by G^1 and the procedure is repeated for all the layers of the LGNN.

The LGNN is trained by adding one layer after the other, until the last one. When training layer k , the weights of the GNNs in the previous layers can be kept fixed or can be fine tuned, as it happens in deep learning frameworks. Each network is trained using the original targets and the graphs constructed by the previous layer, as described before. The targets of each supervised node v in G^{k-1} are equal to original targets, i.e. $\mathbf{t}_v^{k-1} = \mathbf{t}_v$.

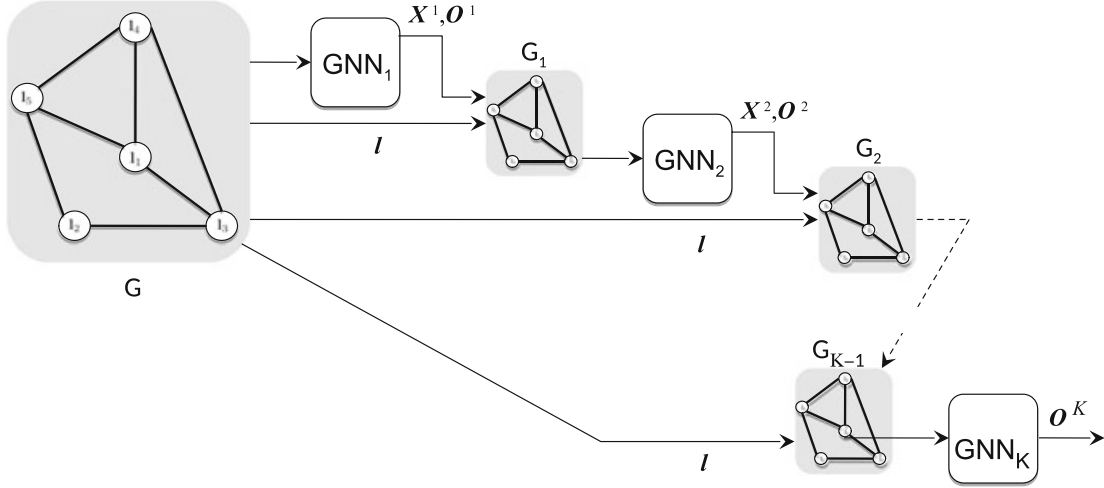


Fig. 4 The Layered GNN processing scheme

The layer-by-layer training procedure makes the learning feasible, reducing problems caused by the gradient vanishing effect due both to the network depth and to the long-term dependency impact on the graph. Each layer develops its expertise exploiting the supervisions provided in the learning set. The stacking of the layers allows the upper levels to take advantage of the knowledge already extracted in the previous layers, as it happens in other cascade neural architectures, following in some sense a boosting scheme. Basically, the added layers can focus their processing only to correct the errors made by previous layers. This processing scheme can reduce the long-term dependency problem, i.e. the difficulty in learning dependencies that involve nodes far from each other in the graph (distance is in terms of the shortest path through the graph edges). In fact, for a given layer k , the state $\mathbf{x}_v^k$ and the output $\mathbf{o}_v^k$ are able to summarize information up to a certain distance in the neighbourhood of node v . The GNN in the following layer will be actually able to exploit the information related to a larger neighbourhood since the nodes in $\text{ne}[v]$ contain additional information on their context in the added input components.

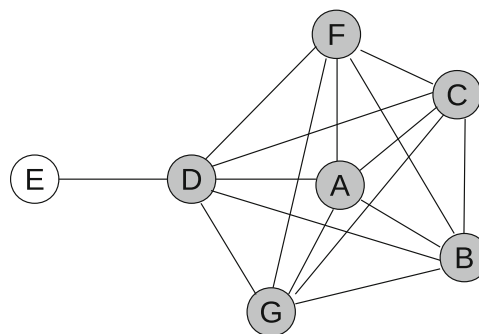
4 Applications to Pattern Recognition

We used an artificial dataset and two real-life benchmarks to test the LGNN model. In particular we decided to test on: clique localization, classification of mutagenic molecules and protein secondary structure prediction.

4.1 Dataset Descriptions

The clique localization problem Given an undirected graph $G = (V, E)$, a clique of size k can be defined as a complete subgraph of k nodes (see Fig. 5). The clique

Fig. 5 An example of a clique in a graph: the gray nodes (A, B, C, D, F, G) are part of the maximum clique, while the white node (E) is out of the identified subgraph



localization can be identified as a particular instance of the subgraph localization task. Maximum clique identification in a graph is NP hard [24] and it has been widely addressed in the computer science literature. This is due to the fact that the applications of maximum clique algorithms are widely spread among various disciplines since cliques are associated with important network properties. For example, in chemistry, bioinformatics and computational biology, clique detection is exploited to identify similarities among molecules, and, in particular, for drug database screening [25]. Another important application of clique searching is for protein structure comparison. The understanding of such structure is in fact vital for the identification of protein functionalities and for the study of protein interactions [26].

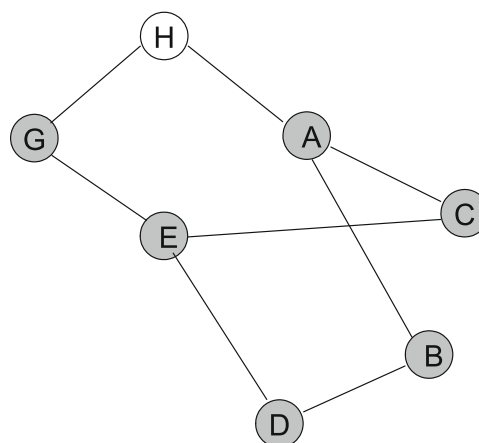
Clique-detection methods are also used for clustering gene expression data [27]. With most of the recent DNA sequencing technologies, the task of finding interesting patterns in long DNA strings is one of the most challenging. In [27], this problem was approached by implementing a parallel algorithm for Maximum Clique detection. Other interesting applications of maximum clique localization are related to other fields. For instance, in social network analysis, cliques correspond to communities, while in the World Wide Web they represent web clusters that share common topics [28]. In [29], a new method for studying and defining overlapping communities was proposed, by defining universal network features and showing their application to the case of protein-protein interaction and web community detection.

The dataset used in our experiments is made of 1,000 random graphs having 20 nodes, containing a clique of size 5. The GNN was trained to approximate the function defined by $\tau(G, v) = 1$ if v belongs to a clique, and $\tau(G, v) = -1$, otherwise. This problem setting was inspired by similar experiments described in [30].

The Mutagenesis problem The study of mutagenic compounds is a well known chemistry and biological challenge. Such compounds may be the cause of cancers and damages to DNA strand molecules [4] as well as they can be responsible for ageing, evolution and human genetic diseases [31]. Therefore the study of mutagenic compounds is of particular interest for pharmaceutical industries, to identify molecules that potentially have a mutagenic activity.

The dataset [4] collects data for 230 nitroaromatic compounds. Such compounds are commonly used as intermediate subproducts of industrial chemical reactions. In this case, the objective is to learn how to predict mutagenic molecules. Many different types of information are included in the dataset, for example the structure of the atom

Fig. 6 Mutagenicity classification. The computation is graph-focused: the mutagenicity of the molecule is computed at node H



bonds and the chemical properties of the molecules. The graphical representation of the compound was designed to encode the bond structure. In particular, the nodes represent the atoms and the edges represent the bonds between them.

Each node is labeled using the information on the chemical properties. The label is a 13 dimensional vector, and it is constituted by features regarding the single atom and also by 4 global features [4]. The single atom features represent the charge of the atom and a one-hot representation of the atom type. The global features, on the other hand, represent the lowest unoccupied molecule orbital, the water/octanol partition coefficient and two precoded structural attributes. Moreover, we decided to adopt the following convention: in each graph, a single node is supervised. This has been chosen to be the first atom of the original dataset (see Fig. 6). The output is 1 in the case of a mutagenic molecule and -1 in the opposite case.

Secondary protein structure prediction Predicting the protein structure from the amino acid sequence is considered a fundamental task in molecular biology. The high throughput of large scale genome sequencing makes available a huge amount of sequence data, that can be exploited by the most recent machine learning and statistical tools to obtain reliable prediction models for protein secondary structures. Helixes, strands and coils are considered the most typical secondary structure regions. The knowledge of the secondary structure is particularly important, as it can be used for the tertiary structure prediction. This last one is of particular interest, as it defines the 3D amino acid positioning and, therefore, determines the functionality of the protein itself.

Secondary structure prediction can be easily modeled as a classification problem by using a graphical structure. Amino acids can be represented as nodes, whereas edges stand for peptide bonds. In this way, we can encode the protein primary structure. The final objective becomes therefore to predict one of the three categories mentioned before, which the amino acid belongs to.

This task can be faced with GNNs learning a function $\tau(G, v)$ that associates the class of the region containing the amino acid corresponding to the node v in the protein represented by the graph G .

The dataset is composed by 2171 proteins and 344,653 amino acids [32]. As before, we store the amino acid available features as a 20 dimensional vector, that represents the one-hot encoding of the type of amino acid. A three-dimensional vector is the target associated to each node, and this contains information regarding the one-hot encoding of the three classes, i.e. *helix*, *coil* or *strand*.

4.2 Experimental Settings

The base GNN model used in the experiments is realized as follows. We implemented the transition function f and the output function g as a static network with three layers (where one is a hidden layer). In particular, we used a linear activation function in the output layer and a hyperbolic tangent function as activation for the hidden layer. The number of hidden units for f and g networks are 5, with a 2-dimensional state. We initialized the GNN weights randomly, in the range $[-0.1, 0.1]$. For each experiment, we split the dataset into three parts: training, test, and validation. The clique localization training set has 600 graphs, while the validation and the test contain 200 examples each. We used a 10-fold cross validation strategy for splitting the mutagenesis dataset and a 5-fold cross validation procedure for the secondary protein structure prediction. This was done accordingly to the papers in which the datasets were originally used.

By a trial-and-error procedure, we chose 2000, 500 and 1000 as the number of training epochs for the clique localization, the mutagenesis and the secondary protein structure prediction, respectively. The network generalization performance was calculated every 10 epochs on the validation set. Then, the network yielding the best validation error was evaluated on the test set.

We exploited the Resilient BackPropagation (RPROP) algorithm for the learning procedure, setting its parameters to standard values [33]. We report the standard deviation and the average accuracy on five repetitions of each experiment. For the case of the clique localization task, we rebuilt the dataset and reinitialized the weights at each repetition. For real-world benchmarks, instead, weights were recomputed for every fold. More precisely, the reported results consist in the macro-average and the macro-standard deviation.

4.3 Experimental Results

Figure 7 shows the results achieved on the clique localization problem. The plots correspond to the three LGNN architectures: propagation of the previous layer output (first column), of the previous layer state (second column), and of the previous layer state and output (third column). In the plots, points represent the average accuracies, while vertical bars represent their standard deviations. These results have been

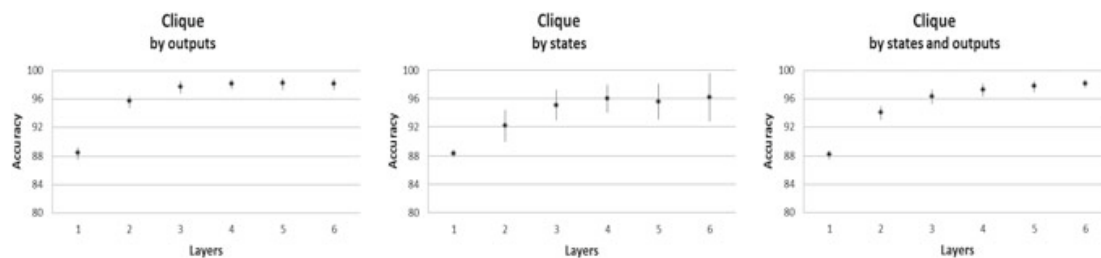


Fig. 7 Results for the clique localization problem. The plots report the average accuracy and the standard deviation (vertical bars). Each column shows the results for the three different LGNN configurations, based on the addition to the labels of the previous layer outputs, of the previous layer state and of both of them

obtained by varying the number of layers of the LGNN. The result for a standard GNN architecture with only one layer is reported as the first value.

The accuracy increases with the number of layers, as Fig. 7 shows. This is particularly clear when the first layer is added, but it appears of less impact when more layers are added to the architecture. In particular, in this case, we can actually observe an oscillation of the results. Moreover, the effect is more evident when propagating the GNN outputs rather than the states. An interesting fact is that using states seems to have an effect on yielding larger variances. Actually, even if, in theory, the state contains the information to reproduce the outputs, however the richness of the state configurations may confuse the learning algorithm, preventing it to extract the information relevant for the specific task. However, overall, we can say that the proposed multilayer architecture always performs better than the standard GNN for all the considered cases.

The average accuracy and the standard deviation for the LGNN in the case of the mutagenesis and the secondary protein structure benchmarks are shown in Fig. 8, in the first and in the second row, respectively. We only report the results in the case in which the node label is enriched by the previous layer output, since this configuration was shown to yield the best results in the case of the artificial dataset. For the secondary protein structure benchmark, the trend of the accuracy with respect to the number of layers has a similar behaviour to that observed for the clique dataset. The results for the mutagenesis dataset, instead, show a larger variance for the test set. Even though the average accuracy improves, the variance is too large to consider such an improvement statistically significant. Overfitting issues may cause the larger variance, as a comparison of test and training sets suggests. In particular, the dimension of the mutagenesis dataset is small and this can generate problems in the generalization. This is a well known issue in machine learning: when the number of layers increases, the architecture is able to approximate a larger set of functions, but if the training set is not statistically significant with respect to the real data distribution, the generalization capabilities of the model may be hindered. We also performed experiments with a single layer GNN and an increased number of states and hidden neurons in order to understand if the performance improvements for LGNNs were actually due to the fact that we are using a larger set of parameters. We evaluated

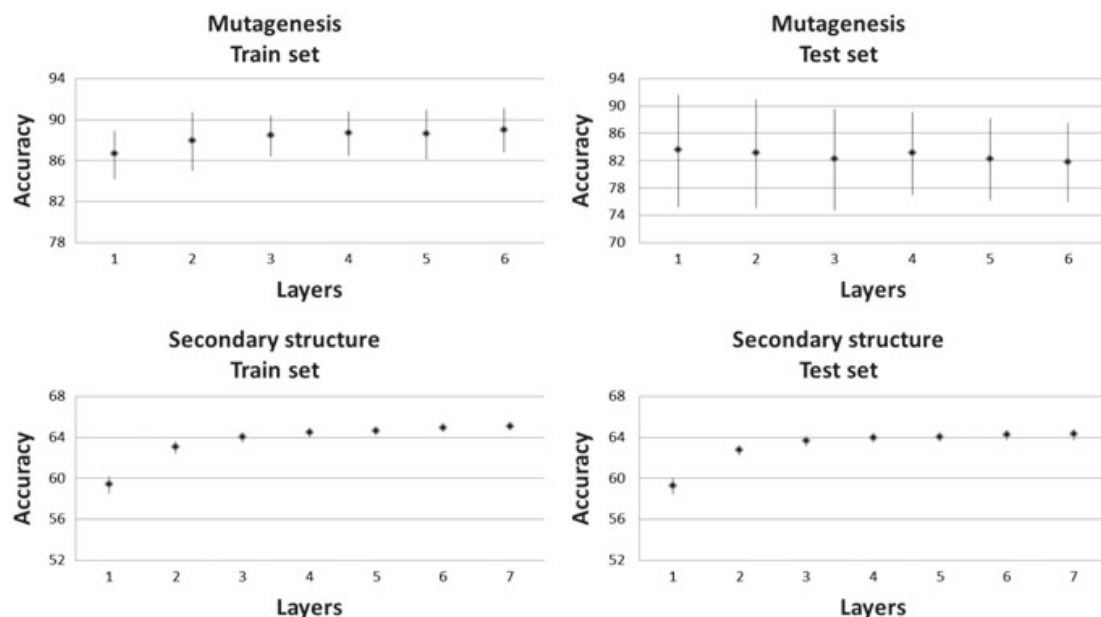


Fig. 8 Average accuracy and standard deviation (vertical bars) on the mutagenesis (first row) and on the secondary protein structure (second row) benchmarks. The first and the second column show the results on the training and the test set, respectively

GNNs with 5, 15, 25, 35 hidden nodes, in the output and in the transition networks, and with state dimensions varying in $\{2, 5, 10, 15\}$. In Table 1, we show the best performance achieved together with the best configuration. These results show that the same performance of an LGNN architecture cannot be obtained only by increasing the number of parameters in a shallow GNN. Moreover, in the case of an LGNN architecture, the amount of computational resources needed by a k -layered architecture is smaller than those required in the case of a GNN, if we consider the case of a GNN with k times more parameters or state variables. In fact, we can prove that the computational time in GNNs is quadratic w.r.t. the state dimension [11].

A comparison with the state-of-the-art on the considered benchmarks was not a main goal of this evaluation. Nevertheless, it is worth mentioning that the mean accuracy obtained by LGNNs on the mutagenesis dataset (83.47%) is not far from the best accuracy reported in literature (88%) [34].

For what concerns the secondary protein structure prediction, the LGNN performance is 10% smaller than the one achieved by Porter [32]. However these two results are not directly comparable. In fact, Porter exploits also a pre- and a post-processing methodology, that was not applied in the reported experiments, due to time and resource constraints. The classifier used in Porter is based on recursive neural networks (RNNs). GNNs and LGNNs are an extension of RNNs and, therefore, we expect that, by applying the Porter pre- and post-processing procedures, similar or better results can be achieved.

Table 1 Accuracy for the best configuration of a single layer GNN when varying the state dimension and the number of hidden nodes, both for transition and output networks.

	State dim.	Num. hidden	Accuracy
Clique Loc.	10	10	97.75
Mutagenesis	2	5	83.47
Secondary Struct.	15	5	60.53

4.4 Transient Pocket Prediction

In the last few years, there has been an increasing interest in developing algorithms for finding transient pockets on the protein 3D surface. Transient pockets are cavities that can appear and disappear on a protein surface due to the protein dynamics. As common pockets, transient pockets can contain active sites, useful for chemical interactions with *ad hoc* ligands. Protein–protein and protein–ligand interfaces are in fact of particular interest for pharmaceutical companies, since the identification of new disease pathways leads also to the development and identification of new drug compounds [35]. The geometrical similarity between the protein surface in the vicinity of the active site and the target molecule gives an important insight to establish how good is a putative pocket to act as an anchorage for a specific drug.

Several methodologies and algorithms have been applied to this task. For example, the Q-site finder [36] is a method used to identify pockets which are unable to accommodate a particular compound, due to potentially conflicting geometries or because of their small size. Other interesting and widely used softwares are LIGSITECS and LIGSITE, that take into account the information regarding the protein surface in the vicinity of the pocket to establish its usability [37]. On the other hand, PocketPicker [35] is a grid-based approach used for the prediction of the presence of a binding site inside the pocket. PocketPicker has been successfully applied to a set of protein–ligand complexes and an extensive evaluation was carried out to establish the quality of the protein binding site prediction.

Nevertheless, standard methods for protein structure prediction do not allow to efficiently and affordably localize transient pockets due to their short persistence and complex pattern, whereas connectionist models for structures seem to capture sufficient information in the vicinity of the putative pocket to predict its appearance, dimension and duration. In particular, in drug discovery, it often happens that many molecules of apparently unrelated structures are active on the same drug target. In this case, constructing a complete edge-weighted graph for describing drugs — with atoms as vertices and edge weights that stand for interatomic distances — means finding the Maximum Common Subgraph between them, on which a clique-detection algorithm can be efficiently applied [38]. Recently, the described approach has been also extended to the case of the identification of complementary protein surfaces. This is particularly important for biomolecules docking. In [39], a graph kernel method is presented for the prediction of DNA-binding sites with proteins. Labeled graphs are used to model the local structure of the two

macromolecules, searching for possible similarities. Finally, in [40], surface local motifs, that are frequently observed in a specific protein group, are extracted, to establish — via common subgraph matching — if they are able to accommodate similar molecules.

Also GNNs and, more specifically, LGNNs, can be used to predict both the appearance and the dimension of pockets and, thanks to the incremental learning procedure, implemented by the cascade of networks, they are also able to take into account information not strictly related to the neighbourhood of the pocket, allowing to predict in advance, with respect to molecular dynamics models, the formation of a new pocket. Preliminary experimental results actually show both a good accuracy and a big time gain compared with classical simulations of molecular dynamics.

5 Conclusions

Graphs are commonly adopted as abstract representations for complex data, including DNA sequences, protein structures, documents, text, and images. In pattern recognition problems, graphs have been used since the early '70s, when they were employed for classifying visual patterns, especially in structural methods. In these approaches, objects are considered as constituted by “basic entities” (i.e. parts of the object homogeneous with respect to some visual features) related to each other. Under this assumption, node labels are descriptors of the parts, whereas edges encode relationships among them. The use of a pattern representation based on graphs implies reformulating all the standard tasks of a recognition system in terms of these data structures, such as classification, that in many approaches requires the comparison between an input and a set of prototypes, and learning, that consists in the process of creating a model for a set of classes given a set of known samples.

To this aim, in the present chapter, we have presented Graph Neural Networks (GNNs) and, in particular, a composite, deep architecture, referred to as Layered GNN (LGNN). In this model, each layer is an expert, implemented by a GNN, that solves the given task exploiting the original data and the computation provided by the previous layers. Intuitively, training the cascaded architecture layer by layer can help in summarizing, within the state of each node, the information related to gradually increasing neighbourhoods. Such an intuition, i.e. that the LGNNs can deal with and reduce the long-term dependency problem, when processing large structures, was experimentally investigated on both artificial and real-world data.

Nevertheless, the strategy, proposed to extend the basic GNN model, is just one of a set of potential alternatives, aimed at recursively learning deep structures. In fact, when the problem to be solved can be naturally partitioned into subtasks, each of them can be addressed separately, gaining partial solutions that can be subsequently used to move towards the overall solution. An example of such a situation is represented by the clique localization problem: a node v , not belonging to an N -clique, is not certainly part of a clique of dimension $N - 1$; instead, a node that belongs to an N -clique can possibly belong also to an $(N + 1)$ -clique. Of course, this decomposition

strategy is a less general approach to efficient learning in GNNs, but a preliminary experimental evaluation reports significant performance improvements whenever it is applicable. Another possibility for learning large structured data consists in decomposing the procedure into two steps, first solving a classification task given the graph labels — based on standard classifiers —, and then exploiting this information to enrich the original labels, keeping intact the graph topology. Even in this case, preliminary results show improved performances w.r.t. the original GNN model. Finally, different combinations of these methods may be considered as the object of future research.

References

1. H. Bunke, K. Riesen, Recent advances in graph-based pattern recognition with applications in document analysis. *Pattern Recognit.* **44**(5), 1057–1067 (2011)
2. M. Bianchini, F. Scarselli, Artificial neural networks for processing graphs with applications to image understanding: a survey, in *Multimedia Techniques for Device and Ambient Intelligence*, ed. by E. Damiani, J. Jeong (Springer, Berlin, 2009), pp. 179–199
3. C. Mooney, G. Pollastri, Beyond the twilight zone: automated prediction of structural properties of proteins by recursive neural networks and remote homology information. *Proteins* **77**(1), 181–190 (2009)
4. A. Srinivasan, S. Muggleton, R.D. King, M.J.E. Sternberg, Mutagenesis: ILP experiments in a non-determinate biological domain, In *Proceedings of the 4th International Workshop on Inductive Logic Programming*, Gesellschaft für Mathematik und Datenverarbeitung MBH, pp. 217–232, 1994
5. J. Cheng, M.J. Sweredoski, P. Baldi, Accurate prediction of protein disordered regions by mining protein structure data. *Data Min. Knowl. Discov.* **11**(3), 213–222 (2005)
6. M. Bianchini, M. Maggini, L. Sarti, Recursive neural networks and their applications to image processing, in *Advances in Imaging and Electron Physics*, vol. 140, ed. by P.W. Hawkes, (Elsevier – Academic Press, 2006), pp. 1–60
7. M. Bianchini, M. Maggini, L. Sarti, F. Scarselli, Recursive neural networks for processing graphs with labelled edges: theory and applications. *Neural Netw.* **18**, 1040–1050 (2005)
8. V. Di Massa, G. Monfardini, L. Sarti, F. Scarselli, M. Maggini, M. Gori, A comparison between recursive neural networks and graph neural networks, in *Proceedings of the IEEE International Joint Conference on Neural Networks*, pp. 778–785, 2006
9. A.-C. Tsoi, M. Hagenbuchner, R. Chau, V. Lee, Unsupervised and supervised learning of graph domains, *Studies in Computational Intelligence - Innovations in Neural Information Paradigms and Applications*, vol. 247 (Springer, Berlin, 2009), pp. 43–65
10. P. Frasconi, M. Gori, A. Sperduti, A general framework for adaptive processing of data structures. *IEEE Trans. Neural Netw.* **9**(5), 768–786 (1998)
11. F. Scarselli, M. Gori, A.-C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model. *IEEE Trans. Neural Netw.* **20**(1), 61–80 (2009)
12. W. Uwents, G. Monfardini, H. Blockeel, M. Gori, F. Scarselli, Neural networks for relational learning: an experimental comparison. *Mach. Learn.* **82**(3), 315–349 (2011)
13. G. Monfardini, V. Di Massa, F. Scarselli, M. Gori, Graph neural networks for object localization, in *Proceedings of ECAI 2006* (IOS Press, 2006), pp. 665–669
14. L. Di Noi, M. Hagenbuchner, F. Scarselli, A.-H. Tsoi, Web spam detection by probability mapping graph-SOMs and graph neural networks, in *International Conference on Artificial Neural Networks* (Springer, Berlin, 2010), pp. 372–381

Granular Computing Techniques for Bioinformatics Pattern Recognition Problems in Non-metric Spaces



Alessio Martino , Alessandro Giuliani  and Antonello Rizzi 

Abstract Computational intelligence and pattern recognition techniques are gaining more and more attention as the main computing tools in bioinformatics applications. This is due to the fact that biology by definition, deals with complex systems and that computational intelligence can be considered as an effective approach when facing the general problem of complex systems modelling. Moreover, most data available on shared databases are represented by sequences and graphs, thus demanding the definition of meaningful dissimilarity measures between patterns, which are often non-metric in nature. Especially in such cases, evolutive and fully automatic machine learning systems are mandatory for dealing with parametric dissimilarity measures and/or for performing suitable feature selection. Besides other approaches, such as kernel methods and embedding in dissimilarity spaces, granular computing is a very promising framework not only for designing effective data-driven modelling systems able to determine automatically the correct representation (abstraction) level, but also for giving to field-experts (biologists) the possibility to investigate information granules (frequent substructures) that have been discovered by the machine learning system as the most relevant for the problem at hand. We expect that many important discoveries in biology and medicine in the next future will be determined by an increasingly stronger integration between the ongoing research efforts of natural sciences and modern inductive modelling tools based on computational intelligence, pattern recognition and granular computing techniques.

A. Martino (✉) · A. Rizzi

Department of Information Engineering, Electronics and Telecommunications,
University of Rome “La Sapienza”, Via Eudossiana 18, 00184 Rome, Italy
e-mail: alessio.martino@uniroma1.it

A. Rizzi

e-mail: antonello.rizzi@uniroma1.it

A. Giuliani

Department of Environment and Health, Istituto Superiore di Sanità,
Via Regina Elena 299, 00161 Rome, Italy
e-mail: alessandro.giuliani@iss.it

Keywords Computational intelligence • Pattern recognition • Machine learning
Granular computing • Bioinformatics • Computational biology • Systems biology
Non-metric spaces analysis

1 Introduction

1.1 *Bioinformatics, Computational Intelligence and Pattern Recognition*

The word ‘bioinformatics’ took different meanings since its introduction around forty years ago [17]. The definition of an autonomous ‘bioinformatics’ field started with the need to efficiently analyse and store increasing amounts of sequence data. Consequently, in the first years of the application of computational science in biology, bioinformatics was mainly devoted to technical and instrumental problems with no relation at all with the core of biological sciences. Computational scientists were hired to give a service to biologists because ‘they were able to play with computers’ in a way not too dissimilar of any laboratory technician taking care of a spectrophotometer properly working.

It is worth noting that the relation between biology and statistical methodology (the first root of pattern recognition approaches in life sciences) started with completely different premises. From the beginning of their relation, in the first years of the last century, biology and statistics interacted on a peer-to-peer basis and many statistical tools were developed in the core of biological community (e.g. Ronald Fisher, one of the fathers of modern statistics, was a geneticist and he developed linear regression in the frame of human genetics and evolution studies [65, 84, 95]).

During the years, the relation of biology with bioinformatics became something more than a purely occasional affair and approached the ‘true-love wedding’ level of the one-hundred years lasting relation between biology and statistics. Notwithstanding that, the term ‘bioinformatics’ is still largely prevalent with respect to other terms lexically more suited for describing the growing maturity of Biology and Computational Intelligence relation, such as ‘computational biology’ and ‘systems biology’.

Besides the terminology, pattern recognition and computational intelligence techniques are nowadays gaining attention from the bioinformatics community [43, 71]. Many machine learning problems that can be instantiated in both biology and medicine are defined on domains in which each entry of the database at hand is a data structure far more complex than a plain real-valued feature vector, such as sequences, graphs, images or often even more complex structures arising from the concatenation of different data types (unconventional, structured data). Dealing with such structured domains usually demands to be able to define custom and meaningful (dis)similarity measures between elements in such unconventional domains, relying on sequence

and graph matching techniques. Specifically, networks (graphs) are nowadays the most powerful approaches to describe the complexity behind biological systems.

In fact, the application of computationally intensive methods to biological problems became strictly intermingled with the actual frontiers of biomedicine and went well beyond the biological polymers sequence analysis, directly tackling the archetypal form of biological objects from protein science to ecology: complex networks, interpreted as simplified, yet powerful, representations of complex systems.

Complex systems are everywhere in nature, as well as in most artificial systems designed and built by mankind (telecommunications and energy distribution systems, as instances). Complex systems are by far more frequent than ‘simple’ ones, which are the true outliers in our world. However, a precise definition of what should be a ‘complex system’ is still a disputable issue. This challenge is due to the fact that complex systems are nowadays a research topic faced by many different scientific areas, such as mathematics, biology, physics, chemistry and engineering, each one bringing its own point of view, concepts and terms into the discussion. Since 1995, when John Horgan published his famous paper entitled “From Complexity to Perplexity” [36] evidencing the lack of a shared and precise definition about complex systems, the debate is still well alive. However, most authors agree in considering the following characteristics as necessary conditions to consider a given system as ‘complex’:

- The system is composed by many mutually interactive elements
- Elements behaviour is characterised by nonlinear dynamics
- The graph representing the causal relationships between elements contains loops

Elements are usually defined as atomic entities at the semantic level chosen for system description. For example, proteins can be considered as atomic entities in the network of chemical reactions in a biological cell; neurons are the basic constituents of the brain, when focusing on purely computational issues; each individual can be considered as an atomic entity in an ecosystem or in a social network. These examples of complex systems underline a property frequently found in such systems, concerning the fact the usually complexity arises in the form of a hierarchical organisation, as nested Systems of Systems. From this last point of view, it is possible to consider causal relations between elements belonging to different levels in the hierarchical organisation. When the network of these relations contains a loop, sometimes it is referred to as ‘strange loop’, i.e. a causal loop between different levels of the hierarchy [35]. This property is strongly related with the emergence of the most interesting behaviours of a given Systems of Systems, when considered as a whole.

In a fundamental paper appeared in 1948 entitled “Science and Complexity” [92], Warren Weaver, one of the fathers of modern information science together with Claude Shannon, proposed a tri-partition of science styles.

Scientific themes can be sub-divided into:

1. Problems of simplicity
2. Problems of disorganised complexity
3. Problems of organised complexity

The first class (simplicity) roughly corresponds to problems that can be solved in terms of differential equations. These ‘simple problems’ are the ones allowing for a high degree of abstraction (e.g. a planet could be considered an abstract dimensionless ‘material point’ for sketching general gravitational laws on the pure basis of its mass and distance from the sun).

Problems belonging to the second class (disorganised complexity) allow for a higher degree of generalisation than first class problems without losing in precision. These problems imply a somewhat opposite style of reasoning: the efficiency does not stem from the possibility to get an efficient abstract description of the involved players, but from totally discarding such ‘atomic’ knowledge in favour of very coarse grain macroscopic descriptors corresponding to gross averages on a transfinite number of atomic elements. This is the case of thermodynamic parameters (e.g. pressure, volume, temperature, etc.). The two above mentioned approaches have drastic limitations of their applicability range: class 1 needs the presence of very few involved players interacting in a stable way with a practically null boundary conditions effect, whereas class 2 needs very large numbers of particles with only negligible interactions among them.

Problems of organised complexity (class 3) arise in all those situations in which many (even if not-so-many as in class 2) elements are involved with non-negligible interactions among them. This is the ‘middle kingdom’ of complexity, where biological systems live and where computational intelligence and pattern recognition can ‘make the difference’.

Network (or graph) is the archetype of organised complexity: a set of nodes (e.g. genes, brain areas, animal species) are each other connected by mutual correlations (edges). The wiring architecture of these graphs can vary in both space and time and it is of utmost importance to get quantitative similarities and differences among them. When graphs are adopted to represent only topological information concerning a set of objects and their relations, the network approach can roughly be described as the answer to the question “what can we derive from the sole knowledge of the wiring diagram of a system?” [28, 58].

The most crucial questions at the frontiers of biomedical sciences demands a reliable answer to the above question. Fields (just to name a few) that are increasing their formalisation in terms of network representations are: neuroscience at both clinical and basic research level [11, 68], biochemistry [5], cancer research [94], structural biology [46], ecology [27].

Moreover, when dealing with fully labelled graphs (where both nodes and edges are associated with possibly structured data), a fundamental topic is how to define proper dissimilarity measure between pairs of such patterns (the graph matching problem [47]).

Modelling a complex system is a matter of identifying the correct level of abstraction, which usually means to extract a hierarchy of information granules, searching for the level of the hierarchy better related to the semantic of the problem at hand. At any level, information granules are nodes of a network, so that the granulation process must deal with the problem of searching for frequent substructures in labelled graphs which, in turn, means to define algorithms able to automatically identify suitable

dissimilarity measures in graph spaces. To this aim granular computing techniques are nowadays a promising tool.

Keeping this general frame in mind, in order to fix clear boundaries to this Chapter, a general definition of computational intelligence and pattern recognition is sketched in the following.

1.2 Theoretical Background and Definitions

Computational Intelligence, formerly known as *Soft Computing* thanks to the seminal work [96], is a set of data processing techniques tolerant to imprecisions, uncertainty, partial truth and approximation (in the data and/or models), aimed to provide robust and low-cost solutions and to achieve tractability when dealing with complexity. Such toolbox includes mostly biologically-inspired algorithms, usually exploiting inductive reasoning (i.e. based on generative logic inferences, such as analogy and induction) [13]. Basically, in this toolbox it is possible to find:

- Artificial Neural Networks
- Fuzzy Logic and Neuro-Fuzzy Systems
- Evolutionary Computation and derivative-free optimisation metaheuristics, such as genetic algorithms and swarm intelligence

Such a (heterogeneous) set of computational tools are usually combined to design powerful data-driven modelling systems. Being able to synthesise a (predictive) model of a given (physical or even abstract) process P is a fundamental topic in all natural sciences, as well as in engineering.

Before the pervasive widespread of digital computing devices, modelling was performed ‘by hand’, mostly relying on field-experts (*analytical modelling*), consisting in identifying meaningful quantities and relations among them and finally writing a system of integro-differential equations as the final output. This implies a clear understanding of the process at hand to be modelled.

However, when a meaningful sampling S of the process P to be modelled is available, a second approach (*data-driven modelling*) consists in writing an algorithm (often suitable to be run on a Von Neumann computing architecture) able to automatically synthesise a model M of P according to some predefined optimality criteria. This modelling approach is nowadays usually referred to as *Machine Learning*. The design and development of such learning systems is basically an engineering problem.

A formal machine learning definition has been given in [59], where the author considers machine learning as the following, well-posed problem:

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

More broadly, machine learning can be defined as a (set of) complex intelligent processing system(s), usually defined by means of adaptive learning algorithms,

able to act without being explicitly programmed or, in other words, able to learn from data and experience.

Pattern recognition techniques fall under the machine learning umbrella, focusing on classification of objects in a given number of categories (classes). Indeed, pattern recognition includes a wide range of techniques employed to solve (properly said) classification problems and clustering problems. Broadly, pattern recognition techniques can generally be divided into two main families: *supervised* and *unsupervised learning*, both of which fall under the aforementioned data-driven modelling paradigm.

For a more formal definition, let us consider an orientated process $P : \mathcal{X} \rightarrow \mathcal{Y}$ where $\mathcal{X}$ is the input space (domain) and $\mathcal{Y}$ is the output space (codomain). Moreover, let $\langle x; y \rangle$ be a generic input-output sample drawn from P , i.e. $y = P(x)$. In supervised learning, a finite set S of input-output observations drawn from P are supposed to be known. Common supervised learning tasks can be divided into two families, depending on the output space nature: *classification* and *function approximation*. In classification, outputs take values from a set of categorical labels, each of which correspond to a given problem-related class (e.g. “sick” or “healthy” in a predictive diagnosis/medicine problem). Conversely, in function approximation (such as regression, interpolation, extrapolation, fitting) outputs take values usually in the real field. Formally, in the former case, $\mathcal{Y}$ is a discrete label set where it is not possible to establish any total ordering between its elements, whereas, in the latter case, $\mathcal{Y}$ can be considered as a normed space.

In unsupervised learning there are no output classes or labels and regularities have to be discovered by considering mutual relations between elements drawn from the input space only. One of the mostly acclaimed unsupervised learning approaches relies on data *clustering* [37]. Aim of a clustering algorithm is to discover groups (clusters) of patterns in such a way that similar pattern will fall into the same cluster, whereas dissimilar pattern will fall into different clusters. Formally, let S be a sampling of a non-orientated process P and let c be the number of clusters, constrained to $2 \leq c \leq |S|$. Aim of a clustering algorithm is to assign to every $x \in \mathcal{X}$ an integer $h \in [1, c]$ starting from the set of c clusters induced over S .

In both of these cases, the goal of a learning machine is to build a predictive model from observations, aiming to discover the underlying model structure. Moreover, learning machines must be able to generalise their discrimination capabilities to previously unseen patterns or, in plain terms, they must be able to assign a label (either a class label or a cluster label) to patterns not belonging to S .

For the sake of completeness, it is worth stressing that clustering and classification algorithms might as well co-operate and shall not be considered as two diametrically opposed techniques. For classification purposes, a rather common approach relies on clustering labelled data without considering their respective labels, then assigning a label to each cluster by considering, for example, the most frequent label amongst the patterns belonging to the cluster itself. Finally, each new pattern is classified according to the nearest cluster’s label. An example of such workflow can be found in [21, 22].

1.3 Chapter Scope

Aim of this Chapter is to review and discuss major issues when dealing with pattern recognition problems in non-metric spaces, namely input spaces for which a (dis)similarity measure might not be metric. As a case study, bioinformatics and computational biology-related problems will be investigated, since in these fields not only pattern recognition has emerged as a breakthrough discipline, but it is also very common to find structured data such as graphs or sequences which lie in non-metric spaces (see Sect. 1). Moreover, biological processes are excellent examples of complex systems, strongly suggesting the use of granular computing techniques for facing the challenging problem of (data-driven) model synthesis.

In Sect. 2 the data-driven modelling steps at the basis of pattern recognition problems will be described in detail, with particular emphasis on classification and clustering, underlying the role of computational intelligence techniques in designing pattern recognition systems.

Section 3 will regard non-metric spaces, remarking some examples of bioinformatics and computational biology-related problems in which structured data are commonly used. Moreover, some important issues when dealing with pattern recognition in non-metric spaces and possible solutions, including information granulation-based techniques, will be discussed.

In Sect. 4 some real case studies of bioinformatics/computational biology problems faced by means of pattern recognition techniques design to work in structured and non-metric domains will be summarised.

Finally, Sect. 5 will draw some conclusions, stressing major advantages of granular computing-based techniques over more ‘traditional’ approaches.

2 Machine Learning Systems Design

In conventional machine learning, a *pattern* is defined by a set of measures related to the original object to be represented, arranged in an array. Each entry (*feature*) is usually a real-valued variable. When a metric dissimilarity measure is implicitly or explicitly fixed in order to compare a pair of such simple data structures, usually it is referred to as a *feature vector*. The multi-dimensional space spanned by feature vectors forms the *feature space*. A well-defined feature space is able to facilitate the modelling process. For example, in the classification (supervised) case a well-designed feature space yields simpler decision surfaces in terms of structural complexity (smooth and regular).

Let us consider a plain supervised pattern recognition (classification) problem, as an instance of the more general machine data-driven modelling paradigm. Recalling Sect. 1.2, aim of a classification system is to assign an input pattern (represented by its feature vector) to one amongst the class labels defining the problem at hand.

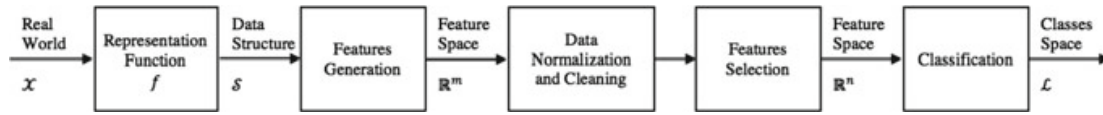


Fig. 1 A simplified pattern recognition system workflow

In Fig. 1, the main steps in order to build a classification system are summarised. First, real-world data, belonging to a generic (and possibly abstract) space $\mathcal{X}$ are casted into a proper data structure $\mathcal{S}$, processable by a computational device, by means of a representation function f which must ad-hoc be chosen for the problem at hand.

From structured data $\mathcal{S}$, a given number m of (usually numerical) features is extracted, thus casting data in $\mathcal{S}$ towards $\mathbb{R}^m$ (the aforementioned feature space).

The two following blocks are not mandatory, but they have been added for the sake of completeness and in order to take into account inevitable uncertainties in data collection and processing. The first block is in charge of data normalisation and cleaning: the former task is sometimes crucial in order to facilitate the classification algorithm under particular circumstances¹; the latter deals with missing and noisy data. An intuitive data cleaning task is, for example, outliers' removal.² Conversely, the Feature Selection block allows to select a significant subset of the previously generated features; indeed, as a general rule, the feature vector should be small, yet informative,³ in order to avoid undesired phenomena such as overfitting and/or the so-called *curse of dimensionality*. Further, it is recommended to get rid of unreliable features and correlations with existing features. At the end of this selection stage, feature vectors will lie in a (possibly) reduced features space $\mathbb{R}^n$, where $n \leq m$. Finally, the set of feature vectors will be used in order to train the classification system, with the final goal of estimating the correct label (identified, for the sake of ease, as an instance of a nominal value set $\mathcal{L}$ in Fig. 1) for any input vector.

For a better understanding of Fig. 1 and all of its steps, let us consider a real-world, Bioinformatics-related scenario, where $\mathcal{X}$ corresponds to the protein space (i.e. the set of real macromolecules). Let us suppose to represent proteins as graphs (cf. Sect. 3.1), then f is an (hypothetical) function which must convert macromolecules into graphs ($\mathcal{S}$). Fortunately, at least from a machine learning point of view, molecular

¹For example, let us consider a classification/clustering algorithm driven by the Euclidean distance. A common problem with the Euclidean distance is that features spanning a wider range of values have more influence in the resulting distance measure, therefore normalising all attributes in the same range (usually $[0, 1]$ or $[-1, +1]$) ensures fair contribution from all attributes, regardless of their original range.

²In Statistics, *outliers* are “anomalous data” that for a given dissimilarity measure lie far away from most observations.

³*Non sunt multiplicanda entia sine necessitate* (Entities are not to be multiplied without necessity), commonly known as “The Ockham’s Razor” Criterion (William of Ockham, circa 1287–1347). This criterion states that among a set of predicting models sharing the same performances, the simplest one (i.e. the one with the simplest decision surfaces) should be preferred. It is for sure one of the fundamental axioms for thoughtful and practical data-driven modelling.

biology helps: 3-dimensional protein structures, mainly gathered by crystallography, are available in online databases (e.g. Protein Data Bank [7]), therefore it is rather easy to build graph-based protein representations, either labelled or unlabelled on nodes and/or edges. The Features Generation block is in charge of extracting numerical features from graphs in $\mathcal{S}$ (cf. Sect. 3.2.1) which, after possible further processing, will be directly fed into the classification/clustering system.

The training phase for a classification system is a rather delicate task and it needs a separate discussion. Indeed, thanks to the training phase, the classification system learns how to map and discriminate input patterns according to their class labels. In other words, it learns the decision surfaces (decision regions boundaries) which separates patterns corresponding to different classes.

A usual procedure for measuring in a fair way the generalisation capability of a classification model consists in splitting the entire available dataset into two non-overlapping subsets, namely the Training Set and the Test Set. Specifically, as far as classification tasks are concerned, one shall figure both Training and Test Sets as composed by $\langle x; y \rangle$ pairs (see Sect. 1.2). The classification system, driven by a training algorithm which strictly depends on the chosen model (e.g. Support Vector Machine, Artificial Neural Network, K -Nearest Neighbours), will use the Training Set in order to learn the input-output mapping. The Test Set will then be used on such trained model, without further adaptive changes, in order to compute its performances (e.g. percentage of correctly classified patterns). For a thoughtful modelling, the two sets (albeit distinct) should satisfactorily represent the same statistical properties of the process to be modelled.

This double-split procedure, however, is not effective since every training algorithm depends on a set of parameters,⁴ which must be tuned with the ultimate goal of maximising the generalisation capability of the synthesised model. In order to find the optimal set of hyperparameters (i.e. model selection) a three-split procedure is usually employed: the whole dataset is split into three non-overlapping parts, namely Training Set, Validation Set and Test Set. The training algorithm, driven by the set of hyperparameters Γ , will again exploit the Training Set and its performances will be evaluated on the Validation Set. The parameters Γ will be tuned in order to maximise the performances on the Validation Set and once the optimal Γ^* has been found, the final performances will be evaluated on the Test Set.

In literature, several ways to perform the aforementioned search for Γ^* have been proposed, amongst which grid search, random search [6] and evolutionary optimisation-based techniques emerge (see Sect. 2.1).

When dealing with unsupervised learning, the scheme reported in Fig. 1 does not change significantly, apart from the rightmost block. Indeed, rather than feature a Classification algorithm, a Clustering algorithm must be placed instead. A clustering algorithm is in charge of returning groups of data (clusters) according to a given (dis)similarity measure and to a predefined objective function.

In literature, three main families of clustering algorithms can be found, which mainly differ for their objective function (i.e. according to which criterion clusters

⁴Also known as *hyperparameters* in the Machine Learning terminology.

should be discovered): *partitional clustering* (e.g. k -means [51, 52], k -medians [10], k -medoids [39]), which split the dataset into k non-overlapping partitions; *hierarchical clustering* (e.g. BIRCH [97], CURE [32]), where clusters are found by building a dendrogram in either top-down or bottom-up approach; *density-based clustering* (e.g. DBSCAN [24], OPTICS [3]), which detect clusters as the most dense regions of the dataset.

Clustering algorithms do need some parameters tuning as well. Selecting their respective optimal value(s) can be done according to some internal validation measures, such as the Silhouette [74] or the Davies-Bouldin Index [19]. Both manual or fully automatic tuning by means of evolutionary optimisation techniques can be employed in unsupervised learning as well.

2.1 Evolutive and Fully Automatic Approaches

Evolutionary optimisation metaheuristics such as genetic algorithms [30], particle swarm optimisation [40], ant colony optimisation [16] and simulated annealing [41], are one of the main topics under the Computational Intelligence umbrella (Sect. 1.2). Such metaheuristics are well suited when the objective function to be optimised is not known in closed-form and gradient-based methods turn to be unfeasible.⁵ Indeed, the decision boundary which separates two or more classes in a classification problem is determined thanks to a sampling of the boundary itself, namely the set of patterns which compose the dataset at hand, along with their respective class labels. As introduced in Sect. 2, they are often used in order to automatise the hyperparameters' tuning for classification and/or clustering algorithms. Further, they can help in conducting the feature selection phase (see Fig. 1). Indeed, one might ask which is the most relevant set of features in order to maximise the classification and/or clustering performances. To this end, evolutionary optimisation metaheuristics play a huge role.

Let us consider a genetic algorithm as an example. One can consider the genetic code to have the form $[I, \mathbf{w}]$ where I , as in Sect. 2, is a set of hyperparameters for the clustering/classification algorithm at hand, whereas $\mathbf{w}$ is an m -length real valued vector which tunes the (dis)similarity measure, core of the algorithm itself.

As far as classification tasks are concerned, a typical workflow might consist in letting each individual in the evolving population to be considered for training the classification model on the Training Set using both the hyperparameters and the (dis)similarity weights specified by its genetic code. The classification model's performance will later be evaluated on the Validation Set and such performance will serve as (part of) the fitness function.⁶ Trivially, at the end of the evolutionary

⁵That is why evolutionary optimisation metaheuristics fall within the *derivative-free* methods.

⁶A common choice for a genetic algorithm fitness function takes into account both the model performance and its structural complexity. Specifically, whilst the former should be maximised, the latter should be minimised in order to avoid overfitting (cf. the Ockham's Razor Criterion).

stage, the best individual will be the one which maximises the performances on the Validation Set and its final performances will be evaluated on the Test Set. An example of such workflow can be found in [55] for classification algorithms and in [21, 22] for re-adaptation of clustering algorithms for classification purposes.

When dealing with clustering algorithms, the overall workflow does not change significantly. However, each individual will process the entire dataset according to the parameters stored in its genetic code and, similarly, since the performances cannot rely on any ground-truth labels, other internal validation measures should be used as the fitness function. An overview of clustering with evolutionary-driven feature selection can be found in [1].

It is worth stressing that, in both cases, the resulting best individual's genetic code contains the set of hyperparameters Γ^* which, along with the weights vector, maximise the algorithm's performances. Specifically, the latter deserves some further notes: if one considers $\mathbf{w} \in [0, 1]^m$, such vector acts as a feature selector, where 0's correspond to features which will not be considered in the (dis)similarity measure, and 1's correspond to features which, conversely, will be considered. The subset of n elements for which $\mathbf{w}$ is not-null can be seen as the reduced features space.

3 Dealing with Non-metric Spaces

So far, the design of a pattern recognition system has been described in its standard and most common form, where patterns are represented by means of real-valued vectors. In these cases, any Minkowski-based (e.g. Euclidean) distances can be good and straightforward candidates. Moreover, such (dis)similarity measures are metric.

Formally, a dissimilarity measure d defined on a generic space $\mathcal{S}$ is a function $d : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ satisfying the following properties:

1.
$$\exists d_0 \in \mathbb{R} \text{ such that } -\infty < d_0 \leq d(x, y) < \infty \quad (1)$$

2.
$$d(x, x) = d_0 \quad (2)$$

3.
$$d(x, y) = d(y, x) \quad (3)$$

for any two objects $x, y \in \mathcal{S}$. If, alongside Eqs. (1)–(3), d satisfies the following two properties

1.
$$d(x, y) = d_0 \text{ if and only if } x = y \quad (4)$$

2.
$$d(x, z) \leq d(x, y) + d(y, z) \quad (5)$$

for any three objects $x, y, z \in \mathcal{S}$, then d is said to be *metric*.

Similarly, in $\mathcal{S}$ it is possible to define a similarity measure $s : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ whether it satisfies the following properties:

$$1. \quad \exists s_0 \in \mathbb{R} \text{ such that } -\infty < s(x, y) \leq s_0 < \infty \quad (6)$$

$$2. \quad s(x, x) = s_0 \quad (7)$$

$$3. \quad s(x, y) = s(y, x) \quad (8)$$

for any two objects $x, y \in \mathcal{S}$. If, alongside Eqs. (6)–(8), s satisfies the following two properties

$$1. \quad s(x, y) = s_0 \text{ if and only if } x = y \quad (9)$$

$$2. \quad s(x, y) \cdot s(y, z) \leq (s(x, y) + s(y, z)) \cdot s(x, z) \quad (10)$$

for any three objects $x, y, z \in \mathcal{S}$, then s is said to be *metric*.

Moreover, it is possible to prove that:

Theorem 1 *If d is a metric dissimilarity measure with $d(x, y) > 0, \forall x, y \in \mathcal{S}$, then $s = a/d$ is a metric similarity measure for $a > 0$.*

Theorem 2 *If d is a metric dissimilarity measure, let d_{max} be the maximum pairwise distance between elements in $\mathcal{S}$, then $s = d_{max} - d$ is a metric similarity measure.*

The above two theorems demonstrate that, under particular circumstances, one can easily ‘switch’ between (metric) similarity and dissimilarity measures in a given input space. Indeed, dissimilarity measures quantify the degree of separation, whereas similarity measures estimate the complementary notion of closeness.⁷

3.1 Examples of Structured Data in Bioinformatics and Computational Biology

Dealing with non-metric spaces is a common issue when unconventional (structured) data, such as graphs or sequences, are considered as the input domain.

As introduced in Sect. 1, especially in bioinformatics and computational biology, patterns are usually described by means of data structures more complex than plain real-valued feature vectors: some common examples include proteins, DNA and RNA, metabolic pathways and brain connectivity networks.

⁷That is why in most of the Chapter, unless explicitly specified, the generic term *(dis)similarity* will be used.

Indeed, DNA and RNA transcripts are usually described as sequences of 4 possible nucleotides: adenine (A), cytosine (C), thymine (T), guanine (G) for DNA and adenine (A), cytosine (C), uracil (U), guanine (G) for RNA.

Proteins can be described by both sequences and graphs. The former representation is more straightforward: a protein is encoded in genes (DNA sequence) which is transcribed into pre-messenger RNA (RNA sequence). The RNA transcript is loaded into the ribosome which reads three nucleotides at the time (codons) and converts each triplet into one of the 20 amino-acids. It is clear that there exist up to three sequence-based protein representations, which mainly differ from their alphabet (4 nucleotides vs. 20 amino-acids) and their length (nucleotide-based sequences are three times longer than amino-acid-based ones). The protein representation as a sequence of amino-acids is also known as *primary structure*.

Graph representations result from a biological step forward in protein biosynthesis. Indeed, when the protein leaves the ribosome, a process called *protein folding* starts, during which the protein folds on itself, leading to a unique three-dimensional structure (also known as the *tertiary structure*). Protein Contact Networks [23] are an example of graph-based protein representation [29], where nodes correspond to amino-acids and edges between any two nodes exist whether their Euclidean distance falls within a given range, typically [4, 8]Å (e.g. [44–46, 54, 55]). The lower bound is usually considered in order to discard trivial backbone first-order neighbour contacts (i.e. sequence proximity), whereas the upper bound is usually defined by taking into account the peptide bonds geometry; indeed, 8Å roughly correspond to two peptide bond lengths or, equivalently, to two Van der Waals radii between residues' alpha-carbon atoms. In their original formulation, Protein Contact Networks are undirected graphs with no labels on nodes and edges: information regarding the type of amino-acid and their respective proximities are deliberately discarded in order to focus on proteins' topological structure and their complex nature.

Metabolic pathways are mainly described by graphs as they can be seen as protein networks and chemical networks. In the former, nodes correspond to proteins, whereas links correspond to physical (protein-protein interaction) and/or functional relations between them. In the latter, links correspond to chemical reactions (catalysed by specific enzymes) transforming the nodes (organic molecules produced – or used – in the metabolic processes) at their extremities into one another.

To our knowledge, the brain is probably the most complex circuit in the Universe, a complex system of nested subsystems, usually modelled as a network, since its functions strictly depend on the anatomical and functional wiring of billions of neurons [11, 31, 75, 88].

While in the case of brain networks based on the anatomical links between parts of the brains (macroscopic scale) or between single neurons in a small brain portion (microscopic scale) it is possible to rely on the assumption of a certain degree of invariance in time,⁸ this is not the case as for functional brain networks (e.g. related to areas metabolic activity correlations observed by Nuclear Magnetic Resonance

⁸Indeed, the anatomical structure changes in the order of months/years depending on the age of subjects.

(NMR) or Positron Emission Tomography (PET)) that modify their wiring patterns on very short time scales [25, 81, 83].

Spontaneous neuronal activity in resting state depends on dynamic communication between brain regions allowing both local segregation and long-distance integration of neuronal processes. Several functional networks in which temporally or spatially coherent connections exist [18]. These networks have been identified in healthy subjects by functional Magnetic Resonance Imaging (fMRI) and by PET, respectively. Both these techniques deal with the quantification of metabolic rate correlation across different brain areas. Specifically, fMRI measures as ‘marker’ the variation of the amount of blood flowing across brain areas (coupled with metabolism by the dynamics between oxidised and reduced haemoglobin) [26], whereas PET focuses on the different metabolic rate of glucose (the most important energy source for brain cells) across different brain areas [79].

Both fMRI and PET techniques define a brain connectivity network in correlative terms: two nodes i, j of the network are linked by an edge if the metabolic rates of nodes i and j are each other correlated (given the quantitative character of the measures used by Pearson correlation coefficient metrics).

3.2 *Pattern Recognition in Non-metric Spaces*

When dealing with complex data structures such as graphs or sequences, the scheme from Fig. 1 should be revisited since patterns cannot be directly described by means of real-valued vectors.

In literature, three major approaches can be found [49, 50]:

1. directly working in the input data structure space, by defining ad-hoc (dis) similarity measures
2. by means of kernel transformations and kernel machines
3. by defining an embedding function from the input space to real-valued vectors

These approaches are summarised in Fig. 2 and, along with the ‘classical’ Feature Generation procedure, will be discussed separately.

3.2.1 *Classical Processing Chain by Feature Generation*

Recalling Sect. 2, given a generic and possibly non-metric input space $\mathcal{S}$, the most straightforward approach consists in defining a mapping function $\phi : \mathcal{S} \rightarrow \mathbb{R}^n$ specifically designed for the input space at hand. In this section, three examples of mapping function suitable for dealing with graphs will be described. Moreover, the additional challenge of dealing with patterns of different size in $\mathcal{S}$ will be discussed. For the sake of argument, let us consider graphs representing proteins since, notably, proteins have different sizes both in terms of primary and tertiary structures, meaning that

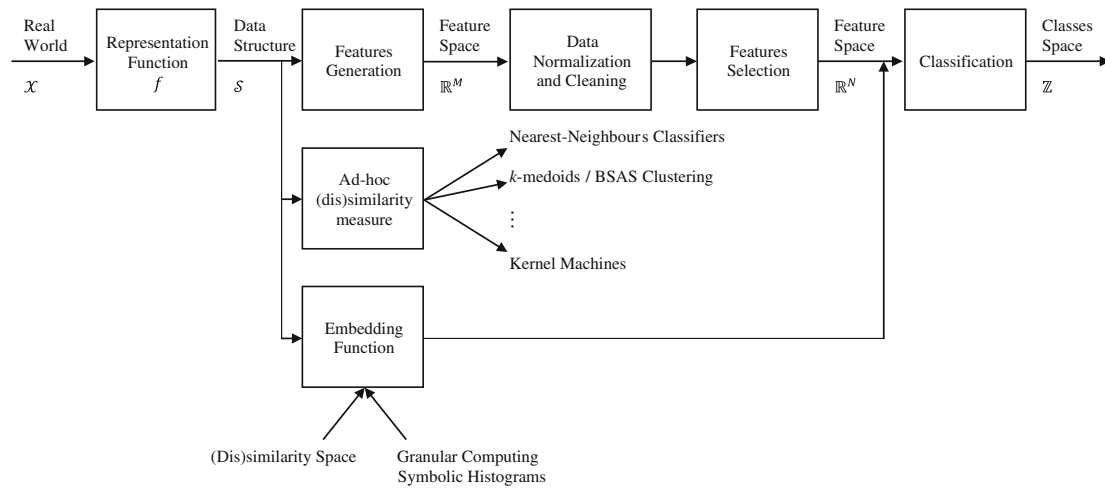


Fig. 2 Overview of possible approaches for pattern recognition in non-metric spaces

their amino-acids sequences and, by extension, their folded 3-dimensional structures have different size.

In [46, 54, 55] a mapping function based on graphs spectra has been proposed. Specifically, each graph has been described by means of its normalised spectrum, i.e. the set of eigenvalues evaluated from its corresponding normalised Laplacian matrix [38]. Such eigenvalues lie in range $[0, 2]$, making such approach suitable for comparing graphs with different sizes. However, the number of eigenvalues composing the spectrum equals the number of nodes in the graph and, in order to overcome this problem, it is possible to estimate the spectral density by means of a kernel density estimator [63]. In this way, the distance between any two graphs can be evaluated by integrating the squared difference between their respective spectral densities all over the $[0, 2]$ range. This evaluation can be performed also in the discrete domain by sampling a finite number (n) of points from such spectral densities (being the support domain equal for all graphs, regardless of their respective sizes). In such finite domain, the distance between two graphs can be evaluated as the considered distance (e.g. Euclidean) between their respective sets of samples.

In [45] a feature-engineering based approach has been employed in order to predict proteins' solubility starting from their topological structures. Several features have been manually selected such as the number of nodes and edges, the number of protein chains, some centrality measures (e.g. closeness and degree) and some physical characteristics (e.g. heat trace, energy). The union of these features forms the feature vector for a given graph.

Other feature extraction procedure(s) can rely on a rather novel field known as Topological Data Analysis [12, 91]. Topological Data Analysis consists in a set of techniques in order to extract information from data starting from topological information by means of dimensionality reduction, manifold estimation and persistent homology in order to study how components lying in a given multidimensional space are connected (e.g. in terms of loops and multidimensional surfaces). This

can be done by starting either by so-called *point clouds*⁹ or by explicitly providing a similarity matrix (cf. the *Kernel Methods* paragraph). Albeit this field has very solid and rigorous foundations (from algebraic topology to pure mathematics), there are very few ‘numerical’ features which can be extracted, mainly the sequence of Betti numbers. Formally, the i th Betti number corresponds to the rank of the i th homology group. In plain terms, the i th Betti number corresponds to the number of i -dimensional ‘holes’ in a topological surface. For example, let us consider a three-dimensional graph, its first three Betti numbers have the following interpretations: the 0-th Betti number corresponds to the number of connected components in the graph; the 1-st Betti number corresponds to the number of 1-dimensional holes (e.g. circular holes); the 2-nd Betti number corresponds to the number of 2-dimensional holes (e.g. cavities). If the multidimensional space under analysis has a finite dimension, the Betti numbers vanish after the spatial dimension (e.g. the number of 4-dimensional holes in a 3-dimensional space is always equal to zero). Whether the Betti numbers can be an effective mapping function for pattern recognition purposes it still an open question.

3.2.2 Ad-Hoc Dissimilarities in the Input Space

One of the mostly acclaimed ad-hoc (dis)similarity measures for structured data are the so-called *edit distances*, according to which the distance between two objects is given by the minimum number of atomic edit operations (usually insertions, deletions and substitutions of elements in the sequence) needed to transform the first object into the second object. As regards strings, the Levenshtein distance [42] is the seminal example of an edit distance, which can be seen as a generalised Hamming distance¹⁰ [33].

The same approach can be used to define dissimilarity measures between graphs as well, leading to the Graph Edit Distances [47, 60], which inherit the idea at the basis of the Levenshtein distance, defining atomic edit operations in both the sets of nodes and edges. In many pattern recognition applications defined in sequences domains the Dynamic Time Warping [76] can be adopted, where the sequence support is explicitly related with time. Specifically, by applying a non-linear distortion on the support independent variable (time), it returns the optimal correspondence (i.e. similarity) between two sequences.

Amongst these methods, the Levenshtein/Hamming distances are well-known to be metric; the same might not be true for Graph Edit Distances (as they might violate the symmetry property – cf. Eqs. (3) and (8)) and Dynamic Time Warping (as it

⁹A finite set of points equipped with a notion of distance in a finite multidimensional space.

¹⁰According to which the distance between two strings of equal length is given by the number of mismatches.

might violate the triangle inequality – cf. Eqs. (5) and (10)). Also, edit distances are not recommended if patterns have a high dimension variability as deletion/insertion costs can easily prevail over substitution costs.

On the plus side, however, methods based on ad-hoc (dis)similarity measures notably work in cases where the pattern recognition system does not need to define an algebraic structure on the input space. For example, let us consider a clustering task to be performed directly into a non-metric space with an a-priori chosen (dis)similarity measure. Algorithms such as k -means or k -medians cannot be considered as good candidates since the former needs to evaluate the component-wise mean amongst the pattern in a given cluster in order to evaluate its representative, whereas the latter needs to evaluate the component-wise median. Therefore, the need to define a meaningful algebraic structure emerges which, however, turns into a non-sense as concerns non-metric input spaces. Suitable clustering algorithm candidates for dealing with non-metric spaces are k -medoids, as discussed in [56], and BSAS [85], since they do rely on (dis)similarity measures only in order to form clusters and to update their representatives. Similarly, as far as classification algorithms are concerned, a good candidate is the K -Nearest Neighbour, since it classifies patterns according to their respective distances rather than defining operators such as the inner product, mandatory in Artificial Neural Networks, or Support Vector Machines, whether equipped with an ad-hoc kernel transformation (see the *Kernel Methods* paragraph).

Kernel Methods

Typically, kernel methods can safely be employed whether the input space has an underlying Euclidean geometry, since they are based on inner products. Given a pair of patterns $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, their inner product is given by:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i \cdot y_i \quad (11)$$

Further, let us consider the instances matrix for the dataset at hand, $\mathbf{X} \in \mathbb{R}^{N_P \times n}$, namely a matrix where each row corresponds to a given pattern. Let N_P indicate the number of patterns. It is possible to define the *kernel matrix*¹¹ as

$$\mathbf{K}_{i,j} = \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad (12)$$

or, in batch fashion

$$\mathbf{K} = \mathbf{X} \cdot \mathbf{X}^T \quad (13)$$

¹¹ Also known as the *Gram Matrix*, after Danish mathematician Jørgen Pedersen Gram.

More in general, let k be a symmetric and positive semi-definite kernel function from the input space at hand $\mathcal{S}$ towards $\mathbb{R}$, i.e. $k : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$ such that:

$$k(\mathbf{x}_i, \mathbf{x}_j) = k(\mathbf{x}_j, \mathbf{x}_i) \quad \forall \mathbf{x}_i, \mathbf{x}_j \in \mathbf{X} \quad (14)$$

$$\sum_{i=1}^{N_P} \sum_{j=1}^{N_P} c_i c_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \forall c_i, c_j \in \mathbb{R}, \forall \mathbf{x}_i, \mathbf{x}_j \in \mathbf{X} \quad (15)$$

As in the inner product case, starting from $k(\mathbf{x}_i, \mathbf{x}_j)$ one can easily evaluate the Kernel Matrix as

$$\mathbf{K}_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j) \quad (16)$$

and if $\mathbf{K}$ is a positive semi-definite kernel matrix, then k is a positive semi-definite kernel function. One of the most intriguing property of kernel methods relies in the so-called *kernel trick* [77]: kernel of the form (14)–(15) are also known as *Mercer's kernels* since they satisfy the Mercer's theorem [57]; they can be seen as the inner product evaluation on a (possibly) infinite-dimensional and usually unknown Hilbert space $\mathcal{H}$. The kernel trick is usually defined by means of the following, seminal equation:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \psi(\mathbf{x}_i), \psi(\mathbf{x}_j) \rangle_{\mathcal{H}} \quad (17)$$

where $\psi : \mathcal{S} \rightarrow \mathcal{H}$ is the implicit and usually unknown mapping function.

Several positive semi-definite functions commonly used as kernels include the linear, exponential, radial basis function and polynomial [77, 78], which are usually employed in kernel machines, such as (non-linear) Support Vector Machines.

However, in many cases, defining the kernel function might not be easy, especially when dealing with non-metric spaces. Regardless of the nature of the input space, it is possible to evaluate the similarity matrix (cf. Sect. 3) $\mathbf{S} \in \mathbb{R}^{N_P \times N_P}$ where

$$\mathbf{S}_{i,j} = s(\mathbf{x}_i, \mathbf{x}_j) \quad (18)$$

If s is a metric similarity measure, it is possible to directly use $\mathbf{S}$ as the kernel matrix, as suggested in [15], or include similarities in widely-known kernel functions (e.g. radial basis function), as suggested in [77].

Conversely, if the (dis)similarity measure is not metric, two mainstream approaches can be followed. The former relies on moving the pattern recognition problem towards a dissimilarity space (as explained in the next paragraph), the latter relies on ‘modifying’ the similarity matrix in order to be a valid kernel matrix (i.e. satisfying Mercer's theorem) [14, 15, 89].

Embedding Functions and Information Granulation

Embedding functions can be seen as particular cases of mapping functions as defined in the *Classical processing chain by Feature Generation* paragraph. Indeed, while both of them aim at moving the problem from a generic input space $\mathcal{S}$ towards

$\mathbb{R}^n$, embedding functions, at least in this context, do rely on other patterns or on substructures extracted from the dataset at hand in order to build such mapping.

A first example of embedding consists in moving the pattern recognition problem into a *dissimilarity space* [66].

In turn, dissimilarity representations can follow two further approaches:

1. Each pattern is described by its own row¹² from the similarity matrix $\mathbf{S}$ (cf. the *Kernel Methods* paragraph); that is, each pattern is described by the distance(s) vector with respect to other patterns (including self-distance)
2. Each pattern is described by the distance(s) vector with respect to a given number of representatives drawn from the input space at hand. Certainly, the selection of such representatives is a crucial task since a) they must well-characterise the decision boundary between patterns in the input space and b) there should be few of those since the number of representatives has a major impact on the model complexity. Representatives selection heuristics range from class-aware random selections to clustering procedures directly in the input space [48] (cf. the *Ad-hoc Dissimilarities in the Input Space* paragraph).

Regardless of which of the two methods is employed, a dissimilarity space can be equipped with algebraic structures and operators, such as the inner product, in order to be suitable with traditional kernel methods [48]. But, more in general, since patterns are now casted in $\mathbb{R}^{N_p}$ (former case) or $\mathbb{R}^R$ (latter case – where R indicates the number of representatives), any “standard” pattern recognition algorithm can be used.

In order to introduce the embedding by means of substructures, let us introduce widely known embedding functions for sequences. Since sequences are finite collections of objects drawn from a finite alphabet (cf. RNA/DNA sequences or proteins’ primary structure, Sect. 3.1) one of the most intuitive approaches relies on histograms. Indeed, a sequence can effectively be described as the number of occurrences of any alphabet symbol within the sequence itself. For ‘simple’ sequences such as nucleotides or amino-acids sequences, histograms defined as above suffice. For example, in [90] a double experiment has been proposed in order to classify proteins starting from their primary structure according to their physiological role; in a first experiment, each protein is described by the number of occurrences of each amino-acid within the primary structure and, in a second experiment, such histogram-based representation has been extended to triplets of amino-acids in order to take into account also information about proximity and ordering. Further, in [53], the histogram-based representation considers pairs of amino-acids whose distance along the protein backbone is within a minimum and maximum value, a-priori defined.

For more complex sequences such as sentences or entire text documents, bag-of-words and word-count models have been proposed, where the alphabet is composed by the set of unique words in the sentence or document. ‘Complex sequences’ such

¹²If the similarity measure at hand is not symmetric, patterns’ distance vectors as taken by rows or columns will be different. In order to overcome this problem, one can ‘force’ a similarity measure to be symmetric by considering $\mathbf{S} := (\mathbf{S} + \mathbf{S}^T)/2$ (e.g. [14]).

as sentences or entire text documents are rather rare (if non-existent altogether) in bioinformatics as such, but bag-of-words models, along with statistical and/or machine learning techniques, have been successfully employed for health analysis and forecasting (e.g. [82] for anastomosis leakage detection, [93] for diabetes-related notes in electronic health records).

In the last years, granular computing [4] emerged as a novel and promising information processing paradigm. In granular computing, atomic quantities known as *information granules* have to be extracted in order to be further studied and analysed, for gathering useful knowledge and insights from data, but finding the adequate level of abstraction for the problem at hand might be a challenging task. Along with symbolic histograms, granular computing can play the role of a promising data-driven framework which can simultaneously deal with embedding functions in non-metric spaces and knowledge discovery. In [8, 9, 69, 72] have been proposed fully automated data-driven and granular computing-based classification systems both for graphs and sequences. These systems are composed by four main macroblocks: motifs extractor, granulator, embedder and classifier.

The motifs extractor is in charge of extracting, according to some heuristics (possibly exhaustively), substructures (i.e. subgraphs/subsequences) from the dataset at hand.

The set of motifs is then forwarded to the granulator which runs a clustering algorithm on it, relying on a suitable inexact matching procedure (i.e. on a given dissimilarity measure in the substructures space), yielding a set of frequent sub-structures (clusters), whose representatives can be considered as candidate information granules (symbols). It is worth stressing that the clustering algorithm works in the input space since motifs are frequent substructures, and that free-clustering algorithms such as BSAS should be preferred, in order to automatically return a suitable number of clusters, avoiding to set it in advance. Further, since the input space might not be metric, a suitable cluster's representative is the medoid (or MinSoD) [20, 56].

The set of information granules are the main input for the embedder block which, according to the *symbolic histograms* approach, maps each pattern into an integer-valued vector. Specifically, each pattern is represented as the number of occurrences of each information granule within the pattern itself. The embedder, therefore, returns a set of vectors which can feed any standard pattern recognition algorithm for classification or clustering purposes.

The whole cascade is driven by a genetic algorithm, following the workflow as described in Sect. 2.1, in order to maximise the classifier's performances. The genetic algorithm acts as an orchestrator, and is in charge of optimising the final classifier synthesis, accomplishing two tasks: under an algorithmic point of view, it automatically tunes the clustering algorithm and possible (dis)similarity measure parameters, maximising the classifier's performances and selecting the subset of information granules better related with the classification task at hand (cf. Feature Selection block in Fig. 1); under a knowledge discovery point of view, since it returns the (sub)optimal set of information granules for the problem at hand.

The latter deserves some further observations. It is clear that embracing a granular computing/symbolic histograms approach is more computationally expensive

than any other technique discussed so far. Indeed, the embedding procedure requires a clustering phase, searching for candidate granules. Even for small datasets, an exhaustive approach for the extractor might be unfeasible (since its complexity is combinatorial with respect to the pattern size and the substructure order), and it must be replaced by a stochastic approach (random subsampling). Moreover, when dealing with sequences or graphs, the (dis)similarity measure adopted by the core clustering procedure is by far more computationally demanding with respect to Euclidian distance performed on plain real-valued vectors. Furthermore, the selection of the most informative information granules, as well as of the best (dis)similarity measure parameters, demands additional computational burden by the evolutionary optimisation, since for every candidate solution it is needed to launch a full classification model synthesis procedure (for example a Support Vector Machine) in order to evaluate its fitness, computed as the performance of the classifier on the Validation Set (cf. Sect. 2.1). For these reasons, the symbolic histograms approach is practically feasible only when relying on parallel/distributed computing software/hardware environments.

But, on the plus side, granular computing-based techniques unleash an invaluable potential thanks to information granules. Indeed, if the training procedure yields a classification model with satisfying performances, able to correctly discriminate the input patterns for the problem at hand, the resulting information granules subset brings useful knowledge on the problem at hand, since information granules are at the basis of the embedding feature space. Information granules selected by the evolutionary optimisation are therefore responsible for the final definition of decision surfaces in that space and, consequently, they can show useful information that can be exploited by field-experts. This is the main advantage of granular computing techniques with respect to competitive approaches: extracting automatically meaningful information granules is useful both under an algorithmic point of view and under the application field point of view (biology, in this case).

As a more concrete example, let us consider a metabolic pathways problem, where metabolic pathways are described by graphs as in Sect. 3.1. One of the information granules might be the citric acid cycle.¹³ The Krebs cycle (in network terms, a motif with a set of nodes lined to form a closed loop) is driven by oxygen and therefore it might be a key granule in order to discriminate between aerobic and anaerobic organisms. For this example a well-known chemical reaction has been considered, but the opposite might also happen: indeed, information granules can *pose* questions other than *confirm* statements: why these information granules are considered as significant for the discrimination/classification problem at hand?

¹³ Also known as the *Krebs cycle*.

4 Case Studies and Applications

In most of the cases introduced in Sect. 3.1, it is almost impossible to project the analysed objects into a proper metric space spanned by a shared set of descriptors without considering some global features (e.g. classical network invariants like degree, characteristic length, closeness centrality, etc.) and thus losing a considerable part of information linked to ‘who-is-connected-with-whom’. On the contrary, such information can be easily recovered projecting the objects into a non-metric space defined by motifs and/or frequent substructures (Sect. 3.2.2).

The need of a non-metric approach is evident in many biologically relevant cases. This need not necessarily derive by the lack of a common feature space, but it is motivated by the importance to individuate particular motifs endowed with a meaningful semantics.

In the field of protein sequence analysis this is the case of the identification of ‘natively unfolded’ tracts. This is a particularly intriguing problem in structural biology [87]. Until the end of last century, the general view of structure/function relation in protein molecules was apparently straightforward (cf. Sect. 3.1): protein primary structures correspond to the amino-acid residues linear ordering along the sequence. The primary structure determines both the mutual position of nearby (secondary structure) and distant along the sequence amino-acid residues (tertiary structure). The specific 3-dimensional arrangement of the protein molecule in turn determines its physiological role [70]. This view was questioned some years ago [87] by the discovery of ‘natively unfolded’ proteins that are molecules that do not have a definite 3-dimensional structure but that, on the contrary, remain in a random coil state until they interact with some partners (e.g. other proteins) and, after the binding, assume a specific 3-dimensional configuration. The same natively unfolded protein (and thus with only one specific sequence) can assume completely different 3-dimensional structures (and functions) depending on the different partners it interacts with. All the vital functions of a cell are managed by the creation of aggregates of different proteins generating a sort of nano-machine performing a specialised task (e.g. energy production, biosynthesis, immune response, DNA repair and duplication, etc.), where natively unfolded proteins are the ‘hubs’ of such protein-protein interaction networks, given their ability to change structure ‘on demand’ and thus to participate to different nano-machines (protein aggregations) [80]. Besides proteins that are natively unfolded in their entirety, all the proteins do have (smaller or longer) tracts that are natively unfolded corresponding to their interaction sites. If the goal is to modify the behaviour of a protein aggregate for a therapeutic intervention (e.g. by a drug binding to the protein molecule) it is of utmost importance to recognise such natively unfolded parts of the molecule from their sequence.

This is a very challenging task for classical machine learning approaches, due to the following reasons:

1. The context dependence of the problem: the same subsequence can be natively unfolded in protein *A* and perfectly folded in protein *B* due the general properties of the entire protein molecule [67]

2. The ambiguous character of the definition of ‘unfolding’: many of the so-called unfolded proteins (or tracts) could be only highly flexible systems that have only one preferred fold without structuring on-demand [34]
3. The dependence on the chemico-physical micro-environment the protein experiences (i.e. pH, molecular crowding, etc.) deciding the disordered/ordered condition [34]
4. The highly variable length of the disordered patterns [34]

This is why (even if never defining explicitly in these terms) all the tentative solutions of the problem used non-metrics approaches that in turn allowed to both select some ‘relatively context independent unfolded motifs’ and individuating some regularities in these motifs [73].

A somewhat related problem is to predict the relative solubility in water of protein molecules. Again, there exist a similar context dependences of the disordered/ordered case and, in [44], the problem was approached by considering several different representations. The protein folding problem has interested biologists for many years: if the native protein structure is ‘encoded’ in its primary structure, is it possible to predict its folded state? Relative solubility in water is the major feature for proteins’ folding propensity. However, some proteins spontaneously fold, whereas other proteins need so-called *chaperones*¹⁴ in order to fold correctly.

Recall from Sect. 3.1 that a protein can be described in different ways by either taking into account its primary or tertiary structure; therefore in [44] a subset of the *Escherichia Coli* proteome has been considered in three different representations: the plain primary structure; an ‘extended’ Protein Contact Network representation (cf. Sect. 3.1) where labels exist on both nodes and edges (nodes labels correspond to one of the 20 amino-acids, edges labels correspond to the Euclidean distance between the two vertices at their extremities); a serialised version of the graph-representation, where each vertex is associate with a 3-dimensional real-valued vector derived from the graph transition matrix. The goal was to predict the relative water solubility of each protein *in vitro* (i.e. without the help of chaperones). Given that water solubility encompasses the ability to reach of a correctly folded structure, this prediction task can be considered as an explorative study in the chemico-physical drivers of folding process.

The different representations allowed us to grasp different aspects of ‘relative folding propensity’ of proteins, being the extended Protein Contact Network the most promising representation.

The impossibility to design a data set on a shared feature space (and consequently the need of non-metric approaches) is evident in neuroscience in the case of comparing different brain connectivity networks [88]. Recall from Sect. 3.1, both fMRI and PET outputs are images of the brain: a quantitative value is attached to each voxel corresponding to the entity of metabolism activity in that location. The voxels are in the order of tens of thousands and their actual quantitative value is not rele-

¹⁴Protein molecules driving the folding of other protein systems.

vant per-se¹⁵: what differentiates healthy and pathological subjects is the degree of organisation (correlation among areas) of the system. The selective breakdown of intrinsic brain networks during the progression from the healthy state to mild cognitive impairment to Alzheimer's disease has been observed using both fMRI and PET. Using the single voxels as nodes can be highly misleading in the comparison of images across patients: not only their high number produces networks very difficult to analyse but the pairing of the voxels across different subjects (i.e. to recognise that the j -th voxel of patient A corresponds to the j -th voxel of patient B) is virtually impossible. To solve the problem anatomical knowledge is considered: the physician segments the brain image into ROIs (Region of Interest) correspondent to the well-known anatomical areas of the brain (e.g. hippocampus, amygdala, cerebellum, etc.) that all patients do have, so ROIs become the nodes and edges correspond to the scoring of a strong correlation between pairs of ROIs.

Alzheimer's disease risk scales with the progressive disruption of 'long range' correlations in favour of 'small scale' correlation between nearby areas [62]. This implies that for discriminating different risk levels it is not possible to rely on shared 'global correlation measures' on the brain, nor on the focusing on 'specific relations' between key areas because they can be very different across different patients, while maintaining the above described pattern of 'decrease in long range and increase in small range correlations'. This situation is solved by non-metric approaches, in which different brain connectivity networks are compared on the basis of the dynamics of 'attachment'-'detachment' from the giant component of the network (the bulk of connected ROIs) on a subject by subject basis [61].

In the case of brain connectivity studies, computational intelligence is having a great expansion and the search for suitable context-dependent metrics for comparing different conditions is highly debated in both clinical and basic research communities.

5 Conclusions and Future Directions

In synthesis, we can surely affirm that non-metric approaches rely on a sufficiently stable and reliable theoretical basis implemented on very efficient algorithms. On the other hand, the 'biological side' generates an ever-increasing amount of data amenable to be faced by computational intelligence approaches. The crucial point (deciding for the success/failure of the particular application) is the choice of a representation located at the most 'fruitful' level of biological organisation. The search for the scale maximising 'non-trivial determinism' is a crucial issue in applied statistics [64] and roughly corresponds to the search of the level where the number (and strength) of correlations between the different pieces of information (e.g. different descriptors) reaches a maximum.

¹⁵Indeed, the absolute entity of metabolic rate can vary for a lot of reasons going from anatomical differences among patients to their actual nutrition state.

Conversely to the classical reductionist tenet, this level (in the case of organised complexity [92]) is seldom located at the most detailed scale of analysis (e.g. single patients in epidemiological studies, single genes, primary structures of proteins, single pixels of an NMR or PET image of the brain, etc.) that in the great majority of cases are dominated by noise [86].

The search for the optimal representation (see the protein solubility case described in Sect. 4) asks for a conscious (and knowledge oriented) decision about the representation level to adopt (e.g. sequence-graph-labelled graph). This choice can only be a mixture of theory and data-driven choices, and thus asks for a real interaction of data scientists and biologists. For this interaction to be fruitful, both the communities must develop a similar language and share at least the basic principles of both the fields.

We think that, beside some ‘bombastic exaggerations’ on the ‘death of science’ to be substituted by a purely data-driven theoretically blind approach [2], the future will be characterised by an increasingly stronger integration between computational intelligence and pattern recognition techniques, and the different application fields. Indeed, computational intelligence techniques rely on data-driven modelling (see Sect. 2), which particularly suits problems where the process to be modelled – or at the heart of the problem itself – is unknown or hard to determine in closed-form (e.g. by analytical modelling).

As far as biology (and related fields) are concerned, computational intelligence and pattern recognition can be seen as useful methodological tools in order to perform “in-vitro experiments” and formulate hypotheses to be, if needed, further investigated by means of proper laboratory equipment by field-experts.

In this Chapter, we reviewed and discussed the major challenges and related *modus-operandi* when dealing with non-metric input spaces in computational intelligence and pattern recognition. By considering bioinformatics and computational biology as application fields, we explored several case studies in which data are conveniently represented by means of complex structures.

We stress that, amongst the three main macro-techniques for solving pattern recognition in non-metric spaces (Sect. 3.2), granular computing seems to be the most appealing in terms of results interpretability and knowledge discovery. Indeed, the automatically extracted information granules are the ones which maximize the classification performances, therefore the most informative and significant for the problem at hand. The set of information granules which, recall, is a set of motifs (i.e. recurrent substructures) can be analysed by field-experts in order to check whether they have some biological soundness and, possibly, boost further research, not only in granular computing and computational intelligence as such, but also in the proper application field in which such techniques have been employed.

Multi-classifier-Systems: Architectures, Algorithms and Applications



Peter Bellmann, Patrick Thiam and Friedhelm Schwenker

Abstract In this work multi-classifier-systems (MCS) are discussed. Several fixed and trainable aggregation rules are presented. The most famous examples of MCS, namely bagging and boosting, are explained. Diversity between the base classifiers is a crucial point in order to build accurate MCS. Several criteria to measure diversity in MCS are defined and a motivation for diversity measures, based on the base classifiers' outputs is given. A case study on pain intensity estimation, based on physiological data streams, is conducted. Within the framework of the case study, different MCS and fusion approaches are evaluated. The case study is conducted on two different data sets, with four and five pain levels respectively, which were induced to the test persons under strictly controlled conditions. The aim of the case study is to implement an automatic pain intensity application system and analyse its effectiveness.

Keywords Pain intensity estimation · Decision fusion · Bagging · Boosting
Random forests · Diversity

1 Introduction

Classification is a huge part of pattern recognition and constitutes an every day phenomenon. Vehicle owners, for example, have to identify (classify) their parking cars before using them. Usually they don't need the licence number for that. It is enough to take a look at other characteristics (features), like the car's brand, colour

P. Bellmann (✉) · P. Thiam · F. Schwenker
Institute of Neural Information Processing, Ulm University,
James-Franck-Ring, 89081 Ulm, Germany
e-mail: peter.bellmann@uni-ulm.de

P. Thiam
e-mail: patrick.thiam@uni-ulm.de

F. Schwenker
e-mail: friedhelm.schwenker@uni-ulm.de

and/or the shape, or more specific things, like rims or tinted windows. Identifying one's own car is one of the easiest imaginable daily issues. But there are several tasks, which are too time-consuming or too complex to solve without computational intelligence. Examples for these kinds of tasks include fingerprint- or DNA-matching. Intelligent software can also be applied as an additional supporter for human beings in classification tasks, such as in lie detection.

Classification per se is the last step of a processing chain in supervised pattern recognition. The first step is the data acquisition, i.e. the reading and recording of data from one or several sensors. Step number two would be some pre-processing, like removing noise or outliers, for example. The third step is usually the feature extraction, which can be done manually based on experts' knowledge or automatically with deep learning architectures. Feature extraction is then usually followed by step four, the post-processing. Post-processing could consist of standardisation, feature reduction, etc. The final step of the processing chain is then the classification.

The first sub-step of classification is to choose an adequate classifier for the given task. There exists a huge variety of different classifiers, for example linear discriminant classifiers [1], k-nearest-neighbour classifiers [2], decision trees [3] and artificial neural networks [4]. Even if one finds the best type of classifiers for the given task, one would not build just one single classifier, since a single classifier might not be able to learn an appropriate decision boundary for the given classification task, or perform unexpectedly bad on unseen data. Thus, it is common to use multi-classifier-systems (MCS) [5–10], i.e. a set of classifiers. Once one has fixed the type(s) of the classifiers to use, the size of the MCS, i.e. the number of classifiers, has to be chosen. After fixing the type(s) and amount of classifiers there are still two crucial design steps to be undertaken. First, one has to choose an aggregation rule for the set of classifiers, which depends on the classifiers' output types. Second, one has to determine the fusion architecture of the MCS. As the data is usually recorded by several sensors, one has to specify at which level and in what way the extracted features from the different data streams have to be merged.

The design of an MCS itself constitutes an essential step. In the ideal case one would like to get a set of classifiers, which are always correct, and take one of it for the whole task. But this is an unrealistic scenario, especially in real world applications. Therefore, the aim is to build an MCS with classifiers which complete each other as good as possible. This means one would like to have classifiers which, first, are as accurate as possible (in regard to their generalisation ability), and second, make different errors. More precisely, one would like to have a set of classifiers, which have a certain level of diversity. Diversity in MCS has been studied over the last years. There is a plethora of approaches to define diversity [11]. The explicit use of diversity measures in MCS can help to improve the performance in classification and related fields [12].

The latter three examples of the here aforementioned classification tasks (fingerprint-, DNA-matching and lie detection) have one common characteristic, which is the fact that the signals used for the classification task are biometric. One challenging field of application, that deals with biometric signals, is affective computing [13–18] (e.g. emotion or pain recognition). Pain recognition is an important

field in e-health, since there are people who are not able to express their health condition in the correct way, or not at all. This applies, for example, to newborn babies, infants, comatose patients, or people with Alzheimer's or other mental diseases. The implementation of an automatic pain recognition system, based on multiple input channels, would help to improve the condition of people, affected by the aforementioned problem of not being able to express their state of health.

The remainder of this work is structured as follows. In Sect. 2, we will list and explain the main reasons for the use of an MCS instead of a single classifier. Section 3 provides common approaches for the combination of classifiers. In Sect. 4, we explain the main methods for creating MCS. Section 5 summarises the different fusion architectures of MCS. In Sect. 6, we discuss diversity in MCS. Different diversity measures are presented and adequate optimisation methods are introduced. Section 7 presents our case study, which deals with pain intensity estimation. Finally, in Sect. 8 we conclude the paper.

2 Motivation for Multi-classifier-Systems

Using an MCS instead of a single classifier increases the complexity of a classification task. Not only one, but a set of classifiers has to be build, trained and stored. And an additional aggregation rule has to be applied. This all is linked to a higher memory usage and a longer running time during the training and operational phase in comparison to a single classifier. Despite the arising difficulty, it is still common to use MCS in classification tasks, since the resulting advantages of an MCS outweigh the aforementioned drawbacks. Dietterich [19] and Polikar [20] suggest different reasons for the use of an MCS instead of a single classifier. In the following, we point out two of the most important ones.

2.1 Computational Motivation

The main goal of each classification task is to learn an underlying decision boundary of the given data which separate points from different classes. The general idea of the divide-and-conquer approach is to divide one complex problem into several easier tasks.

Regarding classification tasks, two problems can arise. First, the underlying decision boundaries might be quite difficult to learn by the given training algorithm. One might get classifiers whose decision boundaries represent local optima. Second, the underlying decision boundaries of the given data might lie outside the given classifier space.

For example, a single linear classifier can't solve a linearly non-separable problem, but by using a set of linear classifiers with an appropriate aggregation rule, we might get a possible solution. Figure 1 shows a basic example for this issue.

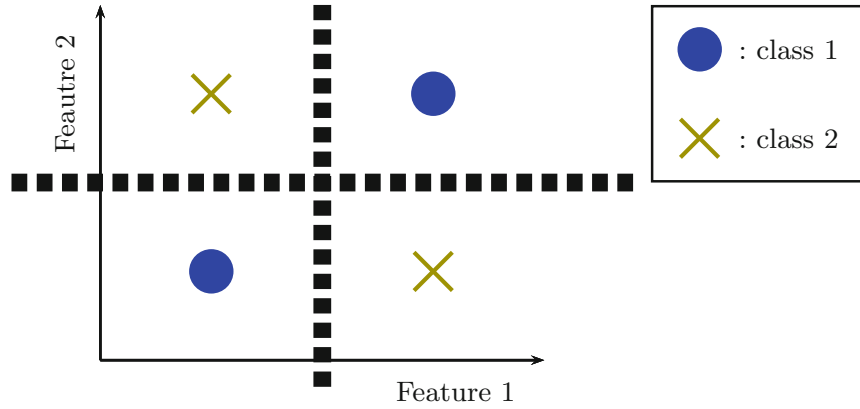


Fig. 1 Linearly non-separable two-dimensional two-class problem that can not be solved by a linear classifier, but it can be solved by multiplying the outputs of two linear classifiers (*parallel to the axis*)

2.2 Statistical Motivation

Given a data set one can divide the set in a training and a validation set. We can build several classifiers and compute their training and validation errors. But the generalization ability can only be estimated, for example by the hold-out or cross-validation methods. It is not guaranteed that classifiers with similar generalization approximations will perform similarly in the test case. By using an MCS of classifiers, we avoid the risk of taking one single classifier which might perform poorly on unseen data.

2.3 Basic Results for Single Classifier Versus MCS

In the following we see the results for a single classifier against an MCS of seven classifiers for the most common combination approaches, namely bagging, boosting and random forests, which will be explained in Sect. 4.1. The data sets used in Table 1 constitute pain intensity estimation tasks, whereby the Pain1 data set represents a 5-class problem, and both Pain2L and Pain2R data sets represent a 4-class problem each. More information will be provided in Sect. 7.1.

Table 1 shows that using an MCS of already a small amount of classifiers can improve the results significantly. However, it is important to mention that the use of an MCS does not guarantee an improvement over one single classifier or its best member. But empirical studies on this topic prove the effectiveness of MCS.

Table 1 Comparison of one single decision tree classifier against an MCS of 7 decision tree classifiers. Every MCS is significantly better than the single classifier and its best member in regard to the Wilcoxon signed rank test with a significance level of $p < 0.05$. Exceptions are: the bagging MCS is not significantly better than the single classifier on Pain2-Left, but still significantly better than its best member; the boosting MCS is obviously worse than its best member on Pain2-Left. The table shows the mean classification accuracies and standard deviations in % in a leave-one-subject-out setting for the recorded biopotentials. The random performance accuracy for Pain1 is 20%, whereas the random performance accuracy for Pain2-Left and Pain2-Right is 25% each, since Pain1 represents a 5-class classification task while Pain2 is a 4-class classification task

Dataset	Single	Bagging		Boosting		Random Forests	
		MCS	Best	MCS	Best	MCS	Best
Pain1	31.17 $\pm$ 8.18	34.32 $\pm$ 8.98	29.79 $\pm$ 7.48	33.30 $\pm$ 5.96	33.08 $\pm$ 5.80	33.70 $\pm$ 8.46	29.44 $\pm$ 7.70
Pain2L	35.35 $\pm$ 5.04	37.42 $\pm$ 7.74	34.28 $\pm$ 5.88	39.35 $\pm$ 6.11	39.39 $\pm$ 5.90	38.65 $\pm$ 7.09	34.87 $\pm$ 6.33
Pain2R	34.83 $\pm$ 6.78	37.95 $\pm$ 8.29	34.65 $\pm$ 6.29	40.39 $\pm$ 6.83	40.19 $\pm$ 6.87	37.78 $\pm$ 7.90	34.35 $\pm$ 5.78

3 Aggregating Classifiers

Once one has built a set of classifiers, it is indispensable to choose an aggregation rule to form the final MCS decision from the individual classifiers' outputs. There exists a variety of different combination rules for classifiers, which can be grouped in trainable and fixed groups or in regard to the used types of the base classifier' outputs. We stick to the first separation. Xu et al. [21] suggest to differentiate between the following types of classifier outputs:

- **Label Outputs:** Given an input x to the classifier, the classifier computes a class label as the output.
- **Continuous Outputs:** Here it is assumed that the classifiers are able to produce a vector of class memberships for each class, for instance with values between 0 and 1. One could also call this kind of outputs *fuzzy outputs*.

Xu et al. [21] also list an additional type of outputs, the so-called *rank outputs*, where the classifier produces an ordered (sub)set of the label set Ω . Hereby, the first element corresponds to the class with the highest probability to be the true class. This kind of outputs can directly be gained from the continuous outputs by ordering the class with the highest support in the first place, the class with the second highest support in the second place, and so on.

Based on the notations defined in [22], we denote the size of an MCS by L and the base classifiers by D_i . Thus, for a given input $x \in X \subset \mathbb{R}^d$, $d \in \mathbb{N}$, the label output of the base classifier i is denoted by $D_i(x)$. Whereas the continuous output of the base classifier i is denoted by $d_i(x) = (d_{i,1}(x), \dots, d_{i,c}(x))$, where c is the number of class labels from the label set $\Omega = \{\omega_1, \dots, \omega_c\}$. The MCS support for the class

ω_j is denoted by μ_j . In each case the MCS chooses the label ω_k as the final decision if

$$k = \underset{j=1}{\operatorname{argmax}}^c \mu_j(x) .$$

3.1 Fixed Combination Rules

Fixed combination rules don't need extra training. Once a set of classifiers has been trained, the classifiers' outputs can be directly combined.

(Weighted) Majority Vote

Probably one of the most intuitive combination methods is the majority vote. Let $x \in \mathbb{R}^d$ be a data point and $v(x) \in \Omega^L$ be the vector with the label outputs from each classifier, i.e. $v(x) := \{D_1(x), \dots, D_L(x)\}$. The majority vote is defined as

$$D(x) = \operatorname{mode} v(x) .$$

In case of a draw, one can choose a class randomly among the winners, thus we may assume that the *mode* operator yields a unique result.

We can also weight the votes of each classifier. Let $b \in \mathbb{R}^L$ be the weight vector for the MCS. The weighted majority vote is then given as

$$D(x) = \underset{j=1}{\operatorname{argmax}}^c \sum_{\substack{i=1 \\ D_i(x)=\omega_j}}^L b_i .$$

The weights of the classifiers can be determined by computing the training or validation error. Usually one would take the validation error, since the classifiers' training accuracies tend to be too optimistic and could lead to overfitting. However, in cases, when there is only a small set of training data available, one might take the training error into consideration. When each classifier is trained on a different subset of the training set, one might also take the training error, or the error on the points, which were unseen by the individual classifiers during the training phase (this kind of error is called *out-of-bag error* in the bagging approach).

Algebraic Combiners

One easy way to merge classifiers is to take their outputs and simply combine them by using some algebraic function,

$$\mu_j(x) = \mathcal{F}(d_{1,j}, \dots, d_{L,j}) .$$

In the following, some existing examples for $\mathcal{F}$ are listed.

Product Rule:

$$\mu_j(x) = \prod_{i=1}^L d_{i,j}(x) .$$

Generalized Mean:

$$\mu_j(x, \alpha) = \left(\frac{1}{L} \sum_{i=1}^L d_{i,j}(x)^\alpha \right)^{1/\alpha} ,$$

with the special cases minimum and maximum for $\alpha \rightarrow \mp\infty$, the geometric mean and the harmonic mean for $\alpha \rightarrow 0$ and $\alpha \rightarrow -1$ respectively, and the arithmetic mean for $\alpha = 1$.

The most common rules are the maximum rule and the arithmetic mean rule. One could also take measures derived from order statistics, like the median, a trimmed mean, or any other algebraic function.

The mean rule is also known as the sum rule, since the maximal sum corresponds to the maximal mean value. In the case of label outputs one can regard the continuous output of each classifier as a binary vector $d_i(x) \subset \{0, 1\}^c \forall 1 \leq i \leq L, \forall x \in X$, where just one entry is equal to one. Taking the mean rule for these kind of vectors leads to the majority vote.

Kittler et al. [23] and Tax et al. [24] conclude that the sum rule is robust and the most resilient one to estimation errors compared to the min, max, product, median and the majority vote rule.

Borda Count

Concerning the original Borda count [25], each classifier creates a rank order of the classes. Let c be the number of classes, then the first ranked one gets $c - 1$ votes, the second placed one gets $c - 2$ votes and the i th placed one $c - i$ votes. The last ranked one gets 0 votes. The votes of each classifier are added for each class and the class with the most votes is defined as the winner.

The Borda count method can be varied by changing the amount of votes given to each candidate or by changing the number of candidates that will receive votes from each classifier. Different variations of Borda count are used all over the world, for example in some political elections, for granting sports awards or in other competitions, like the Eurovision Song Contest.

3.2 Trainable Combination Rules

In contrast to the fixed aggregation rules, trainable combination methods require an extra training before the MCS can be used for the classification task.

Behaviour Knowledge Space Method

The behaviour knowledge space (BKS) method [26] creates a look-up table during the training phase. For each training data point the MCS produces a combination of the classifiers' label outputs. For each combination one has to note the true class of the data point. The look-up table consists of the occurred combinations, each with the class label with the highest appearance.

For example, let's assume we use three classifiers for a two-class problem and the combination $(D_1(x), D_2(x), D_3(x)) = (\omega_1, \omega_2, \omega_1)$ occurred for 10 data points during the training phase. If the majority of these 10 data points had the true class label ω_1 , then every point which leads to the same output combination will be labelled as ω_1 , otherwise as ω_2 .

If there is a draw, then a random choice has to be taken. In the case that one combination does not appear during the training phase, one can stick to the simple majority vote.

Decision Templates

The idea of the decision templates method is based on the so-called decision profiles, introduced by Kuncheva [27]. The decision profile is a matrix with all the vector outputs of the whole MCS:

$$DP(x) = \begin{bmatrix} d_{1,1}(x) & \dots & d_{1,j}(x) & \dots & d_{1,c}(x) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ d_{L,1}(x) & \dots & d_{L,j}(x) & \dots & d_{L,c}(x) \end{bmatrix} \in \mathbb{R}^{L \times c} .$$

The idea of the decision templates method is to save the typical decision profile for each class ω_j . Such a profile is called decision template DT_j and is simply the average among all data points from one class,

$$DT_j = \frac{1}{N_j} \sum_{\substack{x \in X \\ l(x) = \omega_j}} DP(x) ,$$

whereby $l(x)$ denotes the true label of x and N_j the number of objects with true class label ω_j . To label an input data point x , one has to compare $DP(x)$ with all DT_j by using a similarity measure S ,

$$\mu_j(x) = S(DP(x), DT_j) .$$

Sample x is then labelled as the class with the highest similarity. Thus, this method represents a 1-NN classifier, where the prototypes are the decision templates and each data point first has to be transferred to its decision profile before finding its most similar (nearest) neighbour. Extensions of the decision templates approach have been introduced in [28].

Pseudo-Inverse

The Pseudo-Inverse [29] aggregation rule consists of a least-squares linear mapping obtained by computing the Pseudo-Inverse of the base classifiers horizontally concatenated outputs

$$D = [D_1 \dots D_L], \text{ with } D_i = \begin{bmatrix} d_i(x_1) \\ \vdots \\ d_i(x_m) \end{bmatrix} \quad \forall i \in \{1, \dots, L\},$$

where $m \in \mathbb{N}$ is the size of the training set and multiplying it with the corresponding class labels Z :

$$M = \lim_{\alpha \rightarrow \infty} D^T (DD^T + \alpha I)^{-1} Z.$$

The mapping is subsequently applied to the base classifiers' outputs of an unseen sample and the assigned label corresponds to the class with the highest estimated score. Relations between the Pseudo-Inverse solution, sum rule and decision templates are discussed in [30].

3.3 Classifier Selection

Regarding classifier selection [31] the combination method has a selecting rule that can be fixed or learned during the training process. By applying the rule one chooses one or more classifiers for making the final decision for a given input. Classifier selection approaches can be grouped in two categories.

Dynamic Selection:

Given an input sample x . Consider the training or validation set. For each classifier determine

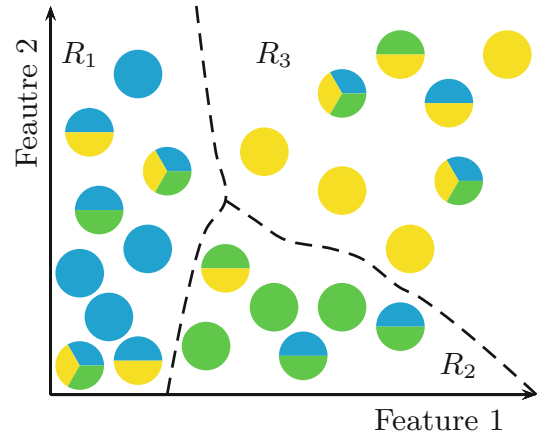
- the k nearest neighbours to x ; or
- the k nearest neighbours to x , which are labelled to the same class as x by the classifier.

Then compute the accuracy among the k neighbours to choose which classifiers have to make the final decision. As in the usual k -NN classification, one can also take the distances to the neighbours into account.

Predefined Selection Regions:

Given a set of classifiers. One can cluster the training or validation set and compute the accuracy of each classifier for every cluster or train the classifiers separately on the clusters. Given an input sample x , one has first to find the cluster of x . Then, one can choose the best classifier(s) assigned to that cluster to label x .

Fig. 2 A two-dimensional classification task where three classifiers (represented by specific colours) are used. The colours of each data point show which classifier labels the sample correctly. The dashed curves separate the feature space into three regions



Regarding Fig. 2, a possible selector could choose the classifier which is represented by the blue colour to classify samples from region R_1 , the one represented by the green colour for samples from region R_2 and data points which lie in region R_3 by the classifier represented by yellow. One famous classifier selection method is the mixture of experts [32] which was originally constructed for neural networks.

4 Designing a Multi-classifier-System

Building an MCS is not a straight forward procedure. By training each classifier on the whole data set, one might get a set of classifiers which are quite similar, or in the worst case, all identical. This contradicts the main idea of the MCS approach. One needs to apply intelligent techniques to build an appropriate MCS. Two of the most popular methods for creating MCS are boosting and bagging with its variants. The main ideas of these methods will be explained in the following subsections.

4.1 Bagging, Random Subspace Method and Random Forests

Bagging: Breiman introduced the bagging method [33] as an acronym for **bootstrap aggregating**. The idea of bagging is the building of the MCS by using different training sets. For each classifier one draws a certain amount of training data with replacement.

Random Subspace Method: The random subspace method [34] is equivalent to bagging. But instead of drawing a random subspace of the training set one draws a random subspace of the feature space for each classifier.

Bagging/Random Subspace Method/Random Forests (Training)**Input:** Training set $X \subset \mathbb{R}^d$, number of classifiers L .

1. Initialization: $\mathcal{D} = \emptyset$.
2. FOR $k = 1, \dots, L$
 - Determine training data S_k for classifier D_k :
 - Draw a subset $S_k \subset X$ (**Bagging**); OR
 - Set $S_k = X$ and draw a subset of the feature space s.t. $S_k \subset \mathbb{R}^p$ with $p \leq d$ (**Random Subspace Method**); OR
 - Draw a subset $S_k \subset X$ and take a subset of the feature space s.t. $S_k \subset \mathbb{R}^p$ with $p \leq d$ (**Random Forests**).
 - Train classifier D_k on S_k and add it to the MCS, i.e. $\mathcal{D} = \mathcal{D} \cup \{D_k\}$.

Output: MCS $\mathcal{D}$.**Fig. 3** Training of bagging, the random subspace method and random forests

Random Forests: Random forests [35] is the combination of bagging and the random subspace method. It is constructed for decision trees. Additionally, one can also vary the parameters of each tree, i.e. minimum leaf size, impurity measure, and so on.

The training phase for each of these methods is shown in Fig. 3 with the notations from [22]. It is proposed to use decision trees and take the majority vote for the final decision. But one could also take a different combination method and choose different classifiers to build the MCS. One of the main advantages of these methods is that each classifier can be created independently, thus the building of the MCS can be parallelised.

4.2 Boosting

The idea of boosting is to add the classifiers sequentially. Like in bagging, one uses a subset of the training data for each classifier. At the beginning, the samples of the individual training set are selected according to uniform distribution. At the end of each step, after a classifier is build, the distribution is updated. The probability to draw data points, that have been misclassified so far, increases. Boosting was created for two-class problems. In the original version one trained three classifiers, and built the final decision via majority voting.

AdaBoost [36], acronym for **adaptive boosting**, is a generalisation of the boosting method. It was also created for two-class problems. The generalisation was made for the number of classifiers. One of the most famous versions for multi-class problems is AdaBoost.M1. Its training phase is explained in Fig. 4, also with the notations from [22].

AdaBoost.M1 (Training)**Input:** Training set $X \subset \mathbb{R}^d$, number of classifiers L .

1. Initialization: $\mathcal{D} = \emptyset$, $\omega^1 \in [0, 1]^N$, $\sum_{j=1}^N \omega_j^1 = 1$.
2. FOR $k = 1, \dots, L$
 - Draw a subset $S_k \subset X$ in regard to the distribution of ω^k and train D_k .
 - Compute the weighted MCS error

$$\epsilon_k = \sum_{j=1}^N \omega_j^k l_k^j,$$

whereby, $l_k^j = 1$, if D_k misclassifies x^j and 0 otherwise.

- If $\epsilon_k = 0$ or $\epsilon_k \geq 0.5$, discard D_k , set ω_j^k to $\frac{1}{N}$ and continue.
- Else compute

$$\beta_k = \frac{\epsilon_k}{1 - \epsilon_k}, \text{ with } \epsilon_k \in (0, 0.5).$$

- Update the weights

$$\omega_j^{k+1} = \frac{\omega_j^k \beta^{1-l_k^j}}{\sum_{i=1}^N \omega_i^k \beta^{1-l_i^j}}, \quad j = 1, \dots, N.$$

Output: MCS $\mathcal{D}$ and the weights $\beta_1, \dots, \beta_L$.**Fig. 4** Training of the AdaBoost.M1 method

Once one trained the MCS $\mathcal{D} = \{D_1, \dots, D_L\}$ with the corresponding weights $\beta_1, \dots, \beta_L$, one can compute the support for class ω_i by

$$\mu_i(x) = \sum_{D_k(x)=\omega_i} \ln \left(\frac{1}{\beta_k} \right).$$

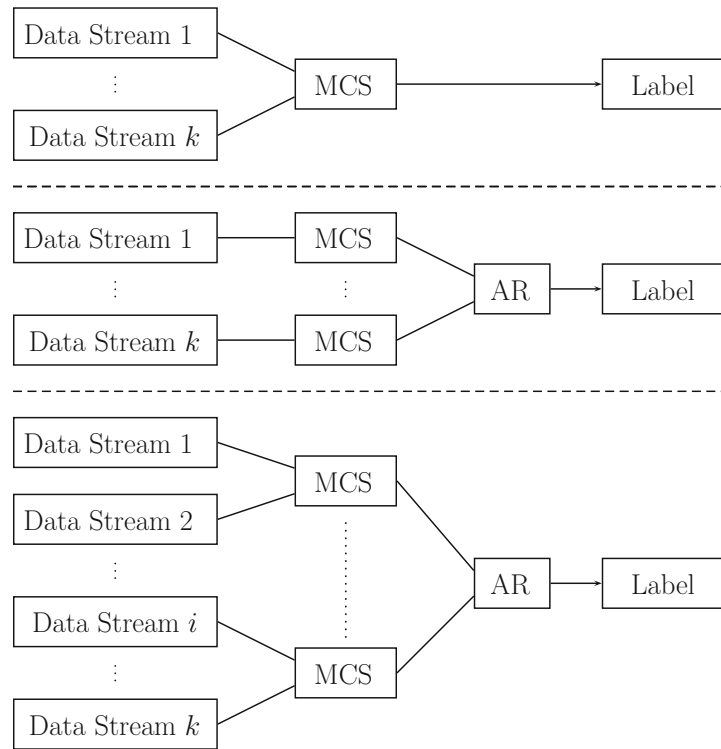
The class with the highest support defines the label for x .

Further variants of boosting are AdaBoost.M2 [36], LogitBoost [37], MultiBoosting [38], AveBoost [39] and AdaBoost-VC [40].

5 Fusion Architectures

Once an MCS is build and an aggregation rule determined, one has to design the MCS architecture. Usually, classification tasks make use of data recorded from different sources, especially in real world applications. After the features are extracted from the different data streams, one has to merge them. There exist different fusion architectures. The two main approaches are the early and late fusion. But one can also combine both approaches to get the hybrid fusion. In the following, the fusion architectures are discussed.

Fig. 5 Fusion Architectures.
Top: Early Fusion. **Middle:** Late Fusion. **Bottom:** Hybrid Fusion. AR stands for aggregation rule



- **Early Fusion:** Early fusion is the most straightforward approach. One takes the features and merges them to one vector for each data point.
- **Late Fusion:** In the late fusion approach the feature space is divided naturally (based on the sources of the data streams, e.g. audio, video, etc.) or artificially into several subspaces. For each subspace, one trains a classifier or an MCS and aggregates their outputs for the final decision.
- **Hybrid Fusion:** The hybrid fusion approach is a combination of the early and late fusion.

Figure 5 shows the differences between the three fusion methods.

Both main fusion architectures have their benefits and disadvantages. Regarding early fusion, one does not need an extra layer for the aggregation of the classifier systems, which is needed in the late fusion approach.

In case that some data from one or more of the data streams are missing, one has to compensate this problem in the early fusion approach by taking the average, median or taking the values from the nearest neighbours of the given data. In the case of the late fusion approach, one can just leave the MCS out, which are responsible for the missing data streams. This can be a comfortable solution for on-line classification tasks. Table 2 summarises the main benefits and disadvantages of both fusion approaches.

As in Sect. 2, some basic results for the data sets described in Sect. 7.1, are given in Table 3 to show the effectiveness of the fusion approaches.

Table 2 Main benefits and disadvantages of the early and late fusion architectures

Approach	+	–
Early	No additional aggregation rule	Complexity due to high dimensionality dealing with missing data
Late	Missing data can be left out	Additional aggregation rule

Table 3 Comparison of one single decision tree classifier against bagged late fusion MCS of 7 and 7×7 decision tree classifiers. Every 7×7 MCS is significantly better than the single classifier in regard to the Wilcoxon signed rank test with a significance level of $p < 0.05$. To mention is, the MCS with 7 classifiers is significantly worse than the single classifier on the Pain1 data set. The table shows the mean accuracies and standard deviations in % for the leave-one-subject-out setting for the biopotentials of the given data sets. The random performance accuracy for Pain1 is 20%, whereas the random performance accuracy for Pain2-Left and Pain2-Right is 25% each

Data Set	Single	Late fusion 7 classifiers	Late fusion 7×7 classifiers
Pain1	31.17 ± 8.18	28.89 ± 7.93	33.61 ± 9.44
Pain2-Left	35.35 ± 5.04	35.54 ± 5.46	38.91 ± 7.44
Pain2-Right	34.83 ± 6.78	34.70 ± 6.35	39.45 ± 7.68

6 Diversity

Methods like bagging and boosting were constructed to create MCS that are somehow diverse. But what does diversity among classifiers mean? One could say that classifiers with different accuracies on the same sets are diverse. However, two classifiers with a similar or even the same accuracy might make different errors. This banal observation is the motivation to apply so-called pruning methods, whereby a *large* MCS is decreased or a *small* MCS is increased until a certain size or an *optimal* MCS, in relation to the classification performance, is found. The removing/adding step in the pruning process is done by using an evaluation measure, which consists of a specific diversity measure or a combination of diversity and accuracy measures. There are many approaches to measure the diversity of an MCS explicitly. Usually the measures are separated into two groups, pairwise and non-pairwise measures. In the following we will list some of the most intuitive measures of each group and add an additional category of diversity measures, which are defined for label outputs of classifiers.

6.1 Pairwise Measures

Let $\mathcal{M}$ be a diversity measure. With $\mathcal{M}_{i,j}$ we denote the diversity value for the pair of classifiers (D_i, D_j) . The diversity value of the whole MCS is then computed for all pairwise measures as

Table 4 Table with the variables used for defining pairwise diversity measures

$a + b + c + d = 1$	D_j correct	D_j wrong
D_i correct	a	b
D_i wrong	c	d

$$\mathcal{M} = \frac{2}{L(L-1)} \sum_{i=1}^{L-1} \sum_{j=i+1}^L \mathcal{M}_{i,j} .$$

Most of the pairwise measures are based on Table 4.

Two simple and quite intuitive pairwise measures are defined as follows:

- **Disagreement Measure.** The disagreement measure is the probability that two classifiers disagree:

$$\mathcal{M}_{i,j} = b + c . \quad (1)$$

- **Double-Fault Measure.** The double-fault measure is the probability that both classifiers make an incorrect decision:

$$\mathcal{M}_{i,j} = d . \quad (2)$$

A higher value in (1) means higher diversity, while a higher value in (2) means lower diversity. Further pairwise measures are, for example, the Q-statistic [41], correlation and product-moment correlation [42].

6.2 Non-pairwise Measures

The non-pairwise measures compute the diversity value by using the whole MCS at once. Let $Y(x)$ be the number of classifiers that classify instance x correctly, i.e.

$$Y(x) := \sum_{i=1}^L \mathbb{1}_{\{D_i(x)=I(x)\}}(x) .$$

The Entropy Measure:

The entropy measure is based on the thought that for an object $x \in X$ the MCS has the highest diversity if $\lfloor L/2 \rfloor$ votes are correct and the other $L - \lfloor L/2 \rfloor$ votes are wrong. Based on this, the entropy measure [43] is defined as

$$E = \frac{1}{N} \frac{1}{L - \lfloor L/2 \rfloor} \sum_{x \in X} \min\{Y(x), L - Y(x)\} .$$

Measurement of Interrater Agreement:

Let $\bar{p}$ denote the mean accuracy of the classifiers, then the interrater agreement is defined as

$$\kappa = 1 - \frac{\frac{1}{L} \sum_{x \in X} Y(x)(L - Y(x))}{N(L - 1)\bar{p}(1 - \bar{p})}.$$

Kuncheva and Whitaker [11] have shown that the interrater agreement is a multiple of the disagreement measure, i.e.

$$\kappa = 1 - \left(\frac{1}{2\bar{p}(1 - \bar{p})} \times \text{disagreement value of the MCS} \right).$$

This is an interesting observation, since the interrater agreement is a non-pairwise measure. Further non-pairwise measures are, for example, the measure of difficulty [44], the Kohavi-Wolpert Variance [45], the generalised diversity and the coincident failure diversity [46]. Kuncheva [22] calls the variety of measures the *diversity of diversity*.

6.3 Measures for Label Outputs

Most of the diversity measures are defined for the artificial correct-false-output of the classifiers, based on Table 4. Kuncheva calls this kind of outputs the *oracle output* [22]. In the following, we introduce an example to justify why it might be better to use the classifiers' label outputs to compute diversity, in regard to multi-class problems.

Motivation

Let's consider a three class problem ($c = 3$) with the class labels $\Omega = \{1, 2, 3\}$ and a set of five elements $X = \{x^1, \dots, x^5\}$. Furthermore, let $D_1, \dots, D_4$ be four classifiers which are combined to two different MCS, $\mathcal{D}_1 = \{D_1, D_2, D_3\}$ and $\mathcal{D}_2 = \{D_1, D_2, D_4\}$. Let the true and the guessed labels be given as follows, in Table 5.

Considering all diversity measures based on the correct-false-output (see Table 4) the MCS $\mathcal{D}_1$ and $\mathcal{D}_2$ will be regarded as equally diverse. But intuitively one would say that MCS $\mathcal{D}_2$ is more diverse than MCS $\mathcal{D}_1$, since classifiers D_2 and D_3 produce exactly the same outputs on the given data set X , whereas the classifiers of $\mathcal{D}_2$ produce all different outputs on four out of five samples.

Disagreement and Double-Fault

According to [47], the disagreement measure can be naturally extended for the label outputs of classifiers. The disagreement measure for two classifiers is then defined as the probability that the classifiers provide different label outputs. In this sense, we naturally extend the double-fault measure for the classifiers' label outputs. The double-fault measure of two classifiers is then the probability that the two classifiers make the *same* mistakes.

Table 5 Motivational example for the use of diversity measures, based on the classifiers' label outputs. The MCS $\mathcal{D}_1$ (rows 1 to 8) contains two classifiers which provide exactly the same outputs on all of the given samples. The MCS $\mathcal{D}_2$ (rows 9 to 16) consists of three classifiers which provide all different outputs on all of the given samples, except for x^2 . However, in regard to the correct-false-outputs, both MCS are considered the same

MCS $\mathcal{D}_1$	x^1	x^2	x^3	x^4	x^5
$l(x^j)$	1	1	2	2	3
$D_1(x^j)$	1	2	2	2	3
$D_2(x^j)$	2	2	1	1	2
$D_3(x^j)$	2	2	1	1	2
$\mathbb{1}_{\{D_1(x^j)=l(x^j)\}}$	1	0	1	1	1
$\mathbb{1}_{\{D_2(x^j)=l(x^j)\}}$	0	0	0	0	0
$\mathbb{1}_{\{D_3(x^j)=l(x^j)\}}$	0	0	0	0	0
MCS $\mathcal{D}_2$	x^1	x^2	x^3	x^4	x^5
$l(x^j)$	1	1	2	2	3
$D_1(x^j)$	1	2	2	2	3
$D_2(x^j)$	2	2	1	1	2
$D_4(x^j)$	3	2	3	3	1
$\mathbb{1}_{\{D_1(x^j)=l(x^j)\}}$	1	0	1	1	1
$\mathbb{1}_{\{D_2(x^j)=l(x^j)\}}$	0	0	0	0	0
$\mathbb{1}_{\{D_4(x^j)=l(x^j)\}}$	0	0	0	0	0

- **Disagreement measure for classifiers' label outputs:**

$$\mathcal{M}_{i,j}^{\text{lab}} = \frac{1}{|X|} \sum_{x \in X} \mathbb{1}_{\{D_i(x) \neq D_j(x)\}}(x) .$$

- **Double-Fault measure for classifiers' label outputs:**

$$\mathcal{M}_{i,j}^{\text{lab}} = \frac{|\{x \in X : D_i(x) = D_j(x), D_i(x) \neq l(x)\}|}{|X|} .$$

Further measures for label outputs are, for example, the Rényi and Shannon entropy [47]. Considering the MCS $\mathcal{D}_1$ and $\mathcal{D}_2$ we can compute the proposed measure values. If we had computed the disagreement and double-fault values using the correct-false-outputs of the classifiers than the values would be the same for both MCS, $\mathcal{D}_1$ and $\mathcal{D}_2$. But using the measures for the label outputs we get the result that MCS $\mathcal{D}_2$ is more diverse than MCS $\mathcal{D}_1$ and does less similar mistakes (see Table 6). This observation is obviously true.

Therefore, the benefits of using diversity measures for label outputs are:

- We get a deeper differentiation by using label outputs instead of the correct-false-outputs for multi-class problems.

Table 6 Diversity values for label outputs for the MCS $\mathcal{D}_1$ and $\mathcal{D}_2$

	$\mathcal{D}_1$	$\mathcal{D}_2$
Disagreement Measure	$\frac{8}{15}$	$\frac{12}{15}$
Double-Fault Measure	$\frac{7}{15}$	$\frac{3}{15}$

- Usually, we don't need a labelled set for computing the diversity, which means that we can even use artificial data to compute the diversity among a set of classifiers.

Exception for the second bullet point is the double-fault measure.

6.4 Pruning

One way to make use of the diversity measures are the so-called MCS pruning methods [48]. There are two main groups of pruning methods.

- **Backward Elimination:** One creates an MCS and gradually removes classifiers by using a given measure.
- **Forward Selection:** One starts with an empty set and gradually adds classifiers by using a given measure.

The measure can be an accuracy measure, any diversity measure or a combination of both. Applying the backward elimination, one can also first build the MCS, then create subsets of the MCS with similar classifiers in regard to a diversity measure, and finally perform the pruning algorithm for each subset separately. Figure 6 shows one step of the backward elimination pruning.

By using a pairwise diversity measure $\mathcal{M}$ we propose to save computational effort by calculating the diversity matrix $DM(\mathcal{M})$,

One Step Backward Elimination Pruning

Input: MCS $\mathcal{D} = \{D_1, \dots, D_L\}$, evaluation measure $\mathcal{E}$.

1. Define
 - $\mathcal{D}_j := \mathcal{D} \setminus \{D_j\}$
 - $\mathcal{E}_j$ evaluation value for $\mathcal{D}_j$
2. FOR $k = 1, \dots, L$
 - Remove D_k from $\mathcal{D}$ just for step k .
 - Compute $\mathcal{E}_k$.

Output: $\mathcal{D}_{j^*}$ s.t. $\mathcal{E}_{j^*}$ is the optimum value among all $\mathcal{E}_j$.

Fig. 6 One step of the backward elimination pruning in the general form

$$DM(\mathcal{M}) = \begin{bmatrix} 0 & \mathcal{M}_{1,2} & \cdots & \cdots & \mathcal{M}_{1,L} \\ 0 & 0 & \mathcal{M}_{2,3} & \cdots & \mathcal{M}_{2,L} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix} \in \mathbb{R}^{L \times L}.$$

To compute the diversity value for the whole MCS one has to sum up each element of $DM(\mathcal{M})$ and divide it by the number of pairs. To compute the diversity value after removing classifier D_k from the MCS $\mathcal{D}$ one can just cancel row k and column k in $DM(\mathcal{M})$ and do the same calculation for the corresponding number of classifier pairs. The main reasons for applying pruning methods are

1. A smaller MCS needs less memory and computational time for classification.
2. By taking a subset of the original MCS one tries to increase the accuracy.

To increase accuracy one should not try to build the MCS as diverse as possible. Figure 7 shows the relationship of accuracy versus both disagreement measures. Obviously the most diverse MCS are not the most accurate ones. To compute the results in Fig. 7 we used the Pain1 data set (see Sect. 7.1). Furthermore, to induce a range, in which the accuracy values vary significantly, we applied the following steps. First, we partitioned the data set into a relatively *huge* training set and a relatively *small* test set. The training set was used to design the MCS, whereby the test set was used to calculate the accuracy, and the diversity values as well. Then, we gradually

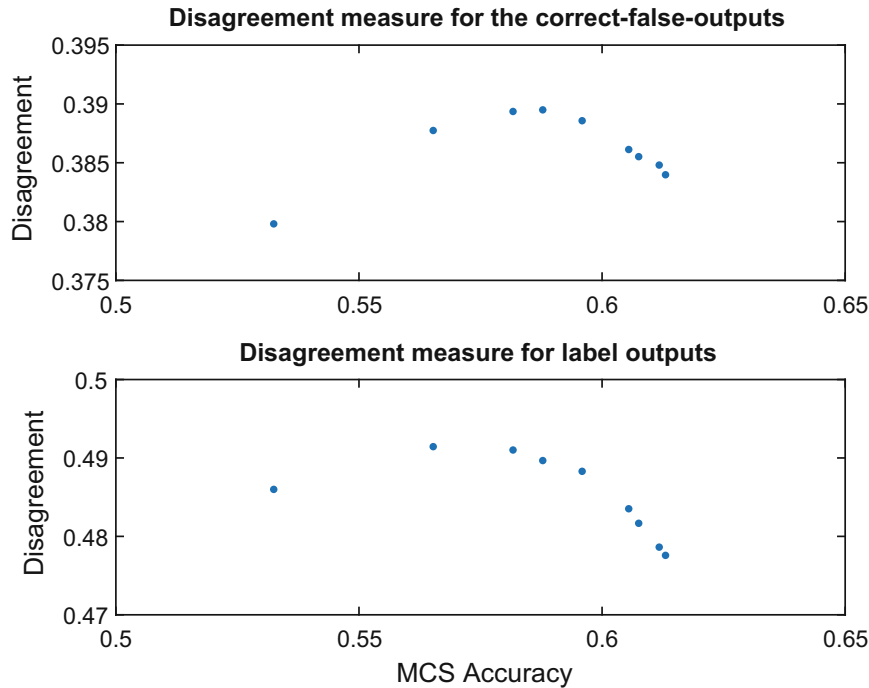


Fig. 7 MCS accuracy versus both disagreement measures on one of the skin conductance level channels of the Pain1 data set with the classes $\Omega = \{T_0, T_2, T_T\}$. The MCS with the highest accuracies are not the most diverse ones. The points form a parabolic shape

changed the ratio of both sets and repeated our experiments, until the training set became relatively *small* and the test set relatively *huge*.

Brown and Kuncheva [49] have shown that by using the disagreement measure - based on the correct-false-outputs - for two-class problems, the MCS majority vote error can be decomposed in three parts, the average individual loss of the classifiers and two diversity terms. One of the diversity terms decreases the MCS error (“good” diversity) and the other diversity term increases the MCS error (“bad” diversity).

7 Case Study: Pain Intensity Estimation

As we mentioned in the introduction, there are several reasons why an automatic pain recognition system would be useful. To take this thought one step further, it might be necessary to be able to estimate different levels of pain, to determine the right therapy for the affected patients, for example. Since pain is a *subjective* experience in the first place, we will ignore data, which is extracted from video and audio signals. Some people may scream when they experience pain, others stay quiet - some people may heavily shake, others remain mainly motionless. Also the recordings of video and audio data can be well-interpreted by human experts. Therefore, the subject of our case study is pain intensity estimation based on biophysiological data. In the first place, we will give a short description of the used data sets, then explain the feature extraction, list some related work on the proposed data sets and finally present the results, followed by a short discussion.

7.1 BioVid Heat Pain Database and SenseEmotion Database

In our case study we use the BioVid Heat Pain Database [50] and the SenseEmotion Database [51] that have been collected at Ulm University. These data sets were specifically recorded for the research field of automatic pain intensity estimation.

The data sets consist of 87 (44 male, 43 female) and 40 (20 male, 20 female) participants, respectively. A thermode was attached to one of the participants’ forearms. In a controlled environment, the participants were subjected to several levels of pain stimuli elicited through the thermode. The pain free baseline temperature was 32°C (T_0) for all participants. The different pain levels were calibrated for each participant individually. Therefore, one raised the temperature slowly to define two thresholds. First, one determined the temperature T_P (pain threshold), when the person felt a change from heat to pain. Then, the temperature T_T (tolerance threshold) was determined, when the person felt that the pain was barely manageable (The maximum allowed temperature was set to 50.5°C). Then the interval $[T_P, T_T]$ was partitioned in three and two sections of equal size for the BioVid Heat Pain Database and the SenseEmotion Database respectively.

Fig. 8 An example for the stimulation and recovering phase for one participant for the BioVid Heat Pain Database. In the SenseEmotion Database there is only one intermediate temperature between T_P and T_T

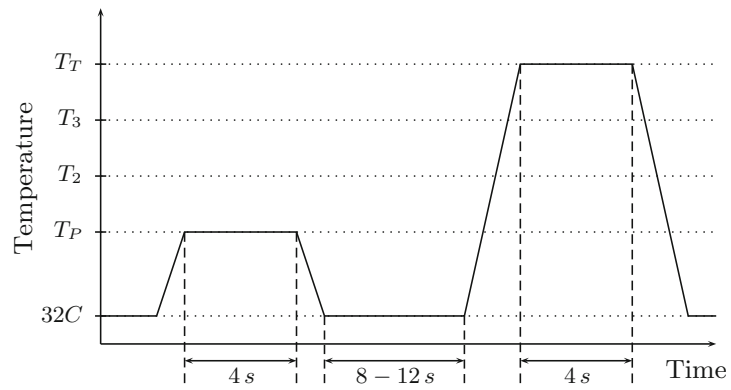


Table 7 Characteristics of both data sets, namely BioVid Heat Pain Database (Pain1) and SenseEmotion Database (Pain2)

Differences between the both data sets	Pain1	Pain2
Number of participants	87 (44 m, 43 f)	40 (20 m, 20 f)
Number of classes	5 ($T_0, \dots, T_4$)	4 ($T_0, \dots, T_3$)
Number of samples per class for each person	20	30
ECG/EMG/SCL/Video	✓	✓
Audio/RSP	—	✓

Each participant was stimulated with all the pain levels 20 and 30 times in a randomized order for the BioVid Heat Pain Database and the SenseEmotion Database respectively. The pain level was held for 4 s each time. Between two pain stimuli there was a break for a random time of 8–12 s (see Fig. 8). During the experimental sessions different video and audio signals were recorded. The physiological data streams consisted of the skin conductance level (SCL), electrocardiography (ECG), electromyography (EMG) and respiration (RSP).

The SenseEmotion Database was also recorded for an emotion recognition task. But since our case study deals with automatic pain intensity recognition we will focus on the experimental part regarding pain stimulation. The reader is referred to [51] to get the full data set description. Furthermore, the experiments on the SenseEmotion Database were carried out twice for each participant. Once for the right forearm and once for the left forearm. We will denote the BioVid Heat Pain Database by Pain1 and the two parts of the SenseEmotion Database by Pain2-Left and Pain2-Right. Table 7 summarizes the main differences between the data sets.

7.2 Related Work on both BioVid Heat Pain Database and SenseEmotion Database

Based on the given data sets, several studies have been conducted. Kächele et al. [52] studied different fusion approaches for the video and biopotential features of

the BioVid Heat Pain Database, for the following person independent (leave-one-subject-out cross-validation setting), binary classification tasks: (T_0, T_1) , (T_0, T_4) , (T_1, T_2) , (T_2, T_3) , (T_3, T_4) and (T_1, T_4) . Furthermore, Kächele et al. [53] also examined different experiments, based uniquely on the biopotentials of the BioVid Heat Pain Database. Thereby, the best performance for the person independent task was reached for (T_0, T_4) , i.e. the task T_0 versus T_4 , with an average accuracy of 85.7%.

Kessler et al. [54] proposed to include remote Photoplethysmography features from the video channels for the pain intensity estimation on the SenseEmotion Database. This approach was used in [55] by Kessler et al. Thereby, for the person independent task T_0 versus T_3 , the fusion of video and biopotential features led to an average accuracy of 71.10% and 71.85%, on the left and right forearm respectively.

Thiam et al. [56] proposed a personalised pain level recognition on the SenseEmotion Database for the task T_0 versus T_3 , based uniquely on video features. In [57] Thiam et al. combined audio and video features of the SenseEmotion Database in both person dependent and person independent scenarios. The following binary classification tasks have been conducted: (T_0, T_1) , (T_0, T_2) and (T_0, T_3) . For the person independent setting, the best performance was reached for the task T_0 versus T_3 , with an average accuracy of 65.89 and 66.76%, on the left and right forearm respectively. By combining all available modalities (audio, video and biopotentials) Thiam et al. [58] reached an average accuracy of 82 and 83%, on the left and right forearm respectively for the task T_0 versus T_3 , in a person independent setting.

7.3 Feature Extraction of the Biopotentials

Prior to the classification experiments, a finite set of descriptors specific to each modality and characterising typical responses of the autonomic nervous system to external stimuli was extracted. The autonomic nervous system regulates visceral activities such as heart rate, respiratory rate, perspiration and muscle activities during periods of stress and emergency [59]. These descriptors are subsequently used as features for each specific modality during the classification experiments. The recorded modalities were first individually pre-processed using a wide range of signal filtering and smoothing techniques (low- and bandpass filtering, Gaussian filtering, signal detrending). This pre-processing step is necessary in order to substantially reduce the amount of noise and artefacts within the signals resulting from the sensibility of the sensors used to perform the recordings, combined with several sources of noise such as unconstrained body motion or progressive detachment of electrodes from the skin during data acquisition.

Subsequently, statistical descriptors from both the temporal domain (*mean signal value, standard deviation, maxima, minima, skewness, kurtosis*) and the frequency domain (*Fourier coefficients, bandwidth, central frequency, mean frequency*) were extracted. Additional descriptors based on entropy (*shannon entropy, fuzzy entropy, sample entropy*) and the signal amplitude (*waveform length, modified mean value of*

the absolute value of the signal) were also extracted. Furthermore, typical descriptors from the ECG signal were extracted based on the differences between consecutive heart beats and the detected PQRS waves (*root mean sum of squared differences, amplitudes of P, Q, R, S and T wavelets*) [53, 60]. Finally, additional statistical descriptors were extracted from both tonic and phasic components of the SCL signal. A total of 194 features were extracted from the BioVid Heat Pain Database, while a total of 335 features were extracted from the SenseEmotion Database. After removing unusable features, i.e. ones that contained NaN (Not-a-Number) values or that were constant for all data points, the bio-physiological feature set specific to the SenseEmotion Database was reduced to 307 features.

7.4 Results

In our case study we did several experiments on person independent pain intensity estimation. For each of the proposed data sets from Sect. 7.1 we did a cross-validation, whereby in each step the data of one person was used as the test set and the remaining data as the training set (leave-one-subject-out setting). We repeated the cross-validation for the methods bagging, boosting and random forests. For each method we computed the accuracies for each single modality and the three different fusion approaches, explained in Sect. 5. We used decision trees as base classifiers. Furthermore, we used the mean as the aggregation rule for both, the late and hybrid fusion approach. In the late fusion approach we trained one MCS for each channel, in the hybrid fusion approach one MCS for each modality. Each MCS consisted of 200 and 300 decision trees for the Pain1 and Pain2 data sets respectively. As shown in Table 7, Pain1 is a 5-class classification task, whereas the Pain2 data sets represent a 4-class classification task each. All of the experiments were conducted with the built-in functions in MATLAB. The parameter setting was left to default. Tables 8, 9 and 10 show the mean accuracies and standard deviations of the evaluated cross-validations on the three proposed data sets respectively.

The boosting results on the Pain2-Right data set (see Table 10) and the basic boosting results from Table 1 for the early fusion lead to one interesting observation regarding the number of classifiers, which are summarized in Table 11. From Table 11 we can see that the increase of the MCS size from 7 to 300 base classifiers could only slightly improve the mean accuracy. In contrast, the median accuracy even decreased, and the standard deviation increased.

Regarding Table 11, it might lead to better results by creating MCS of smaller size, when using the boosting method. Since the boosting method is choosing the training samples selectively in each step, it might lead to overfitting by taking too many samples from one single or a specific group of test persons. Furthermore, the late and hybrid fusion approaches might perform better by using a different aggregation rule, for example a trainable one, like the Pseudo-Inverse proposed in Sect. 3.2. From the Tables 8, 9 and 10 we get the following observations:

Table 8 Comparison between single modalities and fusion approaches on Pain1. Bagging and random forests are both significantly better compared to the boosting method for each fusion approach in regard to the Wilcoxon signed rank test with a significance level of $p < 0.05$. For bagging and random forests the late fusion approach is significantly worse than the other two approaches. The table shows the mean accuracies and standard deviations in % for the leave-one-subject-out setting for the Pain1 data set. The best performing method is represented in bold numbers. The figures in brackets denote the number of channels per data stream. The random performance accuracy for Pain1 is 20%, since it represents a 5-class classification task

Pain1	Single Modalities			Fusion Approaches		
Method	EMG (1)	ECG (3)	SCL (3)	Late	Hybrid	Early
Bagging	23.91 $\pm$ 6.82	26.30 $\pm$ 8.11	38.46 $\pm$ 11.21	37.08 $\pm$ 10.77	38.69 $\pm$ 10.83	39.34 $\pm$ 10.21
Boosting	24.17 $\pm$ 6.12	24.16 $\pm$ 7.51	35.90 $\pm$ 7.43	35.63 $\pm$ 7.28	35.94 $\pm$ 7.78	35.74 $\pm$ 7.19
RF	23.85 $\pm$ 7.15	26.52 $\pm$ 8.57	38.40 $\pm$ 10.90	37.51 $\pm$ 11.06	38.25 $\pm$ 11.11	39.09 $\pm$ 10.85

Table 9 Comparison between single modalities and fusion approaches on Pain2-Left. Only for the boosting method one of the fusion approaches outperforms the SCL modality. But it's still worse than the mean accuracy of the bagged SCL single modality. The table shows the mean accuracies and standard deviations in % for the leave-one-subject-out setting for the Pain2-Left data set. The best performing method is represented in bold numbers. The figures in brackets denote the number of channels per data stream. The random performance accuracy for Pain2-Left is 25%, since it represents a 4-class classification task

Pain2-Left	Single Modalities			Fusion Approaches		
Method	RSP (1)	ECG (2)	SCL (3)	Late	Hybrid	Early
Bagging	33.16 $\pm$ 7.09	33.34 $\pm$ 7.11	43.50 $\pm$ 8.65	41.96 $\pm$ 8.85	42.27 $\pm$ 8.68	43.03 $\pm$ 8.51
Boosting	33.03 $\pm$ 7.52	33.25 $\pm$ 6.24	42.17 $\pm$ 7.93	41.88 $\pm$ 8.16	42.75 $\pm$ 7.85	42.31 $\pm$ 8.08
RF	33.81 $\pm$ 7.08	32.56 $\pm$ 8.27	42.72 $\pm$ 8.64	42.08 $\pm$ 8.95	41.31 $\pm$ 7.57	42.09 $\pm$ 8.79

- The SCL is the best single modality. The accuracy results are significantly better, compared to the other single modalities.
- The SCL modality leads to the best results twice, both times on the Pain2-Left data set.
- The hybrid fusion approach led to the best results on all of the three data sets when using the boosting method.
- The bagging method led to the best overall results on all of the three data sets.

Figure 9 shows some of the presented results in detail.

Tables 8, 9 and 10 show the mean accuracies, which are all below 50% and exhibit relatively high standard deviations. Figure 10 presents the individual accuracies for some of the proposed methods. From Fig. 10 we can observe:

Table 10 Comparison between single modalities and fusion approaches on Pain2-Right. For each method all fusion approaches outperform the best single modality (SCL). The boosting hybrid approach is significantly better than the boosting SCL single modality in regard to the Wilcoxon signed rank test with a significance level of $p < 0.05$, but it's not significantly better than the bagged SCL single modality. The table shows the mean accuracies and standard deviations in % for the leave-one-subject-out setting for the Pain2-Right data set. The best performing method is represented in bold numbers. The figures in brackets denote the number of channels per data stream. The random performance accuracy for Pain2-Right is 25%, since it represents a 4-class classification task

Pain2-Right	Single Modalities			Fusion Approaches		
Method	RSP (1)	ECG (2)	SCL (3)	Late	Hybrid	Early
Bagging	34.23 $\pm$ 6.79	33.82 $\pm$ 8.71	42.26 $\pm$ 9.85	43.16 $\pm$ 8.16	42.15 $\pm$ 7.96	42.34 $\pm$ 8.83
Boosting	33.13 $\pm$ 7.20	33.49 $\pm$ 7.65	40.68 $\pm$ 8.14	41.90 $\pm$ 8.08	42.74 $\pm$ 8.88	40.81 $\pm$ 8.14
RF	33.07 $\pm$ 7.73	33.78 $\pm$ 8.23	41.79 $\pm$ 9.82	42.51 $\pm$ 8.10	42.24 $\pm$ 8.51	41.89 $\pm$ 8.65

Table 11 Comparison of the number of classifiers for the boosting method in the early fusion approach evaluated on the Pain2-Right data set. The table shows the accuracies and standard deviations in % for the leave-one-subject-out setting

Number of classifiers	Mean	Median
$L = 7$	40.39 $\pm$ 6.83	41.84
$L = 300$	40.81 $\pm$ 8.14	41.20

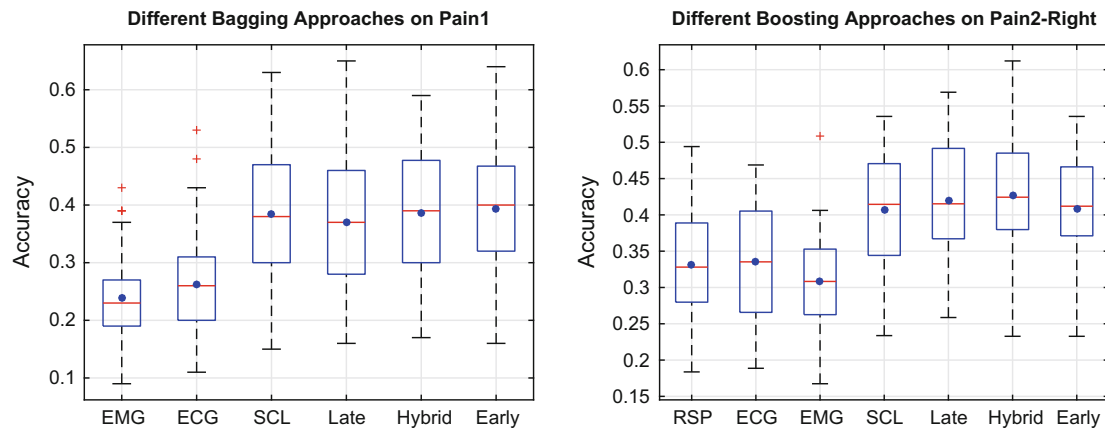


Fig. 9 Boxplots of accuracies of different methods. The dots denote the mean values. **Left:** The early fusion is the best approach regarding the median and mean value. **Right:** The Hybrid fusion approach is significantly better than the best single modality (SCL) in regard to the Wilcoxon signed rank test with a significance level of $p < 0.05$

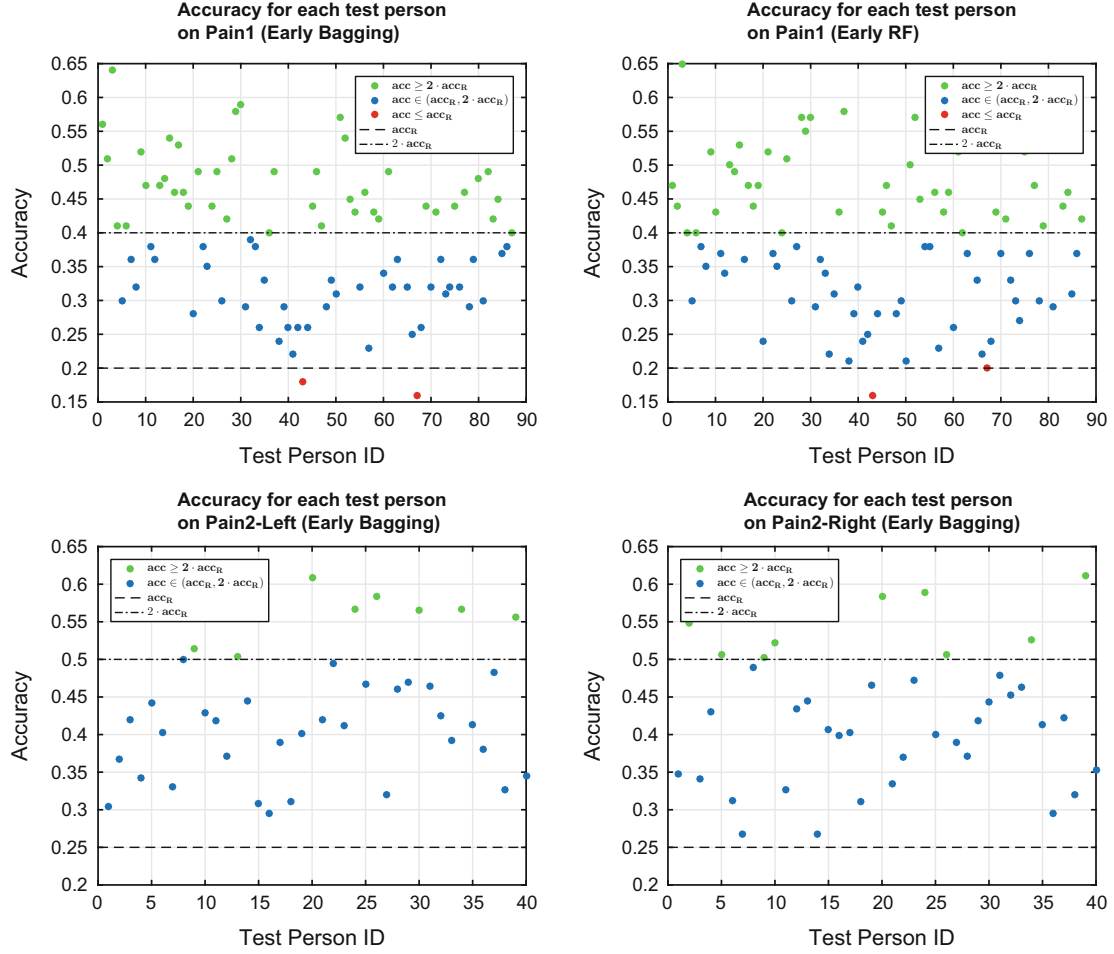


Fig. 10 Individual accuracies for the leave-one-subject-out setting with the early fusion approach. Accuracy is denoted by acc , whereas acc_R indicates the chance level of random guessing. **Top** (Pain1): In the bagging approach there are 44 out of 87 test persons with an accuracy above $2 \cdot acc_R$. In the random forests approach there are 43 out of 87 test persons with an accuracy above $2 \cdot acc_R$. The test persons with the IDs 43 and 67 are in both approaches below the chance level acc_R . **Bottom** (Pain2): There are 8 and 9 out of 40 test persons with accuracies above $2 \cdot acc_R$, on Pain2-Left and Pain2-Right respectively. There is also not even one test person below the chance level acc_R

- The automatic person independent pain intensity estimation (leave-one-subject-out setting) seems to work better for a subset of the test persons, but unfortunately not for all.
- Especially in the Pain1 data set there seem to be at least two test persons, which don't fit to the rest of the set at all.

7.5 Discussion

In our case study we dealt with person independent pain intensity estimation. Thereby, we focused on the physiological data streams of the BioVid Heat Pain Database and

the SenseEmotion Database. The bagging method performed best on all of the three data sets.

The EMG is the worst performing modality. The ECG and RSP modalities lead to similar accuracy results, which lie between the results obtained from the EMG and SCL channels. The SCL is a very strong modality to measure pain. From the experiments we can conclude that the skin conductance is correlated to the temperature stimuli experienced by the test persons. However the information gained from the SCL modality might be only relevant for the pain scenario, in which the pain is induced by heat. Anyway, the results seem not to be reliable enough for a real world application. At this stage, it seems to be reasonable, to simplify the task of the estimation of different pain intensity levels to the distinction between no pain and (strongly experienced) pain, at least for the person independent scenario. An other approach is to train an additional selecting rule, which finds *similar* test persons and restricts the training data in regard to each test person, to improve accuracy, as proposed in [53].

To show the effectiveness of the different fusion approaches one could repeat the experiments with all available modalities, including audio and video features. Furthermore, one could compare the accuracies obtained by using single modalities against all fusion approaches without taking the SCL channels into account.

It also might be useful to repeat the experiments with fixed temperature levels for all test persons. The individual calibration of the stimulation temperatures might have possibly led to a multi-class problem, where each temperature level represents an own class.

8 Conclusion

In this paper we summarised the main aspects of MCS. First, we reflected the main reasons for the use of an MCS and added some basic results to substantiate the effectiveness of the MCS approach. Therefore, we showed that already a *small* amount of classifiers significantly increases the classification performance in regard to the accuracy reached by a single classifier. Then, we described some existing combination methods for classifiers and MCS building approaches to show the variety of MCS designs. We continued by summarising the main fusion architectures, namely early, late and hybrid fusion and pointed out their essential differences, advantages and drawbacks. Every combination of all the three fusion architectures was each combined with the bagging, boosting and random forests approach in our experiments. The bagging method led to the best overall classification results on our proposed data sets in regard to accuracy.

In the second part of the paper we explained the meaning of diversity in the context of MCS. We proposed a short list of existing diversity measures and introduced an example why it might be useful to stick to the classifier's label outputs when dealing with diversity. In this sense we naturally extended the existing double-fault measure to the classifier's label outputs. Then, we discussed the main application of diversity

measures, the MCS pruning. We added another basic result, which underpins that diversity can also be detrimental to the performance of an MCS.

The last part of this paper includes a case study on pain intensity estimation, which is important in the field of e-health. First, we described the BioVid Heat Pain Database and the SenseEmotion Database. We continued with an explanation of the implemented feature extraction of the biopotentials of the proposed data sets. Our experiments show that the implementation of different fusion architectures is useful and is able to lead to an overall accuracy improvement most of the times. Unfortunately, it is not possible to undertake one of the presented pruning methods to achieve higher classification accuracies. The *complexity* of the proposed data sets is obviously quite high, especially for the leave-one-subject-out setting. To undertake effective pruning steps one needs a suitable pruning set (validation set), which is *similar* to the test set. This constitutes the main problem, since the test persons seem to differ significantly, concerning the recorded data. Therefore, to improve the automatic pain intensity estimation for a person independent setting it is necessary to propose new concepts. One possible approach, which was mentioned in Sect. 7.5, is to train the MCS on an appropriate subset of the training data for each test set. Therefore, one has to find test persons which are similar to the test person represented by the test data. The similarity can be calculated in regard to some meta information (test persons' gender, age, etc.) or machine learning based techniques (Euclidean distance, Hausdorff distance, etc.).

Acknowledgements Peter Bellmann is supported by a scholarship of the Landesgraduiertenförderung Baden-Württemberg at Ulm University. Patrick Thiam is supported by the Federal Ministry of Education and Research (BMBF) within the project *SenseEmotion*. The work of Friedhelm Schwenker is partially supported by the Transregional Collaborative Research Centre SFB/TRR 62 *Companion-Technology for Cognitive Technical Systems*, funded by the German Research Foundation (DFG). We gratefully acknowledge the support of NVIDIA Corporation with the donation of the Tesla K40 GPU used for this research.

References

1. R.A. Fisher, The use of multiple measurements in taxonomic problems. *Ann. Eugenics* **7**(2), 179–188 (1936)
2. B.V. Dasarathy, Nearest neighbor (nn) norms: Nn pattern classification techniques. In: IEEE Computer Society Press, Los Alamitos, California (1990)
3. L. Breiman, J.H. Friedman, R.A. Olshen, C.J. Stone, *Classification and Regression Trees*. (Wadsworth, 1984)
4. J.A. Anderson, E. Rosenfeld (eds.), *Neurocomputing: Foundations of Research* (MIT Press, Cambridge, 1988)
5. Kittler, J., Roli, F. (eds.): *Multiple Classifier Systems*, First International Workshop, MCS 2000, Cagliari, Italy, June 21–23, 2000, Proceedings, Lecture Notes in Computer Science, vol. 1857 (Springer, 2000)
6. Schwenker, F., Roli, F., Kittler, J. (eds.): *Multiple Classifier Systems - 12th International Workshop, MCS 2015, Günzburg, Germany, June 29 - July 1, 2015*, Proceedings, Lecture Notes in Computer Science, vol. 9132. Springer (2015)

Appears in: A. Weigend & N. Gershenfeld (Eds.), *Predicting the future and understanding the past* (pp. 243-264). Redwood City, CA: Addison-Wesley Publishing.

Neural net architectures for temporal sequence processing

Michael C. Mozer
Department of Computer Science &
Institute of Cognitive Science
University of Colorado
Boulder, CO 80309-0430

February 13, 2007

Abstract

I present a general taxonomy of neural net architectures for processing time-varying patterns. This taxonomy subsumes many existing architectures in the literature, and points to several promising architectures that have yet to be examined. Any architecture that processes time-varying patterns requires two conceptually distinct components: a short-term memory that holds on to relevant past events and an associator that uses the short-term memory to classify or predict. My taxonomy is based on a characterization of short-term memory models along the dimensions of form, content, and adaptability. Experiments on predicting future values of a financial time series (US dollar–Swiss franc exchange rates) are presented using several alternative memory models. The results of these experiments serve as a baseline against which more sophisticated architectures can be compared.

Neural networks have proven to be a promising alternative to traditional techniques for non-linear temporal prediction tasks (e.g., Curtiss, Brandemuehl, & Kreider, 1992; Lapedes & Farber, 1987; Weigend, Huberman, & Rumelhart, 1992). However, temporal prediction is a particularly challenging problem because conventional neural net architectures and algorithms are not well suited for patterns that vary over time. The prototypical use of neural nets is in *structural pattern recognition*. In such a task, a collection of features—visual, semantic, or otherwise—is presented to a network and the network must categorize the input feature pattern as belonging to one or more classes. For example, a network might be trained to classify animal species based on a set of attributes describing living creatures such as “has tail”, “lives in water”, or “is carnivorous”; or a network could be trained to recognize visual patterns over a two-dimensional pixel array as a letter in $\{A, B, \dots, Z\}$. In such tasks, the network is presented with all relevant information simultaneously.

In contrast, *temporal pattern recognition* involves processing of patterns that evolve over time. The appropriate response at a particular point in time depends not only on the current input, but potentially all previous inputs. This is illustrated in Figure 1, which shows the basic framework for a temporal prediction problem. I assume that time is quantized into discrete steps, a sensible assumption because many time series of interest are intrinsically discrete, and continuous series can be sampled at a fixed interval. The input at time t is denoted $\mathbf{x}(t)$. For univariate series, this input

Figure 1: Abstract formulation of the temporal prediction task. $\mathbf{x}(t)$ denotes a vectorial representation of the input at time t ; $\mathbf{y}(t)$ denotes a prediction at time t based on the input sequence $\mathbf{x}(1) \dots \mathbf{x}(t)$. The short-term memory retains aspects of the input sequence that are relevant to the prediction task.

is a scalar, but to allow for a multivariate series, $\mathbf{x}$ should be considered as vector valued. The output of the predictor at time t , $\mathbf{y}(t)$ is based on the input sequence $\mathbf{x}(1) \dots \mathbf{x}(t)$, and represents some future value of the input or possibly some function of a future value. $\mathbf{y}$ can also be vector valued. A prediction $\mathbf{y}$ can be made at every time step or only at selected times. Prediction involves two conceptually distinct components. The first is to construct a short-term memory that retains aspects of the input sequence relevant to making predictions. The second makes a prediction based on the short-term memory. In a neural net framework, the predictor will always be a feedforward component of the net, while the short-term memory will often have internal recurrent connections. Zhang and Hutchinson (this volume) and de Vries and Principe (1992) also make the distinction between these two components.

In designing a neural net model, one must consider the following issues.

- **Architecture:** What is the internal structure of the short-term memory and predictor networks? Answering this question involves specifying the number of layers and units, the pattern of connectivity among units, and the activation dynamics of the units.
- **Training:** Given a set of training examples, how should the internal parameters in the model—the connection *weights*—be adjusted? Each training example i consists of an input series, $\{\mathbf{x}_i(1), \mathbf{x}_i(2), \mathbf{x}_i(3), \dots, \mathbf{x}_i(t_i)\}$, and an associated series of predictions, $\{\mathbf{p}_i(1), \mathbf{p}_i(2), \mathbf{p}_i(3), \dots, \mathbf{p}_i(t_i)\}$, where t_i is the number of steps in the example, and $\mathbf{p}_i(\tau)$ is a target prediction at time τ . Training a neural net model involves setting its weights such that its predictions, $\mathbf{y}_i(\tau)$, come as close as possible to the target predictions, $\mathbf{p}_i(\tau)$, usually in a least squares sense.
- **Representation:** How should the time-varying input sequence be represented in the short-term memory? The nature and quantity of information that must go into the memory is domain dependent.

In this chapter, I focus on the representation issue. However, the issues of architecture, network dynamics, training procedure, and representation are intricately related and in fact can be viewed as different perspectives on the same underlying problems. A given choice of representation may demand a certain neural net architecture or a particular type of learning algorithm to compute the representation. Conversely, a given architecture or learning algorithm may restrict the class of representations that can be accommodated.

I address the representation issue by characterizing the space of neural network short-term memory models. In the following sections, I present three dimensions along which the memory

Figure 2: Tapped delay line memory model. Each “ Δ ” operator introduces a one time step delay.

models vary: *memory form*, *content*, and *adaptability*. This framework points to interesting memory models that have not yet been explored in the neural net literature.

Forms of short-term memory

Tapped delay line memory

The simplest form of memory is a buffer containing the n most recent inputs. Such a memory is often called a *tapped delay line* model because the buffer can be formed by a series of delay lines (Figure 2). It is also called a *delay space embedding* and forms the basis of traditional statical autoregressive (AR) models. Tapped delay line memories are common in neural net models (e.g., Elman & McClelland, 1986; Elman & Zipser, 1988; Lapedes & Farber, 1987; Plaut, Nowlan, & Hinton, 1986; Waibel, Hanazawa, Hinton, Shikano, & Lang, 1987; Wan, this volume; Zhang & Hutchinson, this volume). These memories amount to selecting certain elements of a sequence $\mathbf{x}(1) \dots \mathbf{x}(t)$, say a total of Ω elements, and forming a state representation $(\bar{\mathbf{x}}_1(t), \bar{\mathbf{x}}_2(t), \bar{\mathbf{x}}_3(t), \dots, \bar{\mathbf{x}}_\Omega(t))$, where $\bar{\mathbf{x}}_i(t) = \mathbf{x}(t - i + 1)$.

A minor extension of this formulation to permit nonuniform sampling of past values involves specifying varying delays such that $\bar{\mathbf{x}}_i(t) = \mathbf{x}(t - \omega_i)$, where ω_i is the (integer) delay associated with component i .

Let me present another formalism for characterizing the delay line memory which is broad enough to encompass other forms of memories to be discussed. One can treat each $\bar{\mathbf{x}}_i(t)$ as a

Figure 3: The kernel functions for (a) a delay-line memory, $\omega = 10$, (b) an exponential trace memory, $\mu = .8$, (c) a gamma memory, $\omega = 6$ and $\mu = .4$, and (d) a gaussian memory.

convolution of the input sequence with a kernel function, c_i :

$$\bar{\mathbf{x}}_i(t) = \sum_{\tau=1}^t c_i(t - \tau) \mathbf{x}(\tau),$$

where, for delay line memories,

$$c_i(t) = \begin{cases} 1 & \text{if } t = \omega_i \\ 0 & \text{otherwise.} \end{cases}$$

Figure 3a shows the kernel function. By substituting a different kernel function, one obtains the forms of memories discussed in the next three sections.

Exponential trace memory

An exponential trace memory is formed using the kernel function

$$c_i(t) = (1 - \mu_i) \mu_i^t,$$

where μ_i lies in the interval $[-1, 1]$ (Figure 3b). This form of memory has been studied by Jordan (1987), Mozer (1989), and Stornetta, Hogg, and Huberman (1988). Unlike the delay line memory, the exponential trace memory does not sharply drop off at a fixed point in time; rather, the strength of an input decays exponentially. This means that more recent inputs will always have greater strength than more distant inputs. Nonetheless, with no noise, an exponential memory

can preserve all information in a sequence. For instance, consider a sequence of binary digits, i.e., $x_i(t) \in \{0, 1\}$ and a memory with $\mu = .5$. The memory becomes a bit string in which the kernel function assigns the input at time $t - \tau$ to bit position $2^{-(\tau+1)}$ of the string. If the memory has finite resolution or is noisy, the least significant bits—the most distant inputs—will be lost. Even with no information loss, however, it is difficult for a neural network to extract and use all the information contained in the memory.

An important property of exponential trace memories is that the $\bar{\mathbf{x}}_i$ can be computed incrementally:

$$\bar{\mathbf{x}}_i(t) = (1 - \mu_i)\mathbf{x}_i(t) + \mu_i\bar{\mathbf{x}}_i(t - 1)$$

with the boundary condition $\bar{\mathbf{x}}_i(0) = 0$. As with the delay line memory, the exponential trace memory consists of Ω state vectors, $(\bar{\mathbf{x}}_1(t), \bar{\mathbf{x}}_2(t), \bar{\mathbf{x}}_3(t), \dots, \bar{\mathbf{x}}_\Omega(t))$. One can view exponential trace memories as maintaining moving averages (exponentially weighted) of past inputs. The μ_i allow for the representation of averages spanning various intervals of time. The exponential trace memory is a special case of traditional statistical moving average models, the MA(1) model. The more general case of MA(q) cannot readily be represented using a trace of the input sequence alone and is dealt with below.

Gamma memory

Two dimensions, *depth* and *resolution* (de Vries & Principe, 1991, 1992), can be used to characterize the delay line and exponential trace memories. Roughly, “depth” refers to how far back into the past the memory stores information relative to the memory size, quantified perhaps by the ratio of the first moment of the kernel function to the number of memory state variables. A low depth memory only holds recent information; a high depth memory holds information distant in the past. “Resolution” refers to the degree to which information concerning the individual elements of the input sequence is preserved. A high resolution memory can reconstruct the actual elements of the input sequence; a low resolution memory holds coarser information about the sequence. In terms of these dimensions, the delay line memory is low depth but high resolution, whereas the exponential trace memory is high depth but low resolution.

de Vries and Principe (1991, 1992) have proposed a memory model that generalizes across delay lines and exponential traces, allowing a continuum of memory forms ranging from high resolution low depth to low resolution high depth. In continuous time, the kernel for this memory is a gamma density function, hence the name *gamma memory*. Although de Vries and Principe deal primarily with continuous time dynamics, there is a discrete equivalent of the gamma density function, the negative binomial, which can serve as the corresponding kernel for discrete-time gamma memories:

$$c_i(t) = \begin{cases} \binom{t}{\omega_i} (1 - \mu_i)^{\omega_i+1} \mu_i^{t-\omega_i} & \text{if } t \geq \omega_i \\ 0 & \text{if } t < \omega_i \end{cases}$$

As before, ω_i is an integer delay and μ_i lies in the interval $[0, 1]$. Figure 3c shows a representative gamma kernel. With $\omega_i = 0$, the gamma kernel reduces to an exponential trace kernel. In the limit as $\mu_i \rightarrow 0$, it reduces to a delay line kernel.

As with the exponential trace kernel, the convolution of the gamma kernel with the input sequence can be computed incrementally (de Vries & Principe, 1991, 1992). However, to compute $\bar{\mathbf{x}}$

for a given μ and ω , which I will denote as $\bar{\mathbf{x}}_{\mu,\omega}$, it is also necessary to compute $\bar{\mathbf{x}}_{\mu,j}$ for $j = 0 \dots \omega - 1$. The recursive update equation is

$$\bar{\mathbf{x}}_{\mu,j}(t) = (1 - \mu)\bar{\mathbf{x}}_{\mu,j-1}(t - 1) + \mu\bar{\mathbf{x}}_{\mu,j}(t - 1)$$

with boundary conditions

$$\begin{aligned} \bar{\mathbf{x}}_{\mu,-1}(t) &= \mathbf{x}(t + 1) && \text{for all } t \geq 0, \text{ and} \\ \bar{\mathbf{x}}_{\mu,j}(0) &= 0 && \text{for all } j \geq 0. \end{aligned}$$

In constructing a gamma memory, the designer must specify the largest ω , denoted Ω . The total number of state vectors is then $\Omega + 1$: $\bar{\mathbf{x}}_{\mu,0} \dots \bar{\mathbf{x}}_{\mu,\Omega}$. Given Ω , the tradeoff between resolution and depth is achieved by varying μ . A large μ yields a high depth low resolution memory, a small μ yields low depth high resolution memory.

Each set of $\Omega + 1$ state vectors with a given μ forms a *gamma family*. The gamma memory can consist of many such families, with family i characterized by μ_i and Ω_i . Of course, this leads to a large and potentially unwieldy state representation. Based on assumptions about the domain, one can trim down the state representation, selecting only certain vectors within a family for input to the neural net predictor, e.g., the subset $\{\bar{\mathbf{x}}_{\mu_i,\Omega_i}\}$. One can view this as a direct extension of the exponential trace memory, which is of this form with $\Omega_i = 0$ for all i . The nonselected vectors must be maintained as part of the recursive update algorithm, but need not be passed to the neural net predictor.

Other forms of memory

The three memories just described are not the only possibilities. Any kernel function results in a distinct memory form. For instance, a gaussian kernel (Figure 3d) can be used to obtain a symmetric memory around a given point in time. What makes the gamma memory and its special cases particularly useful is that they can be computed by an incremental update procedure, whereas forms such as the gaussian require evaluating the convolution of the kernel with the input sequence at each time step. Such convolutions, while occasionally used (Bodenhausen & Waibel, 1991; Tank & Hopfield, 1987; Unnikrishnan, Hopfield, & Tank, 1991), are not terribly practical because of the computational and storage requirements.

Radford Neal (personal communication, 1992) has suggested a class of kernels that are polynomial functions over a fixed interval of time beginning at a fixed point in the past, i.e.,

$$c_i(t) = \begin{cases} \sum_{j=0}^{\Omega} \mu_{ij} t^j & \text{if } t_i^- \leq t \leq t_i^+ \\ 0 & \text{otherwise.} \end{cases}$$

He has proposed an incremental update procedure for memories based on this kernel. The number of memory state vectors, Ω , is the order of the polynomial; the update time is independent of the interval width $t_i^+ - t_i^-$. This appears to be a promising but as yet unexplored alternative to the gamma memory.

For any kernel function, c , which can be expressed as a weighted sum of gamma kernels,

$$c(t) = \sum_{j=0}^{\Omega} q_j c_{\mu,j}(t) \tag{1}$$

where the q_j are the weighting coefficients, the resulting memory can be expressed in terms of the gamma memory state vectors:

$$\bar{\mathbf{x}}(t) = \sum_{j=0}^{\Omega} q_j \bar{\mathbf{x}}_{\mu,j}.$$

de Vries and Principe (1991) show that the gamma kernels form a basis set, meaning that for an arbitrary kernel function c that decays exponentially to 0 as $\tau \rightarrow \infty$, there exists a value of Ω and a set of coefficients $\{q_j\}$ such that Equation 1 holds. This result is easy to intuit for the case of delay kernels, but one can see from this simple case that the required computation amounts essentially to convolving a complex kernel with the input sequence at each time step, and consequently, is not of great practical utility.

Contents of short-term memory

Having described the *form* of the memory, I now turn to the *content*. Although the memory must hold information pertaining to the input sequence, the memory does not have to be of the raw input sequence, as was assumed previously. The memory encoding process can include an additional step in which the input sequence, $\mathbf{x}(1) \dots \mathbf{x}(t)$, is transformed into a new representation $\mathbf{x}'(1) \dots \mathbf{x}'(t)$, and it is this transformed representation that is encoded in the memory. Thus, the $\bar{\mathbf{x}}_i$ are defined in terms of the $\mathbf{x}'$ not the $\mathbf{x}$.

The case considered in previous sections is an identity transformation,

$$\mathbf{x}'(\tau) = \mathbf{x}(\tau).$$

Memories utilizing this transformation will be called *input* or *I* memories. Models in the neural net literature make use of three other classes of transformation. First, there is a transformation by a nonlinear vector function f ,

$$\mathbf{x}'(\tau) = f(\mathbf{x}(\tau)).$$

This transformation results in what I will call a *transformed input* or *TI* memory. Generally f is the standard neural net activation function, which computes a weighted sum of the input elements and passes it through a sigmoidal nonlinearity:

$$f_{\mathbf{w}}(\mathbf{v}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{v}}}. \quad (2)$$

Second, the nonlinear transformation can be performed over not only the current input but also the current internal memory state:

$$\mathbf{x}'(\tau) = f(\mathbf{x}(\tau), \bar{\mathbf{x}}_1(\tau), \dots, \bar{\mathbf{x}}_{\Omega}(\tau)). \quad (3)$$

This leads to a *transformed input and state* or *TIS* memory. Such memories can be implemented in a recurrent neural net architecture in which the $\bar{\mathbf{x}}_i$ and $\mathbf{x}'$ correspond to activities in two layer of hidden units (Figure 4a). Note that this architecture is quite similar to a fairly common recurrent neural net sequence processing architecture (Figure 4b; e.g., Elman, 1990; Mozer, 1989), which I'll refer to as the *standard* architecture.

Third, for *autopredictive* tasks in which the target output, $\mathbf{p}(\tau)$, is a one-step prediction of the input, i.e.,

$$\mathbf{p}(\tau) = \mathbf{x}(\tau + 1),$$

Figure 4: (a) A TIS memory in a neural net architecture. Each rectangle represents a layer of processing units, and each arrow represents complete connectivity from one layer to another. The input layer corresponds to the current element of the time series, $\mathbf{x}(t)$, the first hidden layer to the TIS representation, $\mathbf{x}'(t)$, and the second hidden layer to the memory state, $\{\bar{\mathbf{x}}_i(t)\}$. (b) A standard recurrent neural net architecture, which is like the TIS architecture except that the first two hidden layers are collapsed together, i.e., $\mathbf{x}'(t) = [\bar{\mathbf{x}}_1(t) \dots \bar{\mathbf{x}}_\Omega(t)]$.

one can consider an alternative content to the memory. Rather than holding the actual sequence value, the preceding prediction can be used instead:

$$\mathbf{x}'(\tau) = \mathbf{p}(\tau - 1).$$

This transformation will be called an *output* or *O* memory. Of course, *TO* and *TOS* memories can be constructed by analogy to the TI and TIS memories, suggesting a characterization of memory content along two subdimensions, one being the transformation applied and the other being whether the transformation is applied to the input or previous output. Since it does not make a great deal of sense to ignore input sequence information when it is available, I will include in the category of output memories those that combine input and output information, e.g., by taking a difference between input and output, $\mathbf{x}(\tau + 1) - \mathbf{p}(\tau)$, or by using the input information when it is available, such as during training, and the output information otherwise.

Memory adaptability

The final dimension that characterizes short-term memories is the *adaptability* of the memory. A memory has various parameters— $\{\mu_i\}$, $\{\Omega_i\}$, the $\mathbf{w}$ of Equations 2 and 3—that must be specified.

If the parameters are fixed in advance, one can call the memory *static* because the memory state, $\{\bar{\mathbf{x}}_i(t)\}$, is a predetermined function of the input sequence, $\mathbf{x}(1) \dots \mathbf{x}(t)$. The task of the neural net is make the best predictions possible given the fixed representation of the input history. In contrast, if the neural net can adjust memory parameters, the memory representation is *adaptive*. Essentially, by adjusting memory parameters, the neural net selects, within limits on the capacity of the memory, what aspects of the input sequence are available for making predictions. In addition to learning to make predictions, the neural net must also learn the characteristics of the memory—as specified by its parameters—that best facilitate the prediction task.

The memory model in Figure 2 is an example of a static memory. The model sketched in Figure 4 is intrinsically an adaptive memory because without training the hidden unit responses, the network seems unlikely to preserve sufficient relevant information.

Interesting cases of adaptive memory models in the neural net literature include: the learning of delays (Bodenhausen & Waibel, 1991; Unnikrishnan, Hopfield, & Tank, 1991), the learning of exponential decay rates (Bachrach, 1988; Frasconi, Gori, & Soda, 1992; Mozer, 1989), and the learning of gamma memory parameters (de Vries & Principe, 1992). Every sequence processing recurrent neural net architecture trained with *back propagation through time* (or *BPTT*; Rumelhart, Hinton, & Williams, 1986) or *real-time recurrent learning* (or *RTRL*; Robinson & Fallside, 1987; Schmidhuber, 1992; Williams & Zipser, 1989) is also an adaptive memory model in that the training adjusts $\mathbf{w}$ of Equation 2. The Elman SRN algorithm (Elman, 1990) lies somewhere between an adaptive and static memory because the training procedure—which amounts to back propagation one step in time—is not as powerful as full blown back propagation through time or RTRL.

Static memory models can be a reasonable approach if there is adequate domain knowledge to constrain the type of information that should be preserved in the memory. For example, Tank and Hopfield (1987) argue for a memory that has high resolution for recent events and decreasing resolution for more distant events. The argument is based on a statistical model of temporal distortions in the input. If there is local temporal uncertainty in the occurrence of an input event, then the uncertainty relative to the present time increases with the expected temporal lag of the event. The decreasing-resolution-with-increasing-depth memory suggested by this argument can be achieved by a gamma family, $\{\bar{\mathbf{x}}_{\mu,\omega} : 0 \leq \omega \leq \Omega\}$. As ω increases, the state vector $\bar{\mathbf{x}}_{\mu,\omega}$ represents an increasing depth, decreasing resolution trace of the input.

Memory taxonomy

I have described three dimensions along which neural net memory models vary: memory form (delay line, exponential trace, gamma trace), content (I, TI, TIS, O, TO, TOS), and adaptability (static, adaptive). The Cartesian product of these dimensions yields thirty-six distinct classes.

Table 1 presents some combinations of memory form and content, along with existing models in the literature of each class.¹ The I-delay memory is the simplest class and corresponds to a feedforward network with a delay space embedding of the input sequence. TI-delay memories are the basis of the TDNN (Waibel et al., 1987) and the FIR neural net (Wan, this volume). In these models, each hidden unit—which is a nonlinear transformation of the input—maintains a history of its n most recent values, and all these values are available to the next layer. TI-delay models have also been studied extensively within the more physically oriented neural net community (Kleinfeld, 1986; Sompolinsky & Kanter, 1986; for an overview, see Kühn & van Hemmen, 1991). Herz (1991)

¹The TO and TOS varieties have been omitted because they have received little study.

Table 1: Taxonomy of neural net architectures for temporal processing

memory contents	memory form		
	<i>delay</i>	<i>exponential</i>	<i>gamma</i>
I	Elman & Zipser, 1988; Lapedes & Farber, 1987; Zhang & Hutchinson, this volume	?	de Vries & Principe, 1991, 1992
TI	Waibel et al., 1987; Wan, this volume; Kleinfeld, 1986; Sompolinsky & Kanter, 1986	Bachrach, 1988; Frasconi, Gori, & Soda, 1992; Mozer, 1989	?
TIS	Herz, 1991	Mozer, 1992	?
O	Connor, Atlas, & Martin, 1992	Jordan, 1987	?

proposes a TIS-delay memory in a network whose dynamics are governed by a Lyapunov functional under certain symmetry conditions on the time-delayed weights. Connor, Atlas, and Martin (1992) study a nonlinear neural network ARMA model, whose MA component is constructed from an O-delay memory that retains the difference between the predicted and actual outputs (the latter being identical to the next input). In general, nonlinear MA(q) models can be based on O-delay memories.

Moving to the second column in Table 1, Bachrach (1988), Frasconi, Gori, & Soda (1992), and Mozer (1989) have proposed an TI-exponential memory model and described a computationally efficient algorithm for adapting the μ_i parameters. Mozer's (1992) multiscale integration model is a TIS-exponential memory. The motivation underlying this work was to design recurrent hidden units that had different time constants of integration, the slow integrators forming a coarse but global sequence memory and the fast integrators forming a fine grain but local memory. This memory was adaptive in that the hidden unit response functions were learned, but static in that the μ_i were fixed. Jordan (1987) has incorporated an O-exponential memory in a sequence production network. During training, the network is given target output values, and these are fed back into the output memory; during testing, however, the actual outputs are used instead.

As the third column of the Table suggests, the gamma memory is the least studied form. de Vries and Principe's initial simulations have been confined to an I-gamma memory, although they acknowledge other possibilities.

Beyond the simple memory classes, hybrid schemes can also be considered. The TDNN, for example, is an I-delay memory attached to a series of TI-delay memory modules. I have proposed architectures combining an I-delay memory with a TIS-exponential memory (Mozer & Soukup, 1991) and an I-delay memory with a TI-exponential memory (Mozer, 1989). The possibilities are, of course, diverse, but the certain conclusion is that which class of memory is best depends on the domain and the task.

While this taxonomy represents a reasonable first cut at analyzing the space of memory models, it is not comprehensive and does not address all issues in the design of a memory model. Some models in the literature, although they can be pigeonholed into a certain cell in the taxonomy, have critical properties that are ignored by the taxonomy. For example, Schmidhuber (1992; Schmidhuber, Prelinger, & Mozer, 1993) has proposed a type of I-delay memory in which the memory

size and the ω_i delay parameters are dynamically determined based on the input sequence as it is processed. The idea underlying this approach is to discard redundant input elements in a sequence. (See Myers, 1990, and Ring, 1991, for variants of this idea.)

For the most part, I have also sidestepped the issue of network dynamics, assuming instead the typical back propagation style activation dynamics. There is an interesting literature on temporal associative networks that seek energy minima (e.g., Herz, 1991; Kleinfeld, 1986; Sompolinsky & Kanter, 1986) and networks that operate with continuous time dynamics (Pearlmutter, 1989). However, this work more directly addresses the issue of network architecture, not representation, and as I indicated at the outset, I have focused on representational issues.

The inadequacy of standard recurrent neural net architectures

The standard recurrent neural net architecture for sequence processing tasks in Figure 4b amounts to a trivial case of a TIS-exponential memory with the $\mu_i = 0$ or equivalently, a TIS-delay memory with $\Omega = 0$. I will refer to this limiting case as a *TIS-0* memory. Although I and others have invested many years exploring TIS-0 architectures, the growing consensus seems to be that the architecture is inadequate for difficult temporal processing and prediction tasks.

The TIS-0 architecture seems sufficiently powerful *in principle* to handle arbitrary tasks. One is tempted to argue that, with enough hidden units and training, the architecture should be capable of forming a memory that retains the necessary task-relevant information. This is achieved by adjusting the $\mathbf{w}$ parameters in Equations 2 and 3 using a gradient-descent procedure. *In practice*, however, many have found that gradient descent is not sufficiently powerful to discover the sort of relationships that exist in temporal sequences, especially those that span long temporal intervals and that involve extremely high order statistics. Bengio, Frasconi, and Simard (1993) present theoretical arguments for inherent limitations of learning in recurrent networks. Mozer (1989, 1992) illustrates with examples of relatively simple tasks that cannot be solved by TIS-0 memories. Although TIS-0 memories were inadequate for these tasks, TI-exponential or TIS-exponential memories yielded significantly improved performance. In both cases, the exponential trace components allowed the networks to bridge larger intervals of time. As further evidence, consider the active neural net speech recognition literature. I know of no serious attempts at speech recognition using TIS memories. However, the TDNN architecture—a TI-delay memory—is viewed as quite successful. There have even been interesting explorations using TI-exponential memories (Bengio, De Mori, & Cardin, 1990; Watrous & Shastri, 1987).

With TIS-0 architectures, gradient descent often cannot find meaningful values for the recurrent-connection strengths. Gradients computed by back propagation, using either BPTT or RTRL, tend to be quite small.² For an intuition why this is so, consider the network “unfolded” in time, which is necessary for computing the gradients with BPTT. In the unfolded network, there is a copy of the original network (units and connections) for each time step of the temporal sequence. This turns the recurrent network into a deeply layered feedforward network. Now consider the consequence of changing a weight embedded deep in the unfolded network, say a weight connecting a hidden unit at time 1 to a hidden unit to time 2. Although it will have some consequence at a later time t , the effect is likely to be small, hence so will the gradient. Further, the impact of a connection weight—even if appropriate—will be masked by other weights if their values are inappropriate. This situation is true for feedforward nets as well as recurrent nets, but the problem is exacerbated

²With careful selection of the weights, one can prevent the gradients from shrinking through time (Hochreiter, 1991); however, this imposes serious restrictions on the sorts of memories the network can form.

by the deeply layered structure of the unfolded net. The result is an error surface fraught with local optima.

While this situation may hold for TIS-0 memories, it may not be the case for other recurrent architectures that have a more specialized connectivity, e.g., TI-exponential memories. Further, TIS-0 memories, used *in conjunction with* other types of memory that may help bootstrap learning, are a promising possibility. It also seems well worth investigating classes of recurrent network memory in the taxonomy that have yet to be explored. To the best of my knowledge, these classes include TIS-delay, TI-gamma, TIS-gamma, nonlinear MA models of the O-delay variety, as well as MA models based on O-gamma and O-exponential and the TO and TOS varieties.

Experiments with a financial data series

In the remainder of this chapter, I present experiments examining a financial data series prediction task using three memory types: I-delay, TIS-0, and a hybrid approach combining I-delay and TIS-0. Although these experiments do not explore the more sophisticated memory classes described earlier, they provide a baseline against which future experiments can be compared.

The data series I studied was competition data set C, the tickwise bids for the exchange rate from Swiss francs to US dollars, recorded by a currency trading group from August 1990 to April 1991. “Tickwise” means that the samples come at irregular intervals of time, as demanded by the traders. Each sample is thus indexed by a time of day. There were six prediction tasks in the competition, but I focused on three: predicting the value of the series one, fifteen, and sixty minutes in the future.

The data set appears challenging because the average sampling rate is high relative to the rate of trends in the data. Although the nature of the data is that it is obviously noisy, smoothing is inappropriate because there may be information in the high frequency changes. Consequently, a high-depth memory would seem necessary. This data set seems ideal for a TIS architecture which, in principle, does not have a fixed memory depth.

The training data consisted of ten contiguous spans of time with unspecified gaps between them, in total 114 days of data. I treated each day’s data as an independent sequence, i.e., I assumed no useful information was carried over from one day to the next. For each day, I massaged the data to transform tickwise samples to fixed interval samples, one minute apart, operating on the assumption that the value of the series remained constant from the time of a sample until the time of the next sample. This sequence was subsampled further, at an interval of Δ_s minutes. For the one, fifteen, and sixty minute prediction tasks, I used $\Delta_s = 2, 5$, and 10 , respectively. The average trading day spanned 575 minutes, resulting in daily series of lengths 57.5 to 287.5. Of the 114 days of training data, six complete days—chosen at random—were set aside to be used for validation testing. It was on the basis of this validation data that many parameter values, such as the Δ_s , were selected.

The input to the neural net predictor consisted of three values: the day of the week, the time of day, and the change in the series value from one sample to the next, $s(\Delta_s t) - s(\Delta_s(t - 1))$, where $t = 1 \dots T$ is an index over the input sequence and $s(\tau)$ denotes the sample at minute τ from the first sample of a day. The target output of the neural net is a prediction of the change in the series value Δ_p minutes in the future, i.e., $s(\Delta_s t + \Delta_p) - s(\Delta_s t)$.

Each of the input and output quantities is represented using a *variable encoding* (Ballard, 1986), which means that the input and output activities are monotonic in the represented quantity. This is the obvious encoding; I mention it simply because connectionist approaches to representation

(e.g., Ballard, 1986; Smolensky, 1990) offer a range of alternatives that could be fruitfully explored, but I have not done so in the present work.

The activity of each input unit was normalized to have a mean of 0.0 and variance 1.0 over the training set. This leads to a better conditioned search space for learning (Le Cun, Kanter, & Solla, 1991; Widrow & Stearns, 1985). The target output activity was normalized to have mean 0.0 and variance 100.0 over the training set. Despite this large variance, the observed variance in the unit was much closer to 1.0 because strong predictions could not be made due to the uncertainty in the data.

The general architecture studied is shown in Figure 5. The architecture includes two memories—types TIS-0 and I-delay—which feed, along with the inputs directly, into a layer of hidden units, and then map to an output prediction. A symmetric activation function (activation in range -1 to +1) was used for the hidden units, including the hidden units of the TIS-0 memory. (Recall that the TIS-0 memory is implemented as a recurrent network with internal hidden units; see Figure 4b.) The output unit had a linear activation function.

The actual architectures tested were three special cases of this general architecture. The first case eliminated the TIS-0 memory and used a six step I-delay memory. The number of hidden units was varied. When the number of hidden units is 0, this architecture reduces to a linear autoregressive model. The second case eliminated the I-delay memory in favor of the TIS-0 memory. The TIS-0 memory consisted of 15 internal hidden units, plus 5 hidden units in the penultimate layer. The third case—a hybrid architecture—included both the TIS-0 and I-delay memories, with parameters as above. The choice of number of hidden units and number of delays was based on validation testing, as described below.

The various networks were trained using stochastic gradient descent, whereby the weights were updated following presentation of an entire day’s data. Back propagation through time (Rumelhart, Hinton, & Williams, 1986) was used to train the recurrent connections in the TIS-0 memory. Sufficiently small learning rates were chosen that the training error decreased reliably over time; generally, I set the learning rate to .0002 initially and decreased it gradually to .000005 in the first 500 passes through the training data. The networks were initialized with weights chosen randomly from a normal distribution, mean zero, and were normalized such that the sum of the absolute values of the weights feeding into a unit totaled 2.0.

Following each pass through the training data, the errors in the training and validation sets were computed. Rather than training the network until the training set error converged, as is commonly done, training was stopped when the validation error began to rise, a method suggested by Weigend, Rumelhart, and Huberman (1990) and Geman, Bienenstock, and Doursat (1992). In actuality, this involved training the network until convergence, observing the point at which the validation error began to rise, and then restoring the saved network weights at this critical point. At this point, ten final passes were made through the training data, annealing the learning rate to 0. The purpose of this step was to fine tune the weights, removing the last bit of jitter.

I explored many variations of the architecture and training procedure, including: varying the number of hidden units, varying the number of delays in the I-delay memory, trying different values for the Δ_s , removing the day of week and time of day inputs, and generating a larger training set by transforming each day’s data into Δ_s different streams, each offset from the next by one minute. Generalization performance was estimated for each variation using the validation set.³ The parameters and choices as previously described yielded performance as good as any.

³Because the validation set was also used to determine when to stop training, a bias is introduced in using the validation set for selecting the architecture. In retrospect, two distinct validation sets should have been used.

Figure 5: General architecture used for time series experiments. The figure depicts three inputs feeding into two types of memory which then map, through a layer of hidden units, to a single output—the prediction unit. Arrows represent connections between units or sets of units. The architecture includes direct connections from “day of week” and “time of day” to the output, which are not depicted.

For each architecture studied, twenty-five replications were performed with different random initial weights. The ten models that yielded the best validation performance were selected to make predictions for the competition data, which were averaged together to form the final predictions. The purpose of this procedure was to obtain predictions that were less sensitive to the initial conditions of the networks. Similar procedures have been suggested by Lincoln and Skrzypek (1990) and Rosenblatt (1962).

Competition test data

I did not submit an official entry to the Time Series Competition. However, shortly after the close of the competition deadline, I began experimenting with data set C, with no knowledge of the series beyond what was available to competition entrants. I used the TIS-0 version of the architecture to obtain 15 and 60 minute predictions for the competition data. Performance on this series is measured in units of *normalized mean squared error* or *NMSE*. The normalization involved is to divide the mean squared error by the error that would have been obtained assuming that the future value of the series is the same as the last observed value. This normalization term is the least squares prediction assuming a random walk model for the series. NMSE values less than 1.0 thus indicate that structure has been extracted from the series. After submitting my predictions to Andreas Weigend, he reported back that I'd done a bit better than entrants to the competition: the 15 minute prediction yielded a NMSE of .859 and the 60 minute prediction a NMSE of .964.

At the Time Series Workshop, Xiru Zhang quite validly argued that the competition data set simply did not contain sufficient test points to ascertain reliability of the predictions. We agreed to perform additional experiments using larger test sets.

Extended competition test set

The first set of experiments utilized the competition training set, and hence the models developed for the competition, but test data came from the gaps in the training set. Recall that the training set consisted of ten contiguous time spans with gaps in between. The gaps covered 69 days, more than sufficient for an extended test set. All data in the gaps was used for testing, with the exception of the first hour of each day, in order to provide an initialization period for the recurrent TIS-0 architecture.

The results for 1, 15, and 60 minute predictions are shown in Table 2. The I-delay architecture was tested with different numbers of hidden units, and the TIS-0 model in the configuration described earlier. The I-delay architecture with zero hidden units is equivalent to a linear autoregressive model, AR(6). To a first order, all architectures performed the same, slightly better than a random walk model and also better than Zhang and Hutchinson (this volume). I attempted to determine the reliability of differences in the NMSEs, but did not get very far. A straight t-test on the squared errors for the individual predictions yielded no significant differences. Noting that the data distribution strongly violated the assumption of normality, I applied a log transform to the squared errors, which still did not achieve a normal distribution but came a bit closer. After the log transform, reliable differences between NMSEs were obtained for pairs of conditions whose NMSEs differed by roughly .001 or more. No matter, performance was discouragingly poor in all conditions.

Finnoff, Hergert, and Zimmermann (1992) report performance comparisons across a broad spectrum of tasks for a variety of different training techniques. While their explorations included variations on the techniques I used, they found that a relatively simple technique—stopping training

Table 2: NMSE for extended competition test set

<i>architecture</i>	<i>1 minute prediction (18465 data points)</i>	<i>15 minute prediction (7246 data points)</i>	<i>60 minute prediction (3334 data points)</i>
I-delay, 0 hidden	.998	1.001	.999
I-delay, 1 hidden	.998	1.001	.996
I-delay, 3 hidden	.998	1.000	.998
I-delay, 5 hidden	.998	1.000	.998
I-delay, 10 hidden	.998	.999	.999
TIS-0	.998	.999	.997

Table 3: NMSE for 1985–87 test data

<i>architecture</i>	<i>1 minute prediction (57773 data points)</i>
I-delay, 0 hidden	.999
I-delay, 5 hidden	.985
I-delay, 10 hidden	.985
I-delay, 20 hidden	.985
TIS-0	.986
hybrid TIS-0 and I-delay	.986

based on a validation set and then training further using weight decay—appeared to work very well across tasks. I experimented with this method, but obtained results essentially the same as those reported in Table 2. They also used a much larger validation set. Being concerned that my small validation set was responsible for performance, I ran further experiments in which the validation corpus was increased to 25% of the training set, but this manipulation also had little effect on performance.

The 1985–87 corpus

The extended competition test set was drawn from the same time period as the training data. Xiru Zhang suggested an additional data set in which the testing corpus came from a different time period than the training corpus. He proposed training data from the same exchange rate series spanning the time period 6/85–5/86, and testing data spanning the time period 6/86–5/87. This encompassed 248 days of training data, and a comparable amount of testing data. I selected 37 days of the training data—roughly 15%—for validation testing. Following Zhang and Hutchinson, I explored only the one minute prediction task. The first hour of each day’s data was excluded from testing, for reasons indicated earlier.

The results are summarized in Table 3 for a variety of architectures. Here, the I-delay architecture with 0 hidden units—equivalent to a linear AR(6) model—performed significantly worse than the other architectures but there was no reliable difference among the other architectures.

Zhang and Hutchinson report a NMSE of .964 on the same data set, better than the .985 I obtained. However, it is important to emphasize that the two performance scores are *not* comparable. Zhang and Hutchinson threw out cases—from both training and test sets—for which a tickwise series values was not available 60 ± 30 seconds from the time at which a prediction was made. Hence, their predictions were *conditional on assumptions about the time of occurrence of*

future ticks. At the point at which a prediction is made, they cannot possibly know whether the assumptions are valid. Therefore, their approach cannot be used as a predictive model without a secondary component that predicts whether their prediction criteria will be satisfied. Alternatively, they could retrain and retest their network on all data, not just the data satisfying their criterion.⁴

Making such a conditional prediction is likely a simpler task than making an unconditional prediction, because it ensures some homogeneity among the cases. In effect, it throws out a large number of the “no change” predictions; in my test set, the fraction of “no change” future values was 71% versus only 41% in Zhang and Hutchinson’s set.

A perhaps more interesting measure of performance is whether the network can correctly predict the change in direction of the series: down, no change, or up. To explore this measure, it is necessary to quantize the network outputs, i.e., to specify a criterion, call it c^+ , such that all predictions greater than c^+ are classified as “up”, and a second criterion, c^- , such that all predictions less than c^- are classified as “down.” Predictions between c^- and c^+ are classified as “no change.” The principled approach to selecting c^- and c^+ involves selecting the values that yield best classification performance on the training set; this involves an exhaustive search through all possible values. Instead, I used a simple technique to determine c^+ : c^+ was chosen such that the fraction of training set predictions greater than c^+ equaled the fraction of training cases whose target prediction was “up”, and likewise for c^- . Using these values of c^- and c^+ on the test set, the 10-hidden I-delay network yielded a performance of 63.2% correct predictions. This sounds pretty good, but in reality is quite poor because 71% of all cases were “no change” and the net could have obtained this performance level simply by always responding “no change.” The network has a much greater ability to predict the direction of change *conditional upon a change having occurred*: It correctly predicts the direction on 58.5% of test cases, whereas always predicting the most frequent category in the training set obtains a performance of only 49.9%. If one is seriously interested in predicting direction of change, it seems sensible to train a network with three output units, one for each mutually exclusive prediction, perhaps using a normalized exponential output function (Bridle, 1990; Rumelhart, in press) to obtain a probability distribution over the alternative responses.

Conclusions

In this work, I have begun an exploration of alternative architectures for temporal sequence processing. In the difficult time series prediction problem studied, a nonlinear extension of autoregressive models—the basic I-delay architecture—fared no worse than a more complex architecture, the hybrid I-delay and TIS-0. However, it’s impossible to determine at present whether the prediction limitations I found are due to not having explored the right architecture, the right preprocessing or encoding of the input sequence, the right training methodology, or are simply due to there being no further structure in the data that can be extracted from the series alone without the aid of other financial indicators. The studies of Zhang and Hutchinson (this volume) of the same data series using an I-delay memory but with a different input/output encoding and training methodology yielded comparable results, supporting the hypothesis that the encoding and methodology is not responsible for the poor predictability. At very least, my results can serve as a baseline against which researchers can compare their more sophisticated architectures, representations, and training methodologies. I look forward to reports showing improvements over my work.

⁴In defense of Zhang and Hutchinson’s decision, they believed that the competition set up implied there would be a value near the prediction time, and thus it was to their advantage to make use of this information.

My aim in presenting a taxonomy of architectures is to suggest other possibilities in the space to be explored. I don't claim that the taxonomy is all encompassing or that it is necessarily the best way of dividing up the space of possibilities. However, it seems like a reasonable first attempt in characterizing a set of models for temporal processing commonly used in the field.

Acknowledgements

My thanks to Andreas Weigend and Neil Gershenfeld for organizing the Santa Fe Workshop, and in particular for Andreas for his abundant feedback at all stages of the work and suggestions for extensions. Jim Hutchinson, Radford Neal, Jürgen Schmidhuber, and Fu-Sheng Tsung, and an anonymous reviewer provided very helpful comments on an earlier draft of the chapter. This research was supported by NSF Presidential Young Investigator award IRI-9058450, grant 90-21 from the James S. McDonnell Foundation, and DEC external research grant 1250.

References

- Bachrach, J. (1988). Learning to represent state. Unpublished master's thesis, University of Massachusetts, Amherst.
- Ballard, D. H. (1986). Cortical connections and parallel processing: Structure and function. *The Behavioral and Brain Sciences*, 9, 67-120.
- Bengio, Y., De Mori, R., & Cardin, R. (1990). Speaker independent speech recognition with neural networks and speech knowledge. In D. S. Touretzky (Ed.), *Advances in neural network information processing systems II* (pp. 218-225). San Mateo, CA: Morgan Kaufmann.
- Bengio, Y., Frasconi, P., & Simard, P. (1993). The problem of learning long-term dependencies in recurrent networks. *Proceedings of the IEEE International Conference on Neural Networks*. To appear.
- Bodenhausen, U., & Waibel, A. (1991). The Tempo 2 algorithm: Adjusting time delays by supervised learning. In R. P. Lippmann, J. Moody, & D. S. Touretzky (Eds.), *Advances in neural information processing systems 3* (pp. 155-161). San Mateo, CA: Morgan Kaufmann.
- Bridle, J. (1990). Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. In D. S. Touretzky (Ed.), *Advances in neural information processing systems 2* (pp. 211-217). San Mateo, CA: Morgan Kaufmann.
- Curtiss, P., Kreider, J., & Brandemuehl, M. (1992). *Adaptive control of HVAC processes using predictive neural networks* (Technical Report). Boulder, CO: University of Colorado, Joint Center for Energy Management.
- de Vries, B., & Principe, J. C. (1991). A theory for neural networks with time delays. In R. P. Lippmann, J. Moody, & D. S. Touretzky (Eds.), *Advances in neural information processing systems 3* (pp. 162-168). San Mateo, CA: Morgan Kaufmann.